



**Silesian University
of Technology**

Faculty of Automatic Control, Electronics and Computer Science

ENSEMBLES OF SUPPORT VECTOR MACHINES WITH EVOLUTIONARILY OPTIMIZED HYPERPARAMETERS AND TRAINING SETS

This thesis has been submitted in partial fulfilment of the requirements
for the degree of Doctor of Philosophy

mgr inż. Wojciech Dudzik

Supervisor: **dr hab. inż. Michał Kawulok, prof. PŚ**

Co-supervisor: **dr hab. inż. Jakub Nalepa, prof. PŚ**

Gliwice, March 2023

Contents

1	Introduction	1
1.1	The Research Hypotheses	4
1.2	Published papers	5
1.3	Structure of this dissertation	5
2	Theory and literature review	9
2.1	Support vector machines	9
2.1.1	Hard margin SVMs	10
2.1.2	Non-linear SVMs	15
2.1.3	Kernel selection	18
2.1.4	Performance evaluation	20
2.2	Evolutionary algorithms	23
2.3	Literature review	26
2.3.1	Optimization of SVM hyperparameters	27
2.3.2	Selection of SVM training set	36
2.3.3	Feature selection	43
2.3.4	Multi-aspect SVM optimization	45
2.3.5	Building classification ensembles	46
2.3.6	Summary	48
3	Proposed methods	51
3.1	A genetic algorithm for optimizing SVM hyperparameters (GAHP)	52
3.2	Evolutionary algorithm for feature selection (EFS)	57
3.3	Alternating algorithm for optimization of SVM hyperparameters and training set selection	60

3.4	Simultaneous optimization of training and feature set and SVM hyperparameters	62
3.5	Adaptive RBF kernel	66
3.6	Building ensembles	79
3.7	Summary	85
4	Experimental validation	87
4.1	Datasets	87
4.2	Experimental setup	93
4.3	Results	93
4.3.1	Visualizations of principles	94
4.3.2	Comparison of proposed algorithms	105
4.3.3	Comparison with other methods	106
5	Conclusions	117
5.1	Future work	119
	Appendix	XXV
.1	Recursive features elimination	XXV
	List of symbols	XXVII

Chapter 1

Introduction

Nowadays, the field of machine learning and artificial intelligence (AI) is experiencing significant growth and is considered to be one of the most rapidly evolving fields in computer science. The advent of large-scale data collection and storage, coupled with advancements in computational power, has made it possible to develop machine learning models that are capable of extracting meaningful insights from complex data. As a result, machine learning is now widely used across a range of domains, from healthcare [107] and finance [35] to transportation [138] and entertainment [54]. Despite the significant progress that has been made in the field, the use of these models can still pose challenges to both experienced and inexperienced users. One of the major issues emerges from the large-scale datasets that are often used nowadays. Such datasets can be extremely complex and can be hard to utilize due to high computational costs. As a result, methods for optimizing machine learning models need to be designed to scale to large datasets while maintaining their accuracy and efficiency. Moreover, with a large number of different methods available for performing those tasks, it can be difficult for users to select the most appropriate method for their specific needs.

Classification is one of the fundamental problems in the field of machine learning. It involves the task of predicting the class labels of new data points (samples) based on a model learned on the training dataset of labeled examples. It is a crucial problem in many domains, such as healthcare [130, 102], finance [21], and security [3], where the ability to accurately and efficiently classify samples can provide

significant benefits. The classification can be divided into three categories based on the number of classes. There is the one-class classification that tries to identify objects of a specific class among all others. If there are only two classes, the problem is called binary classification while for any larger number, it is described as multiclass classification. Through the years multiple methods were developed to tackle these problems. It leads to the situation where there is a significant number of available optimization techniques and methods to use, including, but not limited to random forest, k -nearest neighbors (k -NN), artificial neural networks (ANN), and support vector machines (SVMs), can make it difficult for users to determine which approach is best suited for their particular data set. Furthermore, joining multiple models into an ensemble is a widely used technique, which can help to improve the accuracy and robustness of the classifier. The rationale behind ensemble methods is that by combining the predictions of multiple models, we can reduce the variance and bias of the resulting classifier, and thus improve its generalization performance. It, however, brings even more difficulties as there are many different ways of building those ensembles, spanning across bagging [83], stacking [132], and boosting [45] to name the most popular ones. While the problem of appropriate hyperparameters' selection and training of individual models in ensembles still remains to be solved.

SVMs are one of the popular methods for performing binary classification. They work by finding the hyperplane that separates the two classes of data points while maximizing the margin between the hyperplane and closest data points. Those closest points are often selected as support vectors that determine the decision boundary (hyperplane) of the SVM classifier and are crucial for the classification process of new data points. This approach leads to a robust and well-generalizing classifier, which can handle high-dimensional and noisy data [68]. Because of that, they are widely used for binary classification, with a proven track record of high accuracy and robustness in various domains, including bioinformatics [63], finance [78], and engineering [34]. Thanks to that, they are still relevant today in spite of the popularity of deep neural networks (DNNs) and continue to play a critical role in the field of machine learning. While DNNs has revolutionized several fields, including image recognition [27], speech recognition [128], and natural language processing [20], they also have certain limitations. There are new works

that propose to join DNNs capabilities in automatic feature extraction with the robustness of SVMs [8] which can provide state-of-the-art results in various domains. However, the major drawback of such solutions is that the SVMs can be computationally expensive to train. This process is known to have time and memory complexities of $O(t^3)$ and $O(t^2)$, respectively, where t is the size of the training set [104]. What is more, the choice of a kernel function can have a significant impact on the performance of the classifier. Hence combining SVMs with DNNs is difficult as DNNs require vast amounts of data for training which tends to produce enormous datasets that could not be handled by regular SVM classifiers. There are indeed numerous approaches toward improving the scaling properties of SVM. These are among other methods that accelerate the training process by better utilizing hardware or providing an implementation that uses the power of graphics processing unit (GPU) [131]. Other works focus on limiting the size of the training set [97] by selecting the most promising vectors hence accelerating the training process and lowering memory requirements. Although the great successes shown in many papers, the majority of these methods ignore the problem of hyperparameters optimization which is crucial in order to get a good classifier. What is more, many of these method require technical knowledge and proper expertise before use.

Despite the growing popularity of SVMs in machine learning, the optimization of these models remains a complex and challenging task. The large size of datasets often necessitates lengthy computations and requires a vast amount of memory, which can impede the effective training of SVMs in practical applications. In addition, the problem of hyperparameter optimization and the selection of relevant features are often omitted or addressed independently of each other which may negatively affect the performance of the SVM classifier. This dissertation aims to propose effective solutions for optimizing SVM models in the context of binary classification. These solutions will be designed to overcome the challenges posed by large datasets and provide efficient approaches for hyperparameter optimization, training set and feature selection problems simultaneously. Furthermore, the dissertation will introduce new methods for constructing ensembles of SVM models to enhance their performance and extend their capabilities with special attention put on keeping the training process fast. In order to achieve all of the goals evol-

utionary computations will be used which already proved to be a robust solution to many problems while some examples of such techniques were already effectively utilized to optimize SVMs. This work will build upon the success of those methods and will improve them. By developing these new techniques, this dissertation seeks to advance the state-of-the-art SVM optimization.

In the next part of this chapter, the research hypotheses are stated. Then, Section 1.2 provides a summary of papers that are published or are under review which contain the methods described in this dissertation. These methods have already been published by the author in proceedings of international conferences and one in a peer-reviewed journal. Finally, the last section provides a roadmap to the remaining part of this dissertation.

1.1 The Research Hypotheses

There are two research hypotheses formulated for this work:

1. Simultaneous optimization of the training set, the feature set and the hyperparameters of the SVM improve training and classification time compared to other state-of-the-art methods proposed for this purpose without affecting the classification quality.
2. Building ensembles of SVM using evolutionary algorithms provide improved classification performance compared to other SVM ensemble building algorithms and other well-established methods.

To clarify what is meant by “other methods proposed for this purpose” in first hypothesis—these methods will comprise of grid search and optimization algorithms for SVM hyperparameters, and training set selection methods described in existing literature. It is understood that these methods will yield comparable classification metric values to the proposed algorithm without any detrimental effects on classification quality (classification metrics used to test the algorithm will be on similar level to those other methods). The second hypothesis aims to measure the improvement in classification performance using accuracy, F1-score, and MCC score (as described in Section 2.1.4) on several benchmark datasets. Other aspects such as training and classification time will also be analyzed.

1.2 Published papers

Parts of the work presented in this dissertation have already been published in international conferences and in peer-reviewed journals. Table 1.1 shows all of the published papers.

Table 1.1: The list of proposed algorithms that are already published in international conferences and peer-reviewed journals.

Algorithm	Acronym	Reference
AL ternating G enetic A lgorithm for selecting SVM training sets and models	ALGA	[75]
AL ternating M emetic A lgorithm for selecting SVM training sets and models with F eature S election	FSALMA	[38]
E volutionarily-tuned SVMs	ESVM	[36]
S imultaneously- E volved SVMs	SE-SVM	[37]
SVMs with A daptive RBF kernels	ARBF-SVM	[93]
D ata- A daptive SVM	DA-SVM	[39]
C ascades of E volutionarily optimized SVMs	CE-SVM	[40]

It is important to note that the publications listed in Table 1.1 are not the only papers co-authored by the author of this dissertation. Rather, these works represent key contributions that led to the development of algorithms for optimizing SVMs. Any figures that have already been published in one of these works will be clearly indicated in the figure's caption, e.g., "This figure comes from our paper [36]".

1.3 Structure of this dissertation

This work is organized as follows. Chapter 2 provides the background concerning the theory behind SVMs. Here, the importance of hyperparameters in SVM is discussed and presented using visual examples of 2D datasets. Additionally, a

sub-section of the chapter addresses the challenge of evaluating the performance of binary classifiers. The second part of the chapter undertakes a comprehensive review of the existing literature on SVMs. The review is presented in subsections based on the optimization aspects of SVM, including the SVM model, training set, and feature set. In addition, the process of building ensembles, a technique that involves combining several classifiers to enhance performance, is analyzed. Toward the end of the chapter, a summary of all the methods discussed is presented, and their relevance in the current research context is highlighted. This summary consolidates the various approaches to SVM and helps better contextualize the presented work within the broader context of the current research.

Chapter 3 is dedicated to the presentation and analysis of newly proposed solutions. The chapter provides a detailed description of each of the algorithms, outlining their individual features and functionalities. The objective of this chapter is to offer a comprehensive understanding of the solutions, including the rationale behind their design and implementation. Finally, a short summary is presented, which compares the algorithms. This comparison provides readers with a better overview of the provided solutions and offers a basis for understanding the relevance of each of the solutions in addressing the research problem.

Chapter 4 presents an experimental study aimed at assessing the performance of the methods discussed in Chapter 3. The chapter offers a detailed description of the datasets used in the experiments and outlines the configurations used for the algorithms. The objective of this chapter is to provide a thorough analysis of the proposed algorithms and to evaluate their performance. These new algorithms are subjected to a quantitative analysis using 2D artificially created datasets. This analysis is aimed at examining the behavior of the algorithms and enables to compare them visually. Furthermore, the performance comparison of the proposed methods against other state-of-the-art optimization techniques and popular classifiers, using benchmark datasets is presented. This comparison is critical in evaluating the effectiveness of the new methods and in determining their contributions to the field. The statistical significance of the results is also discussed, ensuring that the conclusions drawn from the study are based on reliable evidence.

Chapter 5 of this dissertation summarizes the finding and provides the conclusions drawn from the presented data. This chapter encapsulates the key takeaways

from the study, highlighting the significance of the findings and how they contribute to the existing knowledge in the field. Furthermore, the chapter explores potential avenues for future research, sheds light on the limitations of the study, and identifies areas that require further exploration.

Chapter 2

Theory and literature review

In this chapter, the theory behind SVMs is introduced. Hard margin and soft margin SVMs are discussed (in Section 2.1.1), which focus on linear SVMs. The non-linear SVMs (using kernel functions) are presented separately in Section 2.1.2. The significance of using the appropriate kernel along with its hyperparameters is discussed in Section 2.1.3. The examples showing the impact of those kernels will be presented with 2D datasets and visualized. Later in the chapter, the metrics used to evaluate the performance of SVMs are presented (in Section 2.1.4). This is followed by a brief introduction to the evolutionary computations which are used in all of the proposed methods. This concludes the first part of this chapter. The second part of this chapter is dedicated to the literature review (Section 2.3). This review is divided into multiple sections, concerning the methods for SVM hyperparameters optimization are discussed (Section 2.3.1) followed by methods for training set selection (Section 2.3.2) and feature selection (Section 2.3.3). Afterward, methods that couple any of those optimizations together are discussed in Section 2.3.4. The chapter is concluded with the review of the approaches toward buildings ensembles with the SVMs in Section 2.3.5.

2.1 Support vector machines

SVM is a binary supervised classification algorithm. It was introduced by V. Vapnik et al. in 1992 [19] and later extended for regression tasks. The fundamental

idea of SVM is to find the optimal separating hyperplane between the two classes that maximizes the margin between the closest data points from each class.

Let us define a set $\mathbf{T}$ that consists of t training vectors $x_i \in \mathbb{R}^D, i = 1, \dots, t$, where each vector belongs to one of two classes $y_i \in \{-1, +1\}$ (binary classification problem). We are searching for the function $f(x)$ that separates those vectors into two classes. This function can be defined as:

$$\begin{aligned} f(x_i) &> 0, \forall y_i = 1, \\ f(x_i) &< 0, \forall y_i = -1. \end{aligned} \tag{2.1}$$

First, the problem of determining $f(x)$ on linearly separable data will be considered.

2.1.1 Hard margin SVMs

Linear SVM classifier aims to separate the data in the D -dimensional input space using a hyperplane decision boundary defined as:

$$f(x) : \mathbf{w}^T x + b = 0, \tag{2.2}$$

where w is a hyperplane normal vector $w \in \mathbb{R}^D$, and b is an offset, $b \in \mathbb{R}$. This hyperplane Equation needs to satisfy the conditions defined in Equation 2.1. There might be many hyperplanes equations that will satisfy those conditions. To determine an unambiguous solution, this decision hyperplane is positioned such that the distance between two vectors from opposite classes is maximal (with respect to the hyperplane). Considering data that is linearly separable and contains only two classes ($y_i \in \{-1, +1\}$), training data need to satisfy the following conditions:

$$\begin{aligned} w^T x_i + b &\geq 1, y_i = +1, \\ w^T x_i + b &\leq -1, y_i = -1, \end{aligned} \tag{2.3}$$

which can also be written in a single Equation as:

$$y_i(w^T x_i + b) - 1 \geq 0, y_i \in -1, +1. \tag{2.4}$$

The inequalities shown in Equation 2.3 and Equation 2.4 present two parallel

hyperplanes (i.e. hyperplanes with the same normal vector) which are based on vectors that satisfy following equation:

$$\begin{aligned} w^T x_i + b &= 1, \\ w^T x_i + b &= -1. \end{aligned} \tag{2.5}$$

These vectors are called support vectors (SVs). The distance to the origin is expressed as $|1 - b|/\|w\|$ and $|-1 - b|/\|w\|$ respectively, where $\|w\|$ is the second norm of a vector (called also Euclidean norm). Those planes are presented with dotted lines in Figure 2.1. Please note that with the above definition, there are no vectors in space between those hyperplanes. What is more, the shortest distance to any data vector (from either class) is $1/\|w\|$, hence the maximal margin (m) is equal to:

$$m = \frac{2}{\|w\|}. \tag{2.6}$$

The example of such linear SVM is presented in Figure 2.1b. Moreover, if any of the selected SVs in Figure 2.1b is changed to another vector from the training set, the calculated solution (hyperplane) will be different.

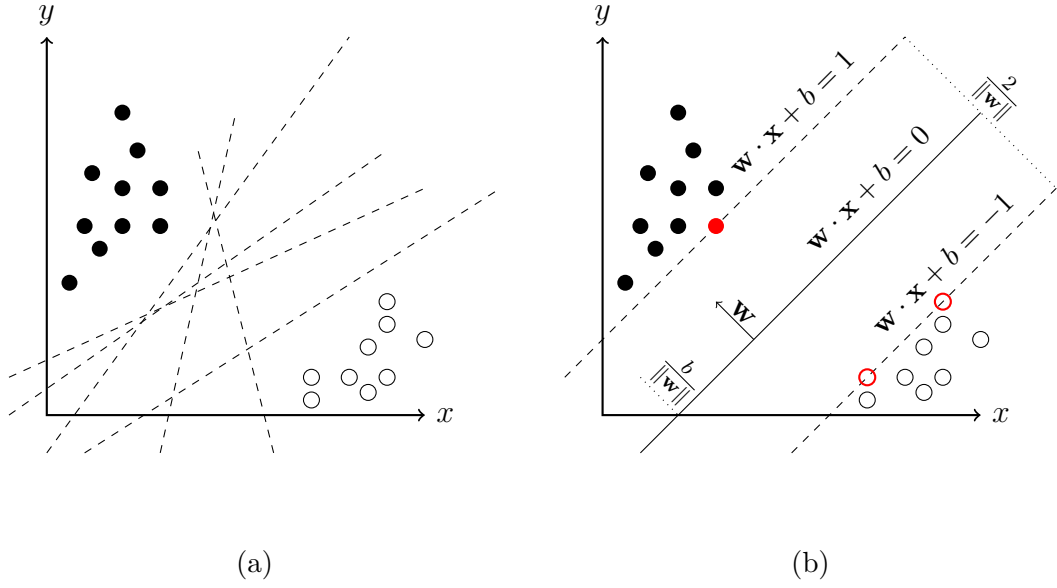


Figure 2.1: (a) An illustrative example of simple 2D datasets with two classes (black and white dots) where multiple separating hyperplanes are denoted with dotted lines. (b) An example of a linear SVM trained on the presented data. The support vectors are denoted with red color, while the decision boundary is presented with a solid line.

As the goal is to maximize the separating margin, the value of $\|w\|$ needs to be minimized:

$$\min_{w,b} \|w\|. \quad (2.7)$$

To simplify the calculation, we can rewrite Equation 2.7 as the quadratic term:

$$\min_{w,b} \frac{\|w\|^2}{2}. \quad (2.8)$$

While solving the optimization problem, we need to respect the constraints from Equation 2.3. Therefore, this becomes a quadratic programming problem (QP). The solution obtained by solving such formulation (called also primal form) is a hyperplane defined by the normal vector which is retrieved from QP. This hyperplane is then used to classify incoming data based on the following decision function:

$$f(a) = \text{sgn}(w^T a + b), \quad (2.9)$$

where a is a data vector.

Equation 2.3 can be re-written to obtain the Lagrangian in its primal form:

$$\mathcal{L}(w, b, \alpha) = \frac{\|w\|^2}{2} - \sum_{i=1}^t \alpha_i y_i (w^T x_i + b) + \sum_{i=1}^t \alpha_i, \quad (2.10)$$

where α_i are the Lagrange multipliers. The advantage of such formulation is the fact that all of the training and test data will appear in the form of a dot product between vectors. This is a vital feature that will allow the expansion of the above formulation for non-linear cases. Finding the hyperplane for SVMs is equivalent to finding the solution to the Karush-Kuhn-Tucker (KKT) conditions. The Lagrangian in the dual form, also known as Wolfe dual, can be written by employing those conditions to obtain the following form:

$$\mathcal{L}_D(\alpha) = \sum_{i=1}^t \alpha_i - \frac{1}{2} \sum_{i=1}^t \sum_{j=1}^t \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j, \quad (2.11)$$

under the following constraints:

$$\sum_{i=1}^t \alpha_i y_i x_i = 0, \alpha_i \geq 0. \quad (2.12)$$

The weight vector of the original problem is expressed as:

$$w = \sum_{i=1}^t \alpha_i y_i x_i. \quad (2.13)$$

The decision function from Equation 2.9 can be obtained by applying Equation 2.13:

$$f(a) = \text{sgn}\left(\sum_{i=1}^t \alpha_i y_i x_i^T a + b\right). \quad (2.14)$$

Soft margin SVMs

In 1995, a method for learning SVMs with a weak margin was published [32]. It is a modification of the SVM classifier that allows the determination of a decision function for non-linearly separable data (see Figure 2.2). Such data often occur in real-world problems due to, among other things, noisy readings or incorrect labeling of training data. Otherwise trying to train hard-margin SVM on noisy data will complicate the boundary decision boundary function, as well as degrade the quality of the classification itself. In order to be able to apply inequalities 2.3, an additional variable must be introduced, which is the cost of the vector misclassification operation:

$$\begin{aligned} w^T x_i + b &\geq 1 - \xi_i, y_i = +1, \\ w^T x_i + b &\leq -1 + \xi_i, y_i = -1, \\ \xi_i &\geq 0, \end{aligned} \quad (2.15)$$

where ξ_i is a slack variable. By taking the slack variable into consideration in the objective function, it can be updated as follow:

$$\min_{w,b,\xi} \frac{\|w\|^2}{2} + C \sum_{i=1}^t \xi_i, \quad (2.16)$$

under the following constraints:

$$\begin{aligned} y_i(w^T x_i + b) &\geq 1 - \xi_i, i = 1, \dots, t, \\ \xi_i &\geq 0, i = 1, \dots, t, \end{aligned} \quad (2.17)$$

where C is the parameter that controls compromise between the margin and the slack penalty. The larger values of C increase the penalty for errors. This new parameter allows for presenting the concept of soft-margin SVMs. Similarly, as in the previous case, the Equation 2.11 can be re-written for soft margin SVM:

$$\mathcal{L}_D(\alpha) = \sum_{i=1}^t \alpha_i - \frac{1}{2} \sum_{i=1}^t \sum_{j=1}^t \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (2.18)$$

is to be maximized subject to

$$\sum_{i=1}^t \alpha_i y_i = 0, \quad (2.19)$$

$$0 \leq \alpha_i \leq C.$$

The weight vector w is the same as in Equation 2.13. In comparison to the previous problem statement, the α_i parameters are additionally bounded by the C value. For very large values of C , this formulation is close to the hard margin SVM in practice. To better illustrate the concept situation of misclassification of a single vector is presented in Figure 2.2. One of the white vectors is on the “wrong” side of the decision hyperplane, therefore, there is a penalty to be accounted for. For every i -th vector, this penalty is equal to $d = \frac{\xi_i}{|w|}$.

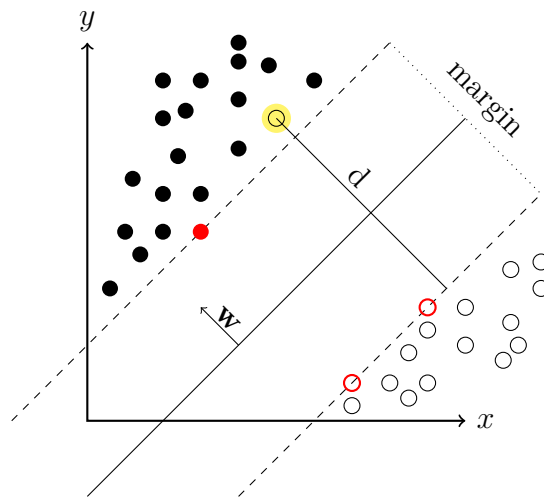


Figure 2.2: An example of the soft-margin linear SVM. The misclassified example is denoted with a yellow circle.

2.1.2 Non-linear SVMs

While the above description of linear SVMs can be used for classification, there are still many real-life problems where a non-linear decision function is required. For this reason, the *kernel trick* was introduced to provide a non-linear separating hyperplane in SVMs [32]. It includes defining a kernel function which is computed

as a dot product of two vectors:

$$K(a_i, a_j) = \phi(a_i)^T \phi(a_j), \quad (2.20)$$

where a denotes the sample (vector) from the input space, and $\phi : \mathbb{R} \rightarrow \mathbb{F}$ is a function that maps the data from the original D -dimensional space into a higher-dimensional space $\mathbb{F}$ and $K : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$. The kernel function needs to satisfy the Mercer Theorem [61]. This allow to write Equation 2.11 as:

$$\mathcal{L}_D(\alpha) = \sum_{i=1}^t \alpha_i - \frac{1}{2} \sum_{i=1}^t \sum_{j=1}^t \alpha_i \alpha_j y_i y_j \mathbf{K}(\mathbf{x}_i^T, \mathbf{x}_j), \quad (2.21)$$

thus the decision function used for classification can be written as:

$$f(a) = \text{sgn}(\sum_{i=1}^t \alpha_i y_i \mathbf{K}(\mathbf{x}_i^T, \mathbf{a}) + b). \quad (2.22)$$

When the kernel function satisfies Mercer conditions, it can be proven after [32] that the dot product is defined in a feature space. Because of that, there is no need to calculate ϕ mapping explicitly. Therefore, kernel trick allows the SVM to operate in the original, lower-dimensional space while still taking advantage of the separation of the data in the higher-dimensional space. It is worth mentioning that in order to classify new vectors there is only a need to calculate the values of a kernel function between support vectors (SVs) and new data sample.

There are several types of kernel functions, including linear, polynomial, and radial basis function (RBF) kernels. The choice of a kernel function will depend on the characteristics of the data and the specific requirements of the task at hand. While this work focuses on RBF and linear kernels (being the most popular ones), there are many more kernels of which a few examples are shown in Table 2.1:

Table 2.1: Popular kernel functions.

Name	Function
Linear	$K(x_i, x_j) = x_i^T x_j$
Polynomial	$K(x_i, x_j) = (x_i^T x_j + g)^d$
Radial Basis Function (RBF)	$K(x_i, x_j) = e^{-\gamma \ x_i - x_j\ ^2}, \gamma > 0$
Sigmoid	$K(x_i, x_j) = \tanh(x_i^T x_j + g)$

The example of the mentioned mapping and learnt hyperplane is presented in Figure 2.3 and 2.4. The image in Figure 2.3 presents the data that cannot be separated linearly. Hence, a proper kernel trick needs to be used in order to effectively apply an SVM to this problem. After learning a hyperplane, which is presented with the solid blue line, between classes can be obtained. In Figure 2.4 is an example of possible mapping the 2D data into 3D space by using the following formula to obtain the third dimension of the data $z = x^2 + y^2$. After applying that mapping, a linear SVM could be trained which will result in the presented hyperplane (a blue plane). Thank to the kernel trick this explicit mapping is not necessary and by applying the proper kernel function the same result could be obtained.

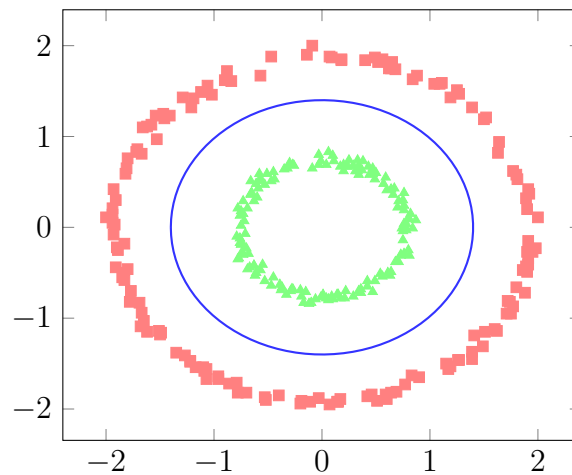


Figure 2.3: An example of data that is non-linearly separable. Green triangles present a positive class while red squares present a negative class. The hyperplane is rendered with a blue solid line.

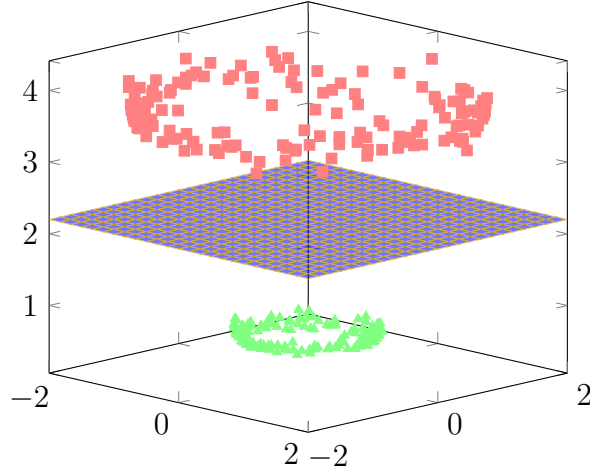


Figure 2.4: One of possible mapping of data from Figure 2.3 into 3D space. Here the decision boundary is seen as a blue 3D plane.

2.1.3 Kernel selection

In the context of SVMs, the kernel function plays a crucial role in the performance of the model. As mentioned previously, the kernel function is used to map the input data into a higher-dimensional feature space, where it becomes possible to find a hyperplane that can linearly separate the data. The choice of the kernel function can significantly impact the model’s accuracy, as well as the computational efficiency of the training process [53]. One problem with selecting the kernel function for an SVM is that there is no general way to determine the “best” kernel function for a given dataset. Different kernel functions may perform better on different types of data, and it is often necessary to try out a number of different kernel functions in order to find the one that works best for a particular dataset. This can be time-consuming and may require significant experimentation [9]. Another issue with kernel function selection is that it can be difficult to determine the appropriate kernel function for a given dataset without a strong understanding of the characteristics of the data and the underlying relationships between the data points. Moreover, many kernels come with a set of hyperparameters which need to be tuned. This often requires performing cross-validation which also requires time

and computing power. This can make it challenging for researchers to effectively use SVMs, particularly if they work with large or complex datasets.

In the following examples, there is the presentation of how selecting a proper kernel function as well as its hyperparameters can affect the final decision hyperplane. For the purpose of better understanding, all of the results of SVMs and their decision boundaries will be visualized on artificially created 2D datasets. First, the importance of C hyperparameter with a linear SVM is presented in Figure 2.5. Large values of C will result in a smaller-margin hyperplane to avoid any misclassifications. Conversely, a very small value of C will cause a larger margin separating the hyperplane, even if that hyperplane misclassifies more points. This effect can be seen in Figure 2.5, where increasing of the C width of the margin is getting smaller which results in a smaller number of support vectors as they are not lying within this margin and effectively do not influence decision function. In this example, the value of C does not affect the generalization performance of an SVM but it is visible that the selection of the proper value can greatly affect the resulting SVM model. In this case, it impacts the number of SVs hence the classification speed will differ significantly between tested values.

The second example presented in Figure 2.6 compares different SVM kernels and their hyperparameters. The first row presents the example of a 2D dataset (please note that dots were enlarged in the visualization of the dataset) and two very similar solutions obtained by the polynomial and RBF kernel. What can be noticed is that in both of those examples, there are very few SVs present. On the other hand, the second row provides examples of poor classification (in comparison to the best solutions obtained). Starting from the left side, the linear kernel is inappropriate for this kind of data as it requires a non-linear decision boundary. The RBF kernel with wrongly optimized γ and C values drastically increases the number of SVs while achieving an accuracy of 0.94. As the last example, the polynomial kernel with $d = 2$ and $C = 0.001$ provides even worse classification than the linear kernel. These are only few examples taken from optimizing those sets of hyperparameters for a given kernel using the grid search algorithm. Even in the case of a fairly simple 2D dataset (where perfect classification is easy to achieve), the values of hyperparameters and kernel selection are crucial in order to achieve good classification performance.

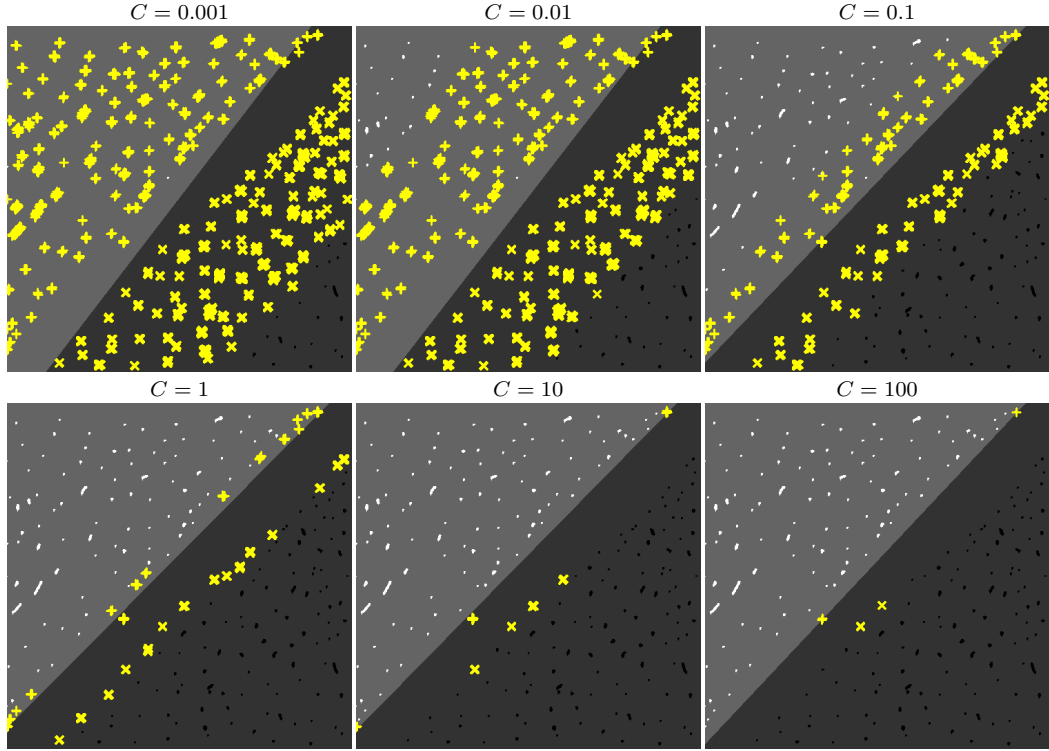


Figure 2.5: Comparison of different C hyperparameter setting for the linear kernel. Yellow crosses denote SVs, whereas dark and light gray denote values of decision function – the boundary between them is the decision hyperplane.

2.1.4 Performance evaluation

A typical way to assess the performance of SVMs (as well as other binary classifiers) is to calculate the confusion matrix, presented in Table 2.2 and derive various metrics from it. There are four numbers that create such a matrix (1) the number of correctly classified vectors from the positive class called true positive (TP), (2) the number of correctly classified vectors from the negative class called true negative (TN), (3) the number of incorrectly classified vectors from the positive class called false negatives (FN) also known as type II error and finally (4) the number of the incorrectly classified vectors from the negative class called false positive (FP) also known as type I error.

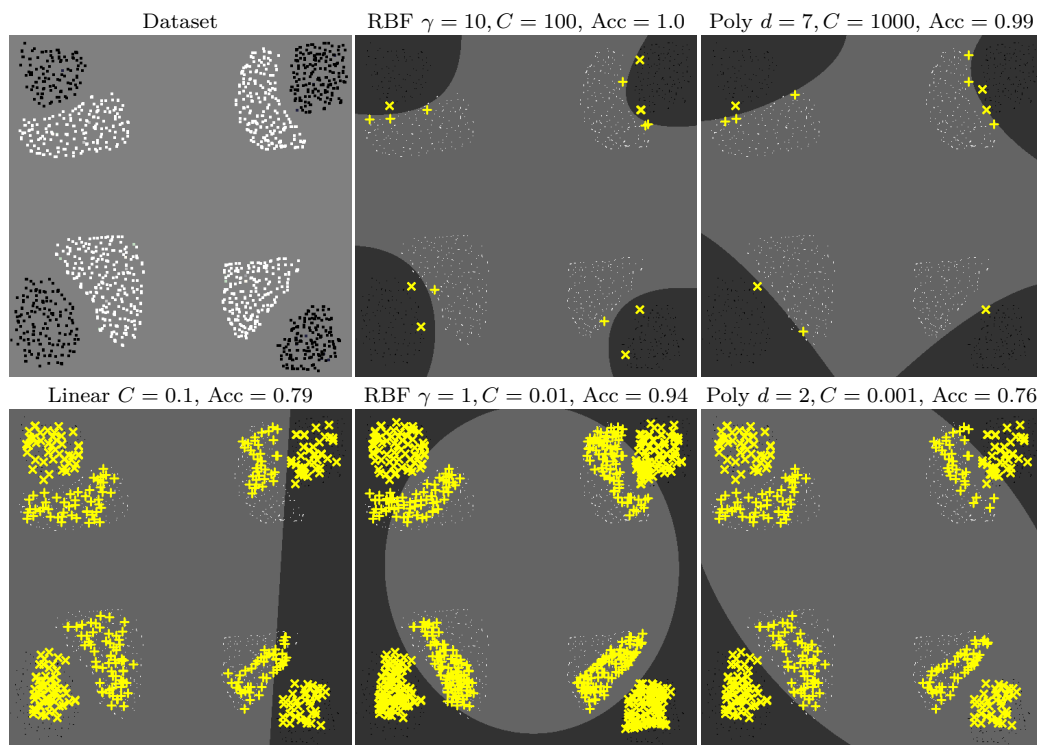


Figure 2.6: Comparison of different SVM kernels (linear, RBF and **polynomial**) and their hyperparameters. Yellow crosses denote SVs, whereas dark and light gray denote values of decision function – the boundary between them is the decision hyperplane. Above each example, the kernel with a set of hyperparameters is given together with the accuracy of the model.

Table 2.2: Visualization of confusion matrix, where green cells represent correct classification and red cell represents different types of errors.

Predicted →	Positive	Negative
Actual ↓		
Positive	True positive	False negative
Negative	False Positive	True negative

Multiple metrics can be derived using those four numbers, such as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad (2.23)$$

$$Precision = \frac{TP}{TP + FP}, \quad (2.24)$$

$$Recall = \frac{TP}{TP + FN}, \quad (2.25)$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot TP}{2 \cdot TP + FN + FP}, \quad (2.26)$$

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (2.27)$$

In this dissertation, all of those values will be presented to compare the performance of SVMs. While all of the above metrics can describe performance, not all of them can be a good choice, e.g., for imbalanced data, it can be easy to achieve very high accuracy by always predicting the same (majority) class. For such cases, F1 score is a better choice for imbalanced data. Although in the case of the F1 score selecting a minority class as a positive one is still important. Despite the crucial importance of selecting proper performance metrics in machine learning, there is no widespread consensus on unified measures. Lately, the Matthews correlation coefficient (MCC) was proposed as being more reliable than accuracy or F1 [29, 30]. One more important aspect to consider is the decision threshold. Naturally, SVMs decide to which class an example belongs based on the sign of the decision function. This value could be adjusted based on the receiver operating characteristic (ROC) curve [43]. This process of threshold adjustment is important when SVM is trained using the training set selection procedure. It may seem counterintuitive at first, as there is a free term b (see Equation 2.22) which is found during the optimization process. However, when only a subset of the training data is used to derive a model of an SVM a part of the information about data characteristics is lost. Due to that adjusting threshold based on the full training set or validation set can boost the SVM performance. The training set selection tends to make SVM training more feasible for large datasets but can negatively affect its final performance. Therefore, the selection of an additional threshold based on

ROC can impact performance positively as the learned term b cannot know the full characteristic of underlying data. An argument to use the area under the ROC curve (AUC) as the performance metric can be made as it does not require setting certain thresholds but it becomes harder to use in an ensemble setting which is also studied in this work. These difficulties include selecting micro or macro averages of AUC of models that are included in the ensemble which have a different interpretation and their selection depends on the dataset. What is more, the presented method (in Section 3.6) build a cascade of SVMs where it is not trivial to define the AUC metric and its interpretation. That is why for building ensembles MCC will be used as a primary metric (although as presented in Section 3.6, the fitness function for evolutionary computation does not use MCC at all).

Moreover, as a time performance measure for SVM the number of SVs can be retrieved, as the classification time depends linearly on the number of SVs. Also, the training time of the classifier could be monitored. Although measuring the training time depends heavily on the implementation of the training algorithm and its usage of advanced CPU features as well as usage of GPU or memory layout of data used during the training. The observed timing differences could be on the order of hundreds of times [131]. That is why training times will be should be compared mainly among proposed algorithms and not directly compared to others methods as there is a great possibility for non-informative or misleading results. To sum up, non-functional aspects of SVMs that can be measured include:

1. number of SVs,
2. classification time,
3. training time.

2.2 Evolutionary algorithms

Evolutionary algorithms (EAs) are a class of optimization techniques that are inspired by the process of natural selection. These algorithms are used to solve a wide range of problems from different fields such as economics, management and engineering [73]. They are particularly well suited for complex problems or ones

with large search spaces that are too difficult to solve using traditional optimization methods, or for which there is no known algorithm [88]. There are multiple branches of EA such as genetic algorithms (GAs), evolution strategies, evolutionary programming, differential evolution, or neuroevolution.

Genetic algorithms (GAs) work by iteratively improving a population of candidate solutions to a problem, through the application of a set of rules that mimic the process of natural selection. Each element in such a population is called a chromosome which is the encoded representation of a solution. Those chromosomes are evolved using specific genetic operators. In the basic GA, those operators include crossover and mutation. The crossover is the process of combining two solutions to create a new solution. It may also involve additional checks if the new solution is feasible. The mutation consists of randomly altering a solution to diversify the search. After such alteration, the fitness value of each individual is assessed and it can be either minimized or maximized depending on the GA design and problem at hand. Later fitness values are used to select individuals for the next generation and creation of new population. Most often selection process keeps the best solution while the rest of the population can be selected in different ways. One of the popular choices is to use a strategy that keeps the N best individuals to maintain the constant size of the population. This process is continued until the stop condition is met. The termination condition is an important step in designing the algorithm and it may include:

- checking the fitness of the best individual in the population (whether the individual fitness is satisfying threshold set before running GA),
- checking the number of generations evolved or time of execution,
- checking the population characteristic (e.g., diversity, if average fitness of population has changed compared to the last generation),
- or checking combinations of the above conditions.

The pseudocode of a general GA is presented in Algorithm 1.

When designing the genetic algorithm the choice of a proper chromosome structure (encoding) and fitness metric is crucial. These are often problem dependent

Algorithm 1 A general scheme of a genetic algorithm (GA).

```

1: Initialize population  $P_a$ 
2: Calculate the fitness of each chromosome in  $P_a$ 
3: while stop condition not met do
4:   Select pairs of chromosomes  $(p_a, p_b)$  from  $P_a$ 
5:   for all pairs  $(p_a, p_b)$  do
6:      $p_c = \text{crossover}(p_a, p_b)$ 
7:      $p_c = \text{mutation}(p_c)$ 
8:     Calculate fitness of  $p_c$ 
9:     Insert  $p_c$  into  $P_b$ 
10:   $P_a = \text{Selection}(P_a, P_b)$ 
11: return fittest chromosome in  $P_a$ 

```

parts of GAs. Representations of an individual are chosen so that its set of genes reflects the solution to the problem. For example, in the travelling salesman problem (TSP), a good representation would be an ordered list of towns to visit [129]. Since the GAs operate on a population of solutions, the search starts from many points scattered across the solution space. This makes GA less likely to get the solution stuck in local minima. It is one of the significant advantages of GAs compared to other methods that operate on a single solution. The fitness function has the task of calculating how close a particular individual is to the optimal (sought) solution. Again referring to the TSP, the evaluation function can be defined as the sum of the distances between the cities on the list.

Despite their successes in solving a wide range of various optimization problems [89], evolutionary algorithms do have some limitations. One of the main challenges with these algorithms is their computational complexity. Evolutionary algorithms typically require a large number of function evaluations in order to find good solutions. In addition, evolutionary algorithms may not always find the global optimum solution to a problem and may be sensitive to the initialization of the population [112].

Another problem often encountered in literature is proving the effectiveness of GAs in an analytical way. The main reason for this difficulty is the “No free lunch” theorem [133], stating that there is no optimization algorithm that will be general and simultaneously optimal for solving any problem. For this reason, if no

assumptions are made about the objective function f , it is difficult to prove the superiority of evolutionary methods over other methods, e.g., random search. A schema theorem was proposed [62] which says that short, low-order schemata (a set of few genes) with above-average fitness increase exponentially in frequency in successive generations. This means that if there are some genes that present better solutions they should be quickly replicated to other chromosomes in the population. The problem is that this theorem holds under the assumption of GA maintaining an infinitely large population and does not always carry over to populations of the defined (and limited) size used in practice. Due to sampling errors GAs may converge on schemata that have no selective advantage. In practice, empirical evidence is accepted as proof of genetic algorithms' advantage over other methods [88].

One more important addition (and improvement) to GAs is the use of local search or other refinement procedures. Such an algorithm is called a memetic algorithm (MA) or a hybrid GA [91]. This improvement over the presented general GA structure provides a better balance between generality and problem specificity. It also addresses the “No free lunch” theorem since MA is tailored for the problem with additional refinement procedures. It can be applied to all of the individuals in the population, as well as try to select the most promising chromosomes. As it tries to improve the search result, it also balances the exploration versus exploitation of promising regions of the solution space and helps to avoid premature convergence and getting stuck in local minima. Due to their advantages, MAs have been proposed for a wide range of applications such as machine learning [92], bioinformatics and medicine [12], engineering [46, 84] and combinatorial optimization [90, 120].

2.3 Literature review

In the following section, the state-of-the-art methods for SVM hyperparameters optimization and the training set, and feature set selection will be reviewed. Afterwards, the literature regarding the optimization of multiple aspects will be analyzed.

2.3.1 Optimization of SVM hyperparameters

Optimization of SVM hyperparameters is an active topic of research. In more general terms, it is a part of various automatic hyperparameter optimizations (HPOs). These include simple techniques such as grid and random search as well as more advanced ones like Bayesian optimization or evolution strategies. Running HPO is not a simple task as it is often viewed as a black-box optimization (BBO) problem and often requires proper configuration which may affect both performance of the HPO algorithm and the quality of results [17]. It is worth noting that such black-box approaches often do not have closed-form mathematical representation, hence there is no analytic gradient information available. What is more, the machine learning models include continuous, categorical, discrete, and conditional hyperparameters which make traditional numerical optimization methods unsuitable for HPO. Another basic and crucial difficulty in such optimization is that in the case of SVMs and other supervised learning methods, the training procedure needs to be run (meaning running optimization process for finding $\|w\|$ in case of SVMs, see Equation 2.21) before measuring the objective function. This process is known to be problematic for SVM in particular due to time and memory complexities [104]. Taken those properties together they define a considerably difficult optimization problem.

There are several main categories into which the hyperparameter optimization algorithms can be divided:

1. general methods for hyperparameter optimization — these methods do not impose any prior knowledge of the SVM structure,
2. methods focused on kernel function selection and determination of its hyperparameters — often focused on several predefined kernels,
3. specialized methods that create new hand-crafted kernel functions.

As already discussed, the general methods should perform on average worse than the specialized ones (due to the “No free lunch” theorem), but they have the advantage of much easier setup and application which may favour their choice in practice. The presented categories are not exhaustive, as there are far more attributes which can be taken into consideration, such as the exploration vs exploitation

trade-off (global vs. local behaviour), parallelizability, noise handling or multifidelity (e.g., using subsets of data for quicker evaluation) [136]. Although sub-sampling the training set is used in some of the techniques [140] (sometimes used in conjunction with surrogate models) in general these are just random samples to lower the cost of optimization like in the case of the Hyperband algorithm [81]. The methods of sub-sampling training data will be revised in more detail in Section 2.3.2.

Grid search and random search

Grid search (GS) and random search (RS) are two popular methods for hyperparameter tuning [15]. Their main advantage is the simplicity of implementation (which makes them easily available in many software packages e.g., scikitlearn [103]) and easy parallelization as each evaluation is independent of the others. GS works by evaluating the Cartesian product of a user-specified finite set of values. Assuming there are k parameters and each of them has n distinct values, it creates n^k configurations and trains a model for each of them. The best combination of hyperparameters values is then chosen based on the performance of the model on a validation set. It is easy to see that GS has an exponential time complexity of $O(n^k)$. In consequence, it is not well-suited for problems with large numbers of hyperparameters. What is more, even though it is considered an exhaustive search method, it is often not able to find global optimum in the case of continuous hyperparameters [15]. This has two main reasons: GS cannot exploit the well-performing regions by itself and requires a predefined finite set of values to test. Therefore GS should be considered only for relatively small configuration spaces (low dimensional spaces e.g., 2D) in order to be effective. There are many improvements already proposed to GS including:

- Automatic narrowing of search space, sometimes called hierarchical GS [125].
- Using subsamples of data for initial approximation of promising regions.
- Early stopping.

Due to its availability and simplicity it is still widely used for hyperparameter search in low-dimensional spaces.

RS is similar to GS in the manner of evaluating all of the candidates' configurations. The main difference comes in how these candidates are sampled in a random manner. The number of such samples is predefined and each hyperparameter is sampled based on specified distribution with lower and upper bounds. As the number of samples is fixed, there are only n unique configurations to be tested making its time complexity $O(n)$. In theory, if the configuration space is large enough, then the global optimum or its approximation can be detected. Another advantage over GS is that hyperparameters combinations coming from the specified distributions improves system efficiency by reducing the probability of wasting much time on a small poor performing region.

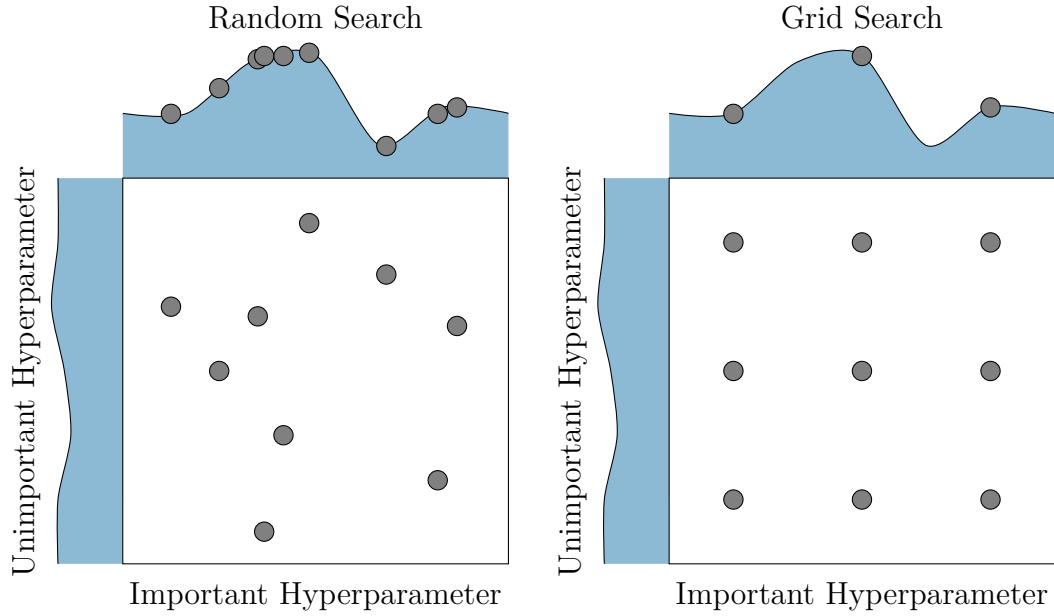


Figure 2.7: Example of RS and GS of nine trials for optimizing function $f(x, y) = g(x) + h(x)$. The importance of the hyperparameters is illustrative and can be obtained *a posteriori*. This figure is inspired by [15].

This situation is illustrated in Figure 2.7 where nine trials of different hyperparameters are presented. There are two different hyperparameters that differently affect end results, denoted as important and unimportant. In the case of GS points evenly cover the original 2D space while their projections over either of the hyper-

parameters produce an inefficient coverage of just three points. In contrast to that uniformly sampled points are less regular in the original space but far better cover each hyperparameter subspace. In that case, the advantage of RS is exploring nine different values of an important hyperparameter whereas GS checks only three. Although as discussed by Bergstra [15] this is a rule rather than an exception, GS could still be useful in selecting SVM hyperparameters as considering RBF kernel for example, there are just two hyperparameters to select and we limit the number of trials needed for grid search by sampling on a logarithmic scale between lower and upper bounds (as recommended by the LibSVM creators [24]). In both cases (RS and GS) their usage is much better than manually tuning as it saves time and increases the reproducibility of results (for RS it is important to get the same seeds for the random number generator).

There is still ongoing research in improving RS algorithms. The work by Florea et al. [44] focused on reducing the computational complexity of the RS method by limiting the number of trials needed to find a good solution. They achieved it by introducing a dynamically computed stopping criterion. That algorithm is divided into two parts. In the first one, a small number of predefined trials are evaluated and the best solution is remembered. In the second part, they try to find a new solution better than the one found before. If such a solution is found the algorithm is terminated. As presented by the authors, this led to a decrease in the number of RS evaluations (SVM trainings) needed without significantly lowering accuracy.

Other interesting works on improving RS are successive halving [71] and Hyperband [81]. The idea behind successive halving is to allocate budgets (B) for each of configuration (n). During the first iteration of the algorithm, all of the configurations are evaluated but within a limited budget ($b = B/n$). Then according to the evaluation results, half of the poor-performing hyperparameter configurations are eliminated and the rest is passed to the next iteration with doubled budgets. This process repeats until there is a final (the best) configuration detected. The main difficulty in running such an algorithm is determining the trade-off between running a few configurations with a high budget or testing more configurations but with lower budgets.

Hyperband algorithm tends to solve this problem. It is a wrapper method of successive halving. It is considering multiple different values for the number of

configurations (n_i) where each such number is allocated with its own budget. The larger the n_i is, the smaller budget is being assigned to it. Then for each such n_i the successive halving procedure is performed. Hyperband needs two parameters to be set: the maximum amount of resources R that can be assigned to one configuration, and the fraction η of configurations that are eliminated each time successive halving is run. The method starts with the most populous group for maximum exploration. Every following iteration reduces the number of configurations by a factor of η , until the final iteration where each configuration is given R resources, which is similar to a traditional random search. This approach allows Hyperband to take advantage of situations where adaptive assignments perform well, while still maintaining satisfactory results when conservative allocations are necessary.

Bayesian optimization

Bayesian optimization (BO) [113] is a powerful optimization technique that is particularly well-suited to problems where the number of function evaluations is limited. It determines the next hyperparameter value based on the previous results of tested hyperparameter values, which avoids many unnecessary evaluations. Thus, it can be considered a sample-efficient method. This approach is particularly useful when the objective function is expensive to evaluate, or when the function is a black box with no gradient information available. However, due to the fact that Bayesian optimization models operate based on the values that were previously tested, they are considered sequential methods and are challenging to parallelize.

In order to select the next values to test, BO builds a probabilistic model of the objective function (called a surrogate model) and then uses this model to guide the search for the optimal solution. The surrogate model aims to fit all the currently observed points into the objective function. Another important part of BO is an acquisition function which determines how new points will be sampled. It needs to balance the trade-off between exploration and exploitation. In exploration, the sample is used to gain knowledge about the areas that have not been sampled yet, while exploitations try to sample the currently most promising regions. Those promising regions are based on the posterior distribution. The general pseudocode for the Bayesian optimization algorithm is presented in Algorithm 2. This

particular example uses the Gaussian process for a surrogate model.

Algorithm 2 Bayesian optimization algorithm with Gaussian process (GP) as a surrogate model. Kernel function refers to the covariance structure of a Gaussian process (GP) model.

- 1: Initialize an empty set of samples S
 - 2: Select a kernel function and a prior distribution for the GP
 - 3: **for** $i = 1$ to n **do**
 - 4: Select the next point x_i to evaluate by maximizing the acquisition function (e.g. expected improvement)
 - 5: Evaluate the objective function $f(x_i)$
 - 6: Add $(x_i, f(x_i))$ to the set of samples S
 - 7: Update the GP model using S
 - 8: **return** x that maximizes $f(x)$
-

In general, Bayesian optimization methods are formalized as Sequential model-based optimization (SMBO). Common surrogate models for BO include Gaussian Process (GP), Random Forest Regressions (RF), and tree Parzen estimators (TPE) [14]. This kind of optimization might seem like a perfect fit for any machine learning algorithm, as each evaluation of objective function is very expensive.

One of the recent examples of using Bayesian optimization with the Gaussian Process is presented in predicting household vehicle ownership [135]. The authors used described above algorithm for BOGP. They were able to improve the results of the SVM model performance by several percentage points on the test set thanks to BO. What is more, the optimized SVMBO (as named by the authors) performed better than other tested machine learning models including k -nearest neighbors and decision trees.

A combination of selecting random subsets of data for initial evaluation and combining this process with BOGP was proposed by Klein et al. [76]. The authors propose a method called **FA**st **B**ayesian **O**ptimization on **LA**rge **d**ata**S**ets (FABOLAS) which, during sampling for the next solution, selects additional parameter s which is the fraction of data that will be used for evaluation ($s \in (0, 1]$). While the goal is to optimize performance for $s = 1$, it is usually much cheaper to use smaller parts of the dataset while obtained values should still correlate across s domain. Based on some initial experimentation, the authors proposed to

use a logarithmic scale for s as tested on MNIST dataset (a collection of 70,000 handwritten digits that is commonly used as a benchmark) for $s = 1/128$ already yielded representative results. This observation is also used in one of the algorithms presented in this work (see Section 3.5). FABOLAS is often able to discover efficient configurations at a much faster rate, typically 10 to 100 times faster, when compared to related techniques such as Multi-Task Bayesian optimization, Hyperband, and regular Bayesian optimization. The mentioned BOGP has its limitations as it is best suited for working with continuous variables. In the SVM case, this means that BOGP is used for optimizing hyperparameters of the kernel and C value (most often RBF with single γ hyperparameter is selected). Not being able to handle efficiently discrete, categorical, and conditional hyperparameters, other surrogate models were introduced. This includes random forest regression [69] and Tree-structured Parzen estimator [14, 16].

Bayesian Optimization HyperBand (BOHB) [41] is another cutting-edge method for tuning hyperparameters that combines the strengths of Bayesian optimization and Hyperband while avoiding their weaknesses. Traditional Hyperband relies on random search to explore the hyperparameter configuration space, which can be inefficient. BOHB addresses this issue by utilizing Bayesian optimization, which is more efficient and allows for parallel resources to be used to optimize all types of hyperparameters. BOHB uses TPE as its standard surrogate model for Bayesian optimization and employs multidimensional kernel density estimators. It has been demonstrated that BOHB surpasses many other optimization techniques when tuning SVMs and deep learning models. The only limitation of BOHB is that it requires that evaluations on subsets with small budgets are representative of evaluations on the entire training set (which is often true for SVMs [76]), if not it may have a slower convergence speed than standard Bayesian optimization models.

Building specialized kernels

Another approach to optimizing the SVM model is to build a new (custom) kernel function. This can include a combination of different kernel functions e.g., polynomial, linear, and RBF [26, 67]. Different valid kernel functions can be joined using their properties, so if a given function satisfies Mercer conditions and is

considered a valid kernel the same is true about the sum or multiplication of two kernel functions. One more important thing is how some of the kernel functions e.g., RBF are considered local functions as they affect only some regions around SV (if the samples lay further from the SV the kernel function value is 0), while on the other hand linear and polynomial kernels are regarded as global functions as each support vector contributes to the final decision boundary. Most of the time joining those local and global functions tends to provide improvement in classification performance. While this approach can seem tempting, it often results in creating additional hyperparameters that need to be tuned.

Another possible approach could be the selection of the kernel from a set of predefined functions [4]. The so-called weak kernels can be used to find the best linear combination of those in order to apply it to SVM training. The experimentation shows promising results on the used datasets and robustness to problems where feature selection is important.

The technique of genetic programming has also been used for developing and refining kernels, including those specifically designed for natural language processing known as string kernels [117]. Additionally, kernel functions can be tailored and enhanced through the use of reinforcement learning [10]. The selection of the model can also extend beyond commonly-used kernels to include options such as Mahalanobis kernels [72] or kernels that are specifically suited for datasets with varying class densities, known as isolation kernels [124].

Metaheuristics

Last but not least metaheuristic methods were also used to tune SVM hyperparameters, these include, GA and particle swarm optimization as the two most prevalent metaheuristic algorithms. The GAs were already described in Section 2.2. Particle swarm optimization (PSO) is inspired by the behavior of a swarm of birds or a school of fish, by mimicking the behavior of a swarm of particles that move through the search space, where each particle represents a potential solution to the problem. The movement of particles is affected but the current best solution found (by any particle) and their own inertia and momentum parameters.

An example of applying such metaheuristics to optimize SVM can be found

in work by Zhou et al. [143]. They employed three different algorithms: whale optimization algorithm, gray wolf optimization, and moth flame optimization for optimization of SVM hyperparameters (namely C and γ). The presented results were used to predict the advance rate of a tunnel boring machine which is a key parameter for tunneling projects. As presented the moth flame optimization provided the best results although the size of the population of 150 could be used only for small datasets.

Another example of a slightly modified version of PSO called switching-delayed-PSO is presented by Zeng et al. [139]. This modification delays the information of local and global best particles during the velocity update. It allows for improvement in the results by finding the current state of evolutionary computation (convergence state, exploitation state, exploration state and jumping-out state) and adjusting the parameters used by the heuristic. This allowed to provide state-of-art results for the diagnosis of Alzheimer's disease.

The genetic algorithms were also successfully applied to the problem of SVM model selection [80]. The authors propose to combine multiple popular kernel functions and optimize their hyperparameters at the same time. They also used prior knowledge where the hyperparameters are sampled using a logarithmic scale. This early work presents interesting improvements over regular SVMs.

Another interesting extension to classical GAs is incorporating feature weighting into the evolutionary process [121]. It can serve a feature selection (when weight is equal to zero) purpose and further improve the quality of classification. This solution is then applied to intrusion detection problem on which it provides faster training time of SVM and decreased error rate.

Summary

As this problem is still very relevant not only for SVMs but also for other machine learning methods, there is a lot of research in this area. There are great review articles which can provide a good overlook of the field and current research problems as well as available methods [136]. However, it is hard to properly compare many of the presented techniques as they are often used in different sub-fields, where either datasets or implementations are not freely available or some specific

knowledge during the processing could be used.

An interesting summary of currently available methods was made by Wainer et al. [126]. The authors focus purely on optimizing SVM hyperparameters with 18 different methods using over 100 datasets from the UCI repository. Their finding is that none of the presented algorithms shows significant improvement in terms of SVM accuracy in comparison to grid search algorithms. It is worth noticing that the authors took on a practical approach to using many advanced algorithms like PSO based on their default parameters available in the software packages, while e.g., population initialization could play a crucial role in many metaheuristic techniques. Although the performance of SVM is heavily dependent on hyperparameter selection all of the available methods perform equally well when comparing classification performance. It is important to mention that not all of the HPO algorithms take the same amount of time/evaluations (which is the main differentiation between those tested algorithms). Time performance could be a good selection criterion for which algorithm to choose. On the other hand, an argument to use methods that are easier available, easier to implement or tend to present other advantages (such as easy parallelization) could be made. Whichever method is selected, HPO will virtually always guarantee improvement of performance over some default values provided in software packages.

2.3.2 Selection of SVM training set

Generally, training models with enough information is essential to achieve good performance. However, it is common that a training dataset $\mathbf{T}$ contains samples that may be similar to each other (that is, redundant) or noisy. This increases the computation time and can be detrimental to generalization performance. The process of instance selection (also called training set selection) aims to select such subset $\mathbf{T}' \in \mathbf{T}$, which will represent the whole training and achieve similar performance while reducing necessary computation. The problem of the training set selection is similar to feature selection (described in 2.3.3). In the same way, these methods can be categorized in a different manners:

1. **Incremental** method ($\mathbf{T}' = \emptyset$ at the start) or **decremental** method ($\mathbf{T}' = \mathbf{T}$), **Batch** (mark all instances that should be removed and remove them at

once), **Mixed** (preselect a subset, and repeatedly, add or remove instances from this subset), and **Fixed** (size of $\mathbf{T}'$ is predetermined) [47],

2. **Wrapper** The selection criterion is based on the accuracy obtained by a classifier or **Filter** The selection criterion uses a selection function which is not based on a classifier,
3. **Dependent** or **independent** of $\mathbf{T}$ cardinality in terms of computational complexity.

For this dissertation the categories presented in Figure 2.8 will be used to provide an analysis of the field.

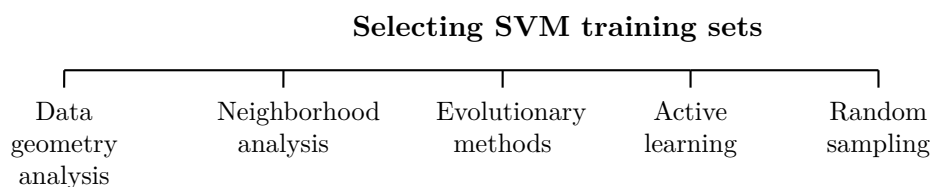


Figure 2.8: General categories of approaches for selecting SVM training sets. This figure is inspired by [97].

Commonly, SVMs can be successfully trained using just a subset of all the data that are available for training. There are RS approaches proposed. Similarly to HPO problems such methods are simple to implement and often sufficient for finding a reduced training set (at least if the size of the reduced set can be estimated *a priori*). Another of their advantage is not depending on the cardinality of $\mathbf{T}$. One of such approaches was proposed by Balcazar [11]. It works by starting with a random subset of $\mathbf{T}$ sampled according to the weights assigned to training vectors. Initially, an SVM classifier is trained using this random subset. Afterwards, the whole training set is analyzed to check which vectors are classified correctly. Then the weights for misclassified vectors are increased so they are more likely to be sampled in next iterations. If the number of iterations is large enough, the important vectors, including SVs, will have a higher weight than other vectors thus the refined set should be composed of those vectors. The main drawback of this

algorithm is the need to set the desired size of $\mathbf{T}'$ beforehand. This might require many trials and could become time-consuming (especially for large datasets). Another problem is that random sampling may ignore any relations which occur in a dataset.

Data geometry analysis, on the other hand, is focused on exploiting such relations. These methods often select vectors from the regions lying close to the separating hyperplane. Of course, this would need to determine the approximate location of the hyperplane in order to find the most important vectors. A subgroup of data geometry analysis employs various clustering techniques to identify the clusters formed by the vectors from the same class [127, 33]. This approach makes it possible to eliminate vectors that are situated within these clusters, as they have a lower probability of being chosen as SVs during SVM training.

Shen et al. [111] proposed a method for eliminating unnecessary training set vectors by analyzing cluster boundaries and examining other inter-cluster relationships. The k -means clustering is used to obtain clusters, and for each cluster, a distance density set is calculated. Vectors that are close to the centroid are considered “dense”, and those that are far from being considered “sparse”. The authors use Fisher’s discriminant analysis to find the boundary between the dense and sparse parts of each cluster and include only the sparse vectors for the selection of a refined set. Additionally, the authors consider removing redundant (one-class) clusters as there is a low probability of selecting SVs in such regions.

The main disadvantages of using clustering techniques are their time complexity and the need to analyze the entire training set. Additionally, there are different crucial hyperparameters of clustering methods which can further affect their results. Geometric information can also be obtained without using clustering. Methods like these include applying β -skeleton algorithms [142] or utilizing other basic classifiers [2] to get this information. Another example of such work includes the usage of the fast nearest neighbor condensation classification rule (FCNN) [7]. SVMs are combined with FCNN, where the vector selection criteria are based on the decision boundary. The FCNN process starts with an initial refined training set that is composed of the centroids generated for each class independently. For each vector, $a \in \mathbf{T}'$, a point belonging to the Voronoi cell (i.e., a set of $\mathbf{T}$ vectors that are positioned closer to a than any other vector in the current $\mathbf{T}'$) of a ,

but labeled with the opposite class, is included in the refined set. The algorithm continues until there are no more vectors from $\mathbf{T}$ to be added to $\mathbf{T}'$.

Another group of algorithms uses neighborhood analysis. It is assumed that the SVs are more likely to lie in heterogeneous regions of the dataset. This is likely to occur, as the decision boundary is often located close to such areas [57]. These algorithms include the usage of ensemble classifiers as done by Guo et al. [58]. They exploit a custom margin definition which is calculated for each vector based on the number of votes of the base classifiers for a given class. These margins lie in the range of $[0, 1]$ where smaller values indicate vectors closer to the hyperplane (as learned by the ensemble). The refined set for SVM is selected by sorting all of the vectors according to their margin values and selecting the smallest ones. As the base classifiers, the authors used decision trees and applied bagging to build an ensemble.

Another work proposes to exploit an induction tree to predict if the vector is likely to become a SV after SVM training [22]. To construct the tree, an SVM is initially trained using a small randomly selected subset of $\mathbf{T}$ to identify which vectors are selected as SVs. The selection process in this method employs a simple heuristic to balance the $\mathbf{T}'$ for SVM training. Afterwards, vectors are passed to the tree to indicate the potential best candidates for SVs. These are included in $\mathbf{T}'$. In summary, in order to effectively use methods that analyze the data geometry or local neighborhood, it is still necessary to examine the entire $\mathbf{T}$ to choose the valuable subset.

Evolutionary algorithms, such as genetic algorithms, have been shown to be effective in finding a reduced training set for SVMs. They typically use a wrapper approach, where the fitness of a reduced set is determined by training an SVM with that set. The population of reduced sets is then evolved to find the best one [74]. Similarly as in the case of random selection [11] it can be difficult to select the optimal size of a reduced set. This led to the development of more advanced adaptive strategies which can tune the size of $\mathbf{T}'$ during the optimization process [94]. The adaptation is based on observing the performance of the whole population and looking for certain signals (e.g., number of SV in relation to the size of $\mathbf{T}'$). Depending on their occurrence, the size of the reduced set can be increased. It is important to note that such a method usually starts with a very small refined

set. More advanced methods incorporating additional information collected during the evolution were also proposed [96, 95]. These specialized evolutionary methods (memetic algorithms) incorporate additional local search procedures in order to better balance exploration and exploitation of the search space.

Another interesting work proposes to incorporate multi-objective optimization techniques as the goal of the training set selection is the reduction of the size of the original $\mathbf{T}$ while maximizing the classification performance (e.g., accuracy). Cheng et al. proposed a method called SDMOEA-TSS [28], which uses multi-objective optimization techniques to reduce the size of the training set while maximizing classification accuracy. The method divides the objective space into several subregions to create a diverse population. The algorithm can be adjusted to prioritize either reducing the size of the training set (SDMOEA-TSS(A)), increasing accuracy (SDMOEA-TSS(C)) or finding a balance between the two (SDMOEA-TSS(B)).

One of the key advantages of the evolutionary methods is that they are independent of $\mathbf{T}$ cardinality when selecting reduced sets. Although these methods provide state-of-the-art results, they often require the initial optimization of the SVM model, such as determining the appropriate kernel function and its hyperparameters, which is usually done through a time-consuming grid search [28, 95]. In the next section, the **Memetic Algorithm to Select Training Data for SVM** (MASVM) [95] will be presented. This is one of the evolutionary approaches for the training set selection designed specifically for SVM and employing the knowledge about SVM classifier. This section is important as it provides insights into the implementation of MASVM and describes this algorithm in detail.

Memetic Algorithm to Select Training Data for Support Vector Machines (MASVM) [95]

In this subsection, the MASVM algorithm will be discussed in more detail as it is utilized for the training set optimization among proposed algorithms. The overview of this algorithm is presented in Figure 2.9.

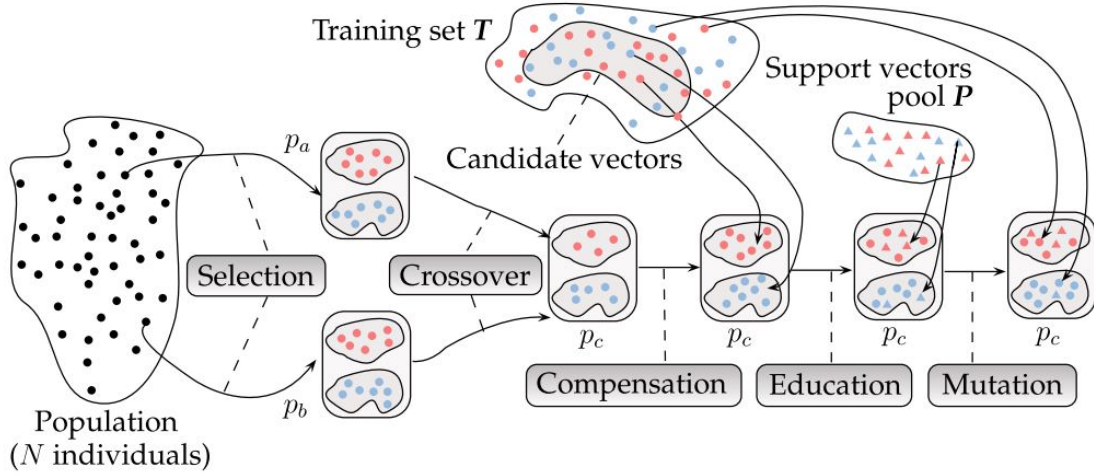


Figure 2.9: Overview of MASVM algorithm. Source: [97].

It starts by creating a population of N individuals by random sampling the training set. Each chromosome is built of K_t vectors from each class (only binary problems are considered so the final size of the chromosome is $2 \cdot K_t$). Thus each chromosome represents a refined training set $\mathbf{T}'$. Once the population is created, the fitness of all individuals is calculated. The fitness calculation includes training N distinct SVMs using the $\mathbf{T}'$ from chromosomes and evaluating their performance on the validation set.

After this initialization phase, N pairs (p_a, p_b) of individuals are selected using the local-global adaptation scheme (LGA), where $p_a \neq p_b$. In this approach, the population is arranged in descending order based on their fitness values and divided into two groups. The first group includes the top $(\epsilon \cdot N)$ individuals with the best fitness, while the other group includes the remaining $(N - \epsilon \cdot N)$ individuals. In the local selection mode, $\epsilon \cdot N$ pairs are created from the well-fitted (first) group while the rest are randomly drawn from the less-fitted part (N pairs are determined to create new population with the same size as initial one). In the global mode, pairs are created by selecting one individual from each group to promote balance and exploration of the solution space. The selection method is adaptively changed during the algorithm's execution.

After creating N pairs, a single child is generated for each pair. This new

individual inherits all of the vectors from both of the parents ($P_c \leftarrow P_a \cup P_b$) while maintaining their uniqueness. The size of the resulting reduced training set $|P_c|$ is randomized, and equals $\max(|P_a|, |P_b|) \leq |P_c| \leq 2K_t$. If the number of distinct vectors inherited from P_a and P_b is smaller than selected $|P_c|$, additional vector are randomly drawn from $\mathbf{T}$ (compensation operation),

In the next step, the education process is employed. There is a chance (determined by the P_e) that training vectors in the offspring solutions that were not selected as SVs (in previous SVM training) will be replaced with random vectors selected from the pool of support vectors. This pool contains all vectors that were chosen as support vectors during the evolution process. Afterwards, the newly created population (P') undergoes mutation with a probability P_m . In this step, a certain percentage ($f_t\%$) of vectors in each individual are selected and replaced with random vectors from the $\mathbf{T}$.

To further take advantage of the vectors that were determined to be important during the evolution process (as they were selected as SVs), the so-called super individuals (SIs) are created and evaluated. The number of SIs is equal to $\alpha \cdot N$, where $1 \geq \alpha \geq 0$. These super individuals consist of up to $2K_t$ randomly chosen and distinct support vectors from the support vector pool. Up to K_t vectors are picked for each class. Note that in case there are not enough vectors in $\mathcal{S}_{\text{pool}}$ this number could be lower.

In the selection of individuals for the next generation the size of the population is kept constant and the best N individuals are chosen from the previous population (P), current (intermediate) population (P'), and the set of SIs (P^{SI}).

In the end, the hyperparameters of the algorithm are adapted and set $K = \rho_K \cdot K$, where $\rho_K = 1 + (\varrho_{sv} - \mathcal{T}_{\text{SV}})/(1 - \mathcal{T}_{\text{SV}})$, $\mathcal{T}_{\text{SV}}$ is the SV threshold and $\varrho_{sv} = \text{SV}(P'_{\text{best}})/(|P'_{\text{best}}|)$, $\text{SV}(P'_{\text{best}})$ denotes the number of SVs in the best individual from the current population, and $|P'_{\text{best}}|$ the size of P'_{best} , alongside. The K_t value can be increased only once in $\mathcal{T}_s$ generations. This help to fully exploit the current value of K_t . Additionally, K_t will remain constant if the average fitness in the population did not increase since the last generation ($\Delta\eta = 0$), or the improvement was greater than $0.5 \cdot \Delta\eta$. If in the last $\mathcal{T}_s$ generations K_t was not updated and the current LGA selection scheme is in the local mode it is switched to the global mode. This provides a better exploration of the solution space. Once K_t is updated, the LGA

is set to local mode again.

The evolution continues until the termination condition has been met. The original proposed stop condition included multiple possible conditions: fitness not growing in a given number of consecutive generations, K_t is not further increased or the desired fitness is reached. During the mentioning of MASVM in this dissertation the average population's fitness is monitored — if its improvement is smaller than $\eta_{\min}$ in two consecutive generations the algorithm is stopped. This condition is based on the observation that the population's diversity is likely very low (as average fitness did not change), thus the probability of improving the fitness of the best individual drastically decreases [116].

One important difference in the above description compared to the original MASVM [95] is omitting of the regeneration process as suggested by Nalepa et al. in subsequent work [96]. This should decrease the running time of the algorithm while it should not affect the end results in terms of the quality of selected $\mathbf{T}'$.

2.3.3 Feature selection

Feature selection is the process of identifying a subset of the most relevant features from a larger set of features for a given prediction task. The goal of this process is to improve the performance of the model by reducing the dimensionality of the input space, and by eliminating irrelevant, redundant, or noisy features that can negatively impact the performance of the model. There was a tremendous effort to develop different methods to address this problem. Feature selection methods can be broadly classified into three categories [13]:

1. **Filter methods** These methods evaluate the relevance of features based on statistical measures or domain knowledge, and select a subset of features independently of the classification model. Examples of filter methods include correlation-based feature selection, mutual information, and chi-squared test [119, 114].
2. **Wrapper methods** These methods evaluate the relevance of features by training and testing different models with different subsets of features, and selecting the subset that optimizes the performance of the model. Examples

of wrapper methods include recursive feature elimination [6], GAs [85], and simulated annealing [1].

3. **Embedded methods** These methods incorporate feature selection into the model training process, by adding a penalty term or a constraint to the optimization function that encourages sparse solutions. Examples of embedded methods include Lasso regression, Ridge regression, and Elastic Net [23, 101].

In practice, the choice of feature selection algorithm depends on the nature of the data, the size of the feature space, computational resources, and the desired performance metrics. For example, filter methods are often used for high-dimensional datasets, while wrapper and embedded methods are better suited for smaller feature spaces [18].

One interesting example of the filter method is proposed by Yao et al. [137]. A new ensemble feature selection method called FS-MRI, which integrates multiple ranking information, has been introduced. The FS-MRI method can automatically determine the threshold function based on the model performance, resulting in the selection of an optimal and stable subset of features. The approach incorporates nine different feature selection methods into its framework.

The SVM Recursive Feature Elimination (SVM-RFE) [59] is a wrapper-based feature selection approach that utilizes the SVM as the base classifier. The method uses the objective function $(1/2)w^2$ (see Equation 2.8) to rank the features by their discriminatory ability. At each step, the feature with the smallest ranking score is eliminated, based on the corresponding component of the weight vector. This process is recursively repeated until all features are ranked in order of importance, using a backward feature-elimination scheme that excludes insignificant features.

A more recent work propose to couple SVM RFE with Binary Biogeography Optimization (BBO) [6]. In order to improve the quality of solutions obtained through the mutation operator and strike a balance between exploitation and exploration, SVM-RFE is embedded into BBO. The resulting approach, BBO-SVM-RFE, shows promising potential in effectively searching the feature space to obtain the optimal combination of features. The obtained results indicate that BBO-SVM-RFE is a reliable method for feature selection.

Besides that, other works in this area include the wrapper method of feature selection with a lot of work put into using evolutionary algorithms [51, 52]. In general, each individual in EA is coding a separate set of features. After applying an appropriate method to produce new solutions the SVM is trained using those feature sets and evaluated using the selected metric. Neumann et al. [100] proposed the modification in the SVM training process to gain information about features that should be used. In order to simultaneously select features and construct a model, the authors have introduced an additional term to the standard cost function of SVM. This added term penalizes the cardinality of the selected feature subset, allowing for the optimization of a modified cost function.

The choice of feature selection algorithm depends on the data, the size of the feature space, computational resources, and the desired performance metrics. With the increasing availability of large datasets, efficient and effective feature selection methods are becoming more important for building accurate and efficient machine learning models.

2.3.4 Multi-aspect SVM optimization

The issues of selecting the appropriate training samples and features, as well as optimizing the hyperparameters of SVMs are closely related, thus there are many methods to address these problems together. The problem where multiple optimization techniques are involved will be referred to as multi-aspect optimization.

Raman et al. proposed a new method for optimizing the hyperparameters and features in SVMs by introducing a hypergraph GA [108]. They included the features selection in the chromosome encoding, along with γ and C values of binary classifiers, which were used to solve multiclass classification. Another similar approach was used in another work, which utilized a nature-inspired multi-verse optimizer (MVO) [42]. Several studies have shown the effectiveness of combining model selection with feature selection [122, 70, 5]. PSO was used to achieve this integration in the work by Huang et al. [65]. These methods demonstrate the usefulness of simultaneously selecting the most valuable features and optimizing the model's parameters.

SVM kernel evolution may also be combined with training set selection. In

research by Nalepa et al. [98] a scheme in which the kernels are elaborated using a neuro-fuzzy system and the training set is selected afterward using an evolutionary algorithm.

There were also several attempts for simultaneous selection of training sets and features. Garcia-Pedrajas et al. proposed a memetic technique [49] with four different local search procedures and a new fitness function which provides a proper balance between selecting the training samples and removing the features. Such combined selection can also be performed in a static manner, to identify the inactive features and samples before the SVM optimization [141]. The article [82] proposes a hybrid memetic algorithm that combines variable neighbourhood search and a memetic algorithm to simultaneously select instances and features. The proposed method for feature and instance selection is utilized to reduce the size of data and train a predictive model for later performance evaluation. Compared to other metaheuristics, the proposed approach strikes a balance between exploration and exploitation, and the results demonstrate its superior robustness over other feature selection techniques.

In the context of SVM model optimization, simultaneous optimization of the three interconnected elements — features, training samples and model — has been shown to lead to improved performance and classification scores. Previous works have focused on optimizing two out of the three elements. However, integrating feature selection into the evolutionary process and optimizing all three elements together in an alternating or simultaneous manner, as demonstrated in the works co-authored by the author of this dissertation [38, 36, 37], has resulted in models with a lower number of SVs and features while maintaining high classification accuracy.

2.3.5 Building classification ensembles

Ensemble learning is a technique used in machine learning to improve the performance of a model by combining the predictions of multiple base models. The idea behind ensemble learning is that multiple models, each trained on the same dataset (not necessarily on the same subset of data), can provide different perspectives on the problem, and by combining their predictions, the final model

can be more robust and accurate than any of the individual models. There are several popular methods for building ensembles in machine learning, including:

1. **Bagging:** This method involves training multiple models on different subsets of the training data, and averaging their predictions. This can be used to reduce the variance of the model and can be applied to both regression and classification problems.
2. **Boosting:** This method involves training multiple models in sequence, where each model is trained to correct the mistakes of the previous model. This can be used to reduce the bias of the model and is most commonly used for classification problems.
3. **Stacking:** This method involves training multiple models on the same data, and using their predictions as input to a meta-model that makes the final prediction. This can be used to improve the performance of the model by combining the strengths of different models.

Ensemble methods generally require more computational resources than separate models, but they can provide significant improvements in model performance. They are especially useful when working with complex problems or large datasets.

There are multiple works that tried to build ensembles of SVMs. Early work proposed a Cascade of SVM [56] that can efficiently parallelize and handle very large problems with hundreds of thousands of training vectors. The training data is divided into subsets and on each part, a separate SVM model is trained. The partial results are combined and filtered in a cascade of SVMs until the global optimum is reached. This approach is memory-efficient and faster than a regular SVM, even with a single pass.

A similar strategy was used by Claesen et al. [31] where multiple SVMs are trained separately on different subsamples of the $\mathbf{T}$, ultimately leading to faster training. To further increase the efficiency, the SVs are analyzed and shared among models so if the same vector is selected as a SV in multiple models it needs only one calculation of the kernel function. This, however, requires the base model to share a single kernel function with the same hyperparameters. One more problem

with that approach is that the size of subsamples used for training needs to be set beforehand so it becomes another hyperparameter that has to be tuned.

The GenBoost-SVM method [109] uses an adaptive boosting algorithm. The authors examined different pre-selections of a training set using GAs to reduce training times and tackle imbalanced data. Additionally, diversity and early stopping were considered to reduce the generalization error. Using different SVM kernels was shown to improve the performance of such a classifier. It can be already noticed that all of the presented works used some methods to reduce the size of the training set.

Pławiak et al. [106] presented a deep genetic cascade ensembles of classifiers. This work presented a stacking classifier with 16 layers of SVMs that are optimized using a GA. Although the presented results show great promises in credit risk scoring achieving state-of-the-art performance, the study focused on small datasets. It might be difficult to scale this solution to big datasets due to the high computational cost.

The already mentioned systems use SVMs only as the base models which makes them a homogeneous ensemble. Joining multiple different classifiers could also boost the performance, as the learning algorithms tend to produce different decision boundaries. That heterogeneity is crucial in an ensemble setting. One such example is joining SVM prediction with k -nearest neighbors algorithm [134]. Using stacking for building ensembles with three different base models [99] was tested by Nanglia et al. It was proved to provide increased performance in comparison to homogenous ensembles as well as other machine learning models.

2.3.6 Summary

The field of machine learning is constantly growing, with a vast number of solutions available for each problem, such as hyperparameter optimization, training set selection, and feature selection. While various techniques have been used in a joint setting, there is still a lack of a single method that can cover all these areas.

The solution proposed in this dissertation will introduce a new method that can efficiently cover hyperparameter optimization, training set selection, and feature selection simultaneously. Moreover, this method could be extended to build a new

kind of ensemble that will utilize both SVM base models and Random forest models to provide a heterogeneous ensemble. Such an ensemble could improve the performance of SVMs, and leverage the advantages of simultaneous optimizations. By combining these different techniques into one cohesive framework, one can leverage their strengths to build more robust and efficient machine learning models.

Chapter 3

Proposed methods

In this chapter, all of the newly introduced methods will be presented. The chapter is structured around describing the building blocks of the methods and putting them into the context of how they were designed and used. Those techniques focus more on combining different aspects (such as training set selection and hyperparameter optimization) rather than on improving a single one of them individually. To make it easier for the reader to clearly see what algorithms were used in a given method Table 3.1 summarizes it. Methods that are seen in the left part of the table are presented in chronological order of being developed (the same as described in the sections). Some of those methods (e.g., ALMA, SE-SVM, or CE-SVM) can be treated as a general framework for the optimization of SVM so they re-use the same algorithms for hyperparameter optimization or training set selection. In the early work it was presented that these solutions could be easily exchanged, where GASVM [74] in ALGA [75] method was changed to MASVM [95] creating ALMA method while the alternating optimization schema remained unchanged. To not double the information the common building blocks of the methods are described separately. The chapter starts with a description of the genetic algorithm for hyperparameter optimization (GAHP) in Section 3.1 followed by the evolutionary algorithm for feature selection (EFS) in Section 3.2. Moreover, these algorithms (GAHP and EFS) are never used separately, meaning that neither of them is tested later in experimental validation (Chapter 4.3). One more of the important building blocks of those algorithms is the training set

Table 3.1: A summary of algorithms used in the proposed methods.

Method	Section	Algorithm used				Simultaneous optimization
		GAHP (Section 3.1)	MASVM (Section 2.3.2)	EFS (Section 3.2)	RFE (Section .1)	
ALMA [38]	3.3	✓	✓	✗	✗	✗
SE-SVM [37]	3.4	✓	✓	✓	✗	✓
ARBF-SVM [93]	3.5	✗*	✓*	✗	✓	✓
DA-SVM [39]	3.5	✗*	✓*	✗	✓	✓
CE-SVM [40]	3.6	✓	✓	✗	✗	✓
ECE-SVM	3.6	✓	✓	✗	✗	✓

* ARBF-SVM and DASVM are not using GAHP but they still optimize SVM hyperparameters.
The MASVM algorithm is modified in those methods as described in Section 3.5

selection. This is done by MASVM, proposed by Nalepa and Kawulok [95], and discussed in detail in Section 2.3.2. As this is an algorithm proposed in literature it is briefly tested to compare it with the proposed methods.

The rest of this chapter is structured as follows. First, the method for alternating optimizations is presented in Section 3.3. Afterwards, the simultaneous optimization is described (Section 3.4) followed by a specialized algorithm that creates an adaptive kernel in Section 3.5. Finally, the algorithms for building an ensemble classifier in form of the cascade are presented (Section 3.6). The chapter is finished with a summary of the methods in Section 3.7 that also briefly discusses works not described here in detail.

During this chapter there is a lot of symbols introduced for the description of the algorithms, all of those are gathered in Table 3.2 where first the general symbol applicable in multiple methods are presented and later section of table presents symbols used only in specified algorithms.

3.1 A genetic algorithm for optimizing SVM hyperparameters (GAHP)

As discussed in Section 2.3.1 and presented in Section 2.1.3, the performance of SVM classifier is heavily dependent on its hyperparameters. One of the popular techniques to perform this optimization is GS algorithm [95, 74]. However, GS require multiple training of SVM classifier which is difficult for large datasets. Whereas, the methods for training set selection were already introduce to tackle

Table 3.2: Summary of symbols used for algorithms description.

	Symbol	Description
General	C	hyperparameter of SVM, penalty for misclassifying training examples
	γ	hyperparameter of RBF kernel
	$\mathbf{T}$	Training set
	$\mathbf{V}$	Validation set
	Ψ	Test set
	η	Fitness value
	$\mathcal{M}$	Set of SVM hyperparameters
	$\mathbf{T}'$	Reduced training set, $\mathbf{T}' \in \mathbf{T}$
	P	Population in evolutionary algorithm
	N	Size of the population
	p_a	Chromosome/individual in the population
	ϵ	
	$\mathbf{F}$	Features set
	$\mathbf{F}'$	Reduced set of features, $\mathbf{F}' \in \mathbf{F}$
	$ \mathbf{T} $	Size of the training set
GAHP	c	Number of classes/labels
	K_t	The number of class examples selected for $\mathbf{T}'$
	α	Parameter of crossover operator
	P_m	Probability of mutation
	u	Mutation scale
EFS	β	Upper range for mutation scale
	l	epsilon for fitness improvement in stop condition
	K_f	Size of the selected feature set
	F_a	Feature set of individual
	E_p	Education probability
	E_r	Education replacement parameter
	M_p	Mutation probability
ALMA	M_r	Mutation replacement parameter
	$\mathcal{F}_{\text{pool}}$	Pool of features used for education
ALMA	Q	Population of $\mathcal{M}$

this issue in majority of cases they used GS for SVM hyperparameters optimization. The goal of introducing the GA for SVM hyperparameter optimization is to improve on the existing training set selection method which uses evolutionary computation. The motivation to go for a GA is that it should be better than GS or RS techniques while it can still be partially parallelized (during the evaluation of individuals). Moreover, it will be natural to join it later with training set selection into simultaneous optimization process (using EAs). On the other hand as presented in [126] even quite simple method should be sufficient to improve upon currently available solution.

In the initial work, the focus was put on the usage of RBF kernel as it is the most popular one. There are multiple reasons why this type of kernel is a popular choice nowadays. Some of those came from its mathematical properties. It is a stationary kernel, which means that it is invariant to translation. A stationary kernel will yield the same value $K(x, y)$ for $K(x + c, y + c)$, where c may be vector-valued of dimension to match the inputs (the linear kernel does not have such a property). The single-parameter version of the RBF kernel has the property that it is isotropic, i.e., the scaling by γ occurs the same amount in all directions. Moreover, it maps vectors into infinitely-dimensional feature space [61]. On the other hand, it was already tested and found to yield the best results in multiple different applications such as text classification [55], hyperspectral image analysis [77], cancer genomics [66] or medical imaging [105]. What is more, throughout the years some intuition behind the γ hyperparameter grew where it can be thought of as a range of influence of SVs. All of that contributed to first selecting RBF kernel and trying to further improve the SVM performance with it.

The coding of chromosomes is one of the most important steps in designing a GA. For the optimization of hyperparameters that are represented as real numbers, it is natural to use them for representation of the problem (real-coded GA)[60]. In the presented solution considering the RBF kernel, there are two hyperparameters C and γ , however, this algorithm can be used to optimize any kernel (as it is later shown in Section 3.6 when multiple populations are evolved independently).

The outline of the algorithm is presented in Figure 3.1. On the right-hand side of this diagram, the process of fitness (η) evaluation is presented. In order to calculate the fitness of an individual (p_i) first the SVM is trained using the $\mathbf{T}$.

Afterwards, the whole validation set ($\mathbf{V}$) is classified and the confusion matrix is calculated. Based on the chosen metric, the final fitness value for chromosome is assigned.

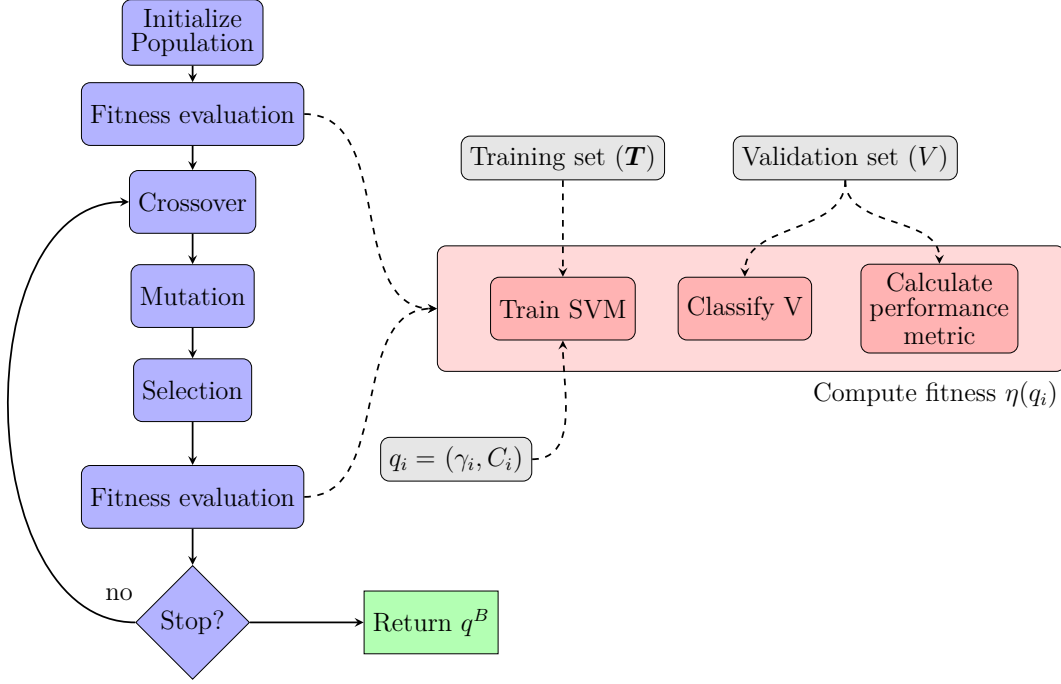


Figure 3.1: Flow diagram of the GA used for hyperparameter optimization.

The algorithm starts by creating the initial population. It is created to cover the initial setup range of values for hyperparameters in a logarithmic way. It calculates the combinations of hyperparameters based on the population size to create a grid. When the size of the population cannot be divided into grid, one individual gets $\gamma = 1/|\mathbf{F}| \cdot \text{var}(\mathbf{T})$, where $\text{var}(\cdot)$ is variance of the dataset, and $|\mathbf{F}|$ is number of features, $C = 1$ (this heuristic is based on sci-kit library [103]). The rest of individuals is sampled in a random manner in range specified in the algorithm configuration. This initial population is then evaluated and N parents pair are selected for the crossover process (where $p_a \neq p_b$, p_a and p_b are chromosomes). It uses global-local selection described in Section 2.3.2 using always the global mode in order to provide a better exploration of search space and eliminate preliminary

convergence. This means that the population is always divided into two parts of equal size and from each part one chromosome is selected in a random manner.

The crossover operator creates a new chromosome (p_c). In total N chromosomes are created and a new population (P') is created (that has the same size the initial one). A new individual is created by using a heuristic crossover [60]. For each hyperparameter, the following equation is applied:

$$p_c = p_b + \alpha \cdot (p_a - p_b), \quad (3.1)$$

where α is a random value drawn from $[0.5, 1.5]$, p_a is a parent with higher fitness value compared to parent p_b . Although α was also proposed to be a constant value, the random one should bring more diverse children, hence enhancing the diversity of the population (even if the same pair is selected twice it can still produce different offspring). For $\alpha = 1$ the offspring will become the same as the parent p_a while for a value equal to zero it will become a copy of the parent p_b . Since GAs should pass on better genes to the next generation the range of random values should be symmetric with respect to 1. In this way, new individuals will be allowed to be in the space around the better of the selected parents.

Afterwards, each individual goes through the mutation process with a probability of P_m . This operator performs a form of local search. If the individual is selected for mutation for each of its values the following Equation is applied:

$$p_c = p_c + (p_c \cdot u \cdot \rho), \quad (3.2)$$

where ρ is a random sign with equal probability and u is a random percentage value drawn from range of 0 to β . The value of β needs to be relatively low and express the maximal percentage value of change. After applying all of the genetic operator's population of size $2 \cdot N$ goes through the selection process. This process selects the N fittest individual to keep the population size constant. The stop condition is based on the growth of the average fitness of the population in two subsequent generations. If this growth is smaller than the set threshold (l) there is little chance that the solution could be further improved and the algorithm ends.

3.2 Evolutionary algorithm for feature selection (EFS)

As argued in Lessmann et al. [80] this process could be viewed as a preprocessing step where possibly expert knowledge about the problem at hand could be used. On the other hand, feature selection is a complicated problem and is mutual dependent with both model optimization as well as training set selection. Both of those approaches were tested, where preprocessing is done by known in literature Recursive Feature Elimination with cross-validation (RFECV) [6] (described in Appendix .1). In the following section, an evolutionary algorithm for feature selection will be presented. This algorithm is used in the SE-SVM method 3.4.

This algorithm was inspired by the MASVM (Section 2.3.2). Feature selection could be seen as a problem that is similar to instance selection in some manner. It aims to find the reduced set of features to improve the performance of the classifier and get rid of noisy features. The instance selection (training set selection) has similar aims, the main difference comes from the reduction of computational complexity, where often it could become more important, for instance selection, to reduce training set size drastically in order to apply computationally complex methods. In the case of feature selection, it might not always be true (please note that this work considers datasets with $|\mathbf{T}| \gg |\mathbf{F}|$). Based on this observation, the same coding of chromosomes as in MASVM (see Section 2.3.2) will be used for the feature selection process.

The outline of the method is presented in Algorithm 3. In order to initialize the population, the algorithm starts with preprocessing of datasets and ranks $\mathbf{F}$. The preprocessing set (line 2) consists of running four distinctive algorithms: mutual information, variance thresholding, RFECV, and stability selection. The RFECV utilizes ExtraTree as a base model and returns a set of selected features which is converted into equal probabilities. So the whole vector of features' importance sums to 1. RFECV is presented in detail in Section .1. Variance thresholding computes the ANOVA F-value providing scores for each feature. The σ is then selected which means that only the top σ of features are selected and the rest gets a score equal to zero. Those scores are then rescaled to sum 1 (so they can be treated as probabilities). Mutual information is calculated for each feature and target

Algorithm 3 Pseudocode for evolutionary algorithm for feature selection.

```

1:  $\mathcal{F}_{\text{pool}} \leftarrow \emptyset$ 
2: FeatureProbability  $\leftarrow$  Preprocessing( $\mathbf{T}$ )
3:  $P \leftarrow$  RouletteWheelInitialization( $N$ , FeatureProbability)
4: for all  $p_i \in P$  do
5:    $\eta_i \leftarrow$  CalculateFitness( $p_i$ )
6: repeat
7:    $\mathcal{F}_{\text{pool}} \leftarrow$  UpdatePool( $P$ )
8:   Parents  $\leftarrow$  SelectParents( $P$ )
9:    $P' \leftarrow$  Crossover(Parents)
10:  Education( $P'$ ,  $\mathcal{F}_{\text{pool}}$ )
11:  Mutation( $P'$ )
12:   $P'_{SI} \leftarrow$  SuperIndividualsCreation( $\mathcal{F}_{\text{pool}}$ )
13:  for all  $p_i \in (P' \cup P'_{SI})$  do
14:     $\eta_i \leftarrow$  CalculateFitness( $p_i$ )
15:   $P \leftarrow$  Selection( $P \cup P'_{SI} \cup P'$ )
16:  Adaptation( $P$ )
17: until StopCondition
18: return Best( $P_{\text{best}}$ )

```

variable (the class to which the vector belongs). These scores always have positive values, and similar to previous methods, are normalized to sum to 1. Finally, the stability selection algorithm [86] uses logistic regression with L1 penalty to estimate feature importance. The values obtained by each of the algorithms are then averaged. These scores are used as probabilities in roulette wheel initialization of feature sets, where K_f features are selected (in line 3). On initialization, only the top τ of features are considered for getting into the chromosome. This should help to provide better individuals at the beginning which is crucial for running simultaneous evolution of training and feature set, and SVM hyperparameters ($\mathcal{M}$) (see Section 3.4)

Afterwards, the fitness of all chromosomes in the population is assessed (line 5) by training SVM classifiers and calculating the confusion matrix on $\mathbf{V}$. The N pairs are selected for crossover using the local-global adaptation scheme (the same as in MASVM algorithm). The population is divided with ϵ into two parts (based on fitness values). The algorithm starts in global mode meaning that parents in each pair come from different parts of population.

Then, for each pair crossover operator is applied. It works by summing up feature sets ($F_c \leftarrow F_a \cup F_b$). As a summation of sets may lead to uncontrolled growth in the size of features selected up to K_f are drawn randomly. Although initially all of the chromosomes have the same size of K_f in later stages of algorithms this changes. As K_f is adapted dynamically, there could be individuals that differ in the size of the feature set. All of those newly created chromosomes create a new population (P').

After P' is created a memetic operation of education is performed. It provides better exploitation of information about the previous individual to enhance the current solution. This information comes from a pool of features that are associated with high-ranked individuals. The pool of features is created and updated (in line 7) by taking λ of the best individual in the population and creating a histogram of features (counting how many times a given feature is present in the population). Only features that occur more than once in the histogram are left in $\mathcal{F}_{\text{pool}}$. Once the pool is created, education is performed with a probability of E_p and replaces E_r of features to the ones that are in $\mathcal{F}_{\text{pool}}$.

Next, each individual is mutated with a probability of M_p . Mutation works by replacing M_r features with ones that are above mean in ranking used for initialization. The initial ranking is used to introduce new features into the population but tries to not add any noise or irrelevant features.

Another memetic operation is super individual creation. These individuals are created from pools of features. Up to, K_f features are selected for those individuals in a random manner and create $\alpha \cdot W$ new chromosomes. In the next step, all newly created individuals are evaluated.

The N fittest individuals are eventually selected to maintain a steady size of the population (line 15). In the end, K_f is adaptively grown based on current fitness improvement. The evolution process continues until there is no improvement in the average fitness of the population greater than the $\eta_{\min}$.

3.3 Alternating algorithm for optimization of SVM hyperparameters and training set selection

Tuning the SVM model is tricky when the training set is to be reduced, because of the mutual dependence—the SVM hyperparameters are usually required to select $\mathbf{T}'$, while depending on the subset used for training, a different SVM model (set of hyperparameters) might be optimal. Previously the main problem of either GASVM or MASVM was the need to setup the hyperparameters of the SVM model. In previous works, the hyperparameters were determined through a GS, which is a computationally intensive process. Consequently, the practicality of the training set selection algorithm was limited, despite its primary objective of enhancing SVM’s usability on large datasets. The algorithm presented in this section was introduced to test the hypothesis that $\mathcal{M}$ optimization can yield good results in combination with training set selection when compared to running the training set selection algorithm with parameters found by GS

The first solution to test this hypothesis was ALGA—**AL**ternating **Genetic Algorithm** for selecting the SVM model and refining the training set [75] (refining means selecting $\mathbf{T}'$). The main concept of this algorithm is to run one optimization at a time but use the results propagated from the other optimization phase. When the current best solution could not be improved further, the algorithm changes its mode. This consists of saving the current best solution and switching to optimize either $\mathcal{M}$ or $\mathbf{T}'$. For the model optimization phase, GAHP described in Section 3.1 is used while training set optimization is based on GASVM algorithm [74]. Although the algorithm provides a modular approach to the problem, the training set selection process was afterward changed to improved MASVM (see Section 2.3.2), which does not need to setup desired size of the training set beforehand. This improved algorithm is designated as ALMA from **AL**ternating **Memetic Algorithm**.

The pseudocode of ALGA/ALMA is given in Algorithm 4. In the beginning (line 1) the population for $\mathbf{T}'$ (designated as P) is generated by randomly drawing vectors from the training set. This population is evaluated with predefined kernel hyperparameters in lines 3-5. Simultaneously population representing SVM hyperparameters (Q) is created (line 2). After fitness values are obtained for those initial

hyperparameters, the best individual in P is selected (p_{Best}). Starting in line 8, the algorithm switches to model optimization phase. First of all the whole population Q is evaluated. What is important evaluation is performed on $\mathbf{T}'$ (p_{Best}). This helps to save a lot of computation time. In each iteration, the best-performing hyperparameters set is saved (line 11). In line 12, all of the genetic operators are applied, a new population is created, evaluated and the selection process is performed (all of those operations are from GAHP algorithm). The model optimization phase iterates until the algorithm used for model optimization stops. In the case of GAHP, the stop condition is based on the growth of the average fitness of all individuals in the population by more than η_{min} . So local stop condition is based on the optimization method selected. It is used to decide when to switch the optimization phases.

Afterwards algorithm switches its mode to optimize $\mathbf{T}'$ (lines 15-20). The same logic is applied here, first, the whole population is evaluated using the best set of hyperparameters from the previous phase (q_{Best}). Currently best $\mathbf{T}'$ is saved in p_{Best} after which one iteration of applying genetic operators of GASVM (or MASVM in case of ALMA) is performed. The local stop condition is the same as with the $\mathcal{M}$ optimization phase (so both phases use the same local stop condition). This alternating process is repeated as long as at least one of two subsequent phases manages to improve the average fitness. This means that the algorithm needs to run at least model optimization phase and the instance selection process (phases goes: $\mathcal{M} \rightarrow \mathbf{T}'$). Otherwise, if any of the phases cannot further improve the solution, the global stop condition is reached and the SVM trained with the best individuals ($p_{\text{Best}}, q_{\text{Best}}$) is returned.

This algorithm was first presented in MCPR conference [75] and was later extended to include the feature selection process. In the beginning, the feature selection was done in a preprocessing step which filtered the feature set. Afterwards, the alternating process was extended to add the third phase with the evolutionary feature selection algorithm presented in Section 3.2. This extended alternating algorithm was published in GECCO conference [36]. All of that development led to creation of the simultaneous optimization algorithm (presented in the next section).

Algorithm 4 Alternating genetic algorithm for optimization of SVM hyperparameters and training set selection

```

1: Initialize population of  $\mathbf{T}'$  ( $P$ ) of size  $N$ 
2: Initialize population of  $\mathcal{M}$  ( $Q$ ) of size  $M$ 
3:  $q_{init} \leftarrow \text{GetDefault}(Q)$ 
4: for all  $p_i \in P$  do
5:    $\eta_i \leftarrow \text{CalculateFitness}(p_i, q_{init})$ 
6:  $p_{\text{Best}} \leftarrow \text{SelectBest}(P)$   $\triangleright$  individual with the highest fitness
7: repeat
8:   repeat  $\triangleright \mathcal{M}$  optimization phase
9:     for all  $q_i \in Q$  do
10:       $\eta_i \leftarrow \text{CalculateFitness}(p^B, q_i)$ 
11:       $q_{\text{Best}} \leftarrow \text{SelectBest}(Q)$ 
12:       $Q \leftarrow \text{IterateModelOptimization}(Q)$ 
13:   until LocalStopCondition
14:   if  $\neg \text{GlobalStopCondition}$  then
15:     repeat  $\triangleright \mathbf{T}'$  optimization phase
16:       for all  $p_i \in P$  do
17:         $\eta_i \leftarrow \text{CalculateFitness}(p_i, q^B)$ 
18:         $p_{\text{Best}} \leftarrow \text{SelectBest}(P)$ 
19:         $P \leftarrow \text{IterateTrainingSetOptimization}(P)$ 
20:     until LocalStopCondition
21: until GlobalStopCondition
22: return ( $p_{\text{Best}}, q_{\text{Best}}$ )

```

3.4 Simultaneous optimization of training and feature set and SVM hyperparameters

The problem of optimizing training data ($\mathbf{T}'$ and $\mathbf{F}'$) and $\mathcal{M}$ are mutually dependent on each other. Hence the, previously presented alternating approach might not be an optimal solution to that problem. It could be easily imagined that performing feature set optimization under suboptimal $\mathbf{T}'$ or $\mathcal{M}$ could lead to obtaining results that are stuck in local optima for $\mathbf{F}'$ (as the search space is very large) or could in extreme cases deteriorate optimization and lead to poor performance of such model (where first bad $\mathcal{M}$ is selected which impact subsequent phases of the algorithm). Therefore, an algorithm to simultaneously optimize all

three components (the $\mathbf{T}'$ and $\mathbf{F}'$ set and $\mathcal{M}$) is proposed and denoted as SE-SVM—Simultaneous **E**volutionary optimization of **S**VM. This algorithm is introduced to prove the first research hypothesis that such simultaneous optimization could provide improved training and classification time without affecting classification quality. This means that during the tests this algorithm should provide pareto-optimal solution regarding the training and classification times coupled with proper classification metrics. By improving upon previously shown alternating scheme and adding feature selection process it is also hypothesised that it should improve classification quality in relation to ALMA method. One of the advantages of this algorithm is starting with relatively small training sets (thanks to training set selection) and small feature sets. This should help to quickly evaluate initial solutions and explore more prominent regions of search space. Moreover, the ability to adapt (growth) the size of selected $\mathbf{T}'$ and $\mathbf{F}'$ should keep the good quality of classification. However, adding feature selection could increase the computation time compared to ALMA or MASVM methods, so it is not expected to outperform these methods in absolute computation time but rather provide pareto-optimal solution.

The design of the chromosome for SE-SVM is shown in Figure 3.2. There are three distinct parts. The hyperparameters and kernel type make up the first part, which has a constant size and can hold any number of arbitrary selections for the kernel type. The latter two parts consist of sets for training vectors and features. The IDs are representing numbers of selected rows (samples) and columns (features), and are maintained unique throughout the evolution process. IDs are being used instead of binary encoding due to the impracticality for large datasets. Furthermore, those sets are expected to grow during the evolution process starting from some small values of K_f for $\mathbf{F}$ and K_t for $\mathbf{T}$. Hence the size of each chromosome is dynamic and can differ as K_f and K_t can grow independently. Note that not even all parts of $\mathbf{T}'$ or $\mathbf{F}'$ in a given population need to have the same size.

Algorithm 5 Pseudocode for simultaneous evolutionary optimization of SVM algorithm

```

1:  $P_{\text{best}} \leftarrow \emptyset, \mathcal{S}_{\text{pool}} \leftarrow \emptyset, \mathcal{F}_{\text{pool}} \leftarrow \emptyset$ 
2: Initialize population  $P$  of size  $N$ 
3: for  $i = 0$  to  $5$  do
4:   for all  $p_i \in P$  do
5:      $\eta_i \leftarrow \text{CalculateFitness}(p_i)$ 
6:   repeat
7:      $\mathcal{S}_{\text{pool}}, \mathcal{F}_{\text{pool}} \leftarrow \text{UpdatePools}(P)$ 
8:      $\text{Parents} \leftarrow \text{SelectParents}(P)$ 
9:      $P' \leftarrow \text{Crossover}(\text{Parents})$ 
10:     $\text{Education}(P^t, P^f, \mathcal{S}_{\text{pool}}, \mathcal{F}_{\text{pool}})$ 
11:     $\text{Mutation}(P')$ 
12:     $P'_{SI} \leftarrow \text{SuperIndividualsCreation}(\mathcal{S}_{\text{pool}}, \mathcal{F}_{\text{pool}})$ 
13:    for all  $p_i \in (P \cup P'_{SI})$  do
14:       $\eta_i \leftarrow \text{CalculateFitness}(p_i)$ 
15:     $P \leftarrow \text{Selection}(P \cup P'_{SI} \cup P')$ 
16:     $\text{Adaptation}(P)$ 
17:  until StopCondition
18:   $P_{\text{best}} \leftarrow P_{\text{best}} \cup \text{GetBest}(P)$ 
19:   $P \leftarrow \text{RegeneratePopulation}$ 
20: return  $\text{Best}(P_{\text{best}})$ 

```

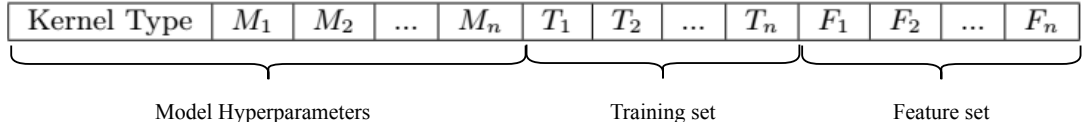


Figure 3.2: The design of chromosome for SE-SVM method. Source: [37].

The pseudocode of this algorithm is presented in Algorithm 5. Although there is only a single population, the parts of chromosomes population regarding $\mathcal{M}$ will be referred to with P^m , for $\mathbf{T}'$ as P^t and for $\mathbf{F}'$ as P^f .

The algorithm starts by creating a new population (line 2) using different initialization methods for each part of the chromosome. As this algorithm is built over previous solutions the details of each operator could be found in proper sections: for kernel evolution GAHP is utilized (see Section 3.1), for training set selection the MASVM is used (see Section 2.3.2) and for feature selection the EFS method

is used (see Section 2.3.3). Afterwards, the population is evaluated and each individual has their fitness value calculated. In line 7, the pools of features and SVs are being updated based on the current population. These pools hold information about the features ($\mathcal{F}_{\text{pool}}$) and vectors ($\mathcal{S}_{\text{pool}}$) that are discovered to be valuable during evolutionary process, and later, help to exploit knowledge learned while running computations. During the first iteration, this is done based on the fitness and SVs of the initial population. In the next step, parents are selected using a local-global adaptation scheme (Section 2.3.2). This works similar to EFS and MASVM, the algorithm starts in global mode. The population is divided into two parts, by sorting individual based on their fitness and ϵ threshold. In global model parents are selected from different part of populations while in local mode both are selected within the same group. The crossover operator (line 9), creates N new individual which are joined into a new population (P') of the same size as the original one. Here proper operators are applied over each part of the chromosome. The kernel type is kept constant during the evolution process where the hyperparameters are crossed over using Equation 3.1 and sets of $\mathbf{T}'$ and $\mathbf{F}'$ from two individuals are summed separately. The size of these sets are limited by the current values of K_t for $\mathbf{T}'$ and K_f for $\mathbf{F}'$. If either of those sets growth bigger, then a random selection process is employed to maintain proper size. In the next line, the parts of the population representing $\mathbf{T}'$ and $\mathbf{F}'$ (P^t and P^f respectively) are going through the operation of education. This operation involves using previous information learnt during the evolution to improve the solution. For each type of set a pool of promising vectors (built-up from SVs— $\mathcal{S}_{\text{pool}}$) and features (built by creating a histogram of features used in best-fitted individuals— $\mathcal{F}_{\text{pool}}$) is used to replace vectors and features present in the chromosome. Afterwards, in line 11 whole new population goes through the process of mutation. When a chromosome is chosen for mutation, all three parts undergo this process, and each segment (part of chromosome) employs a uniquely designed operator for $\mathcal{M}$ (GAHP), $\mathbf{T}'$ (MASVM), and $\mathbf{F}'$ (EFS). Before the evaluation and selection process, so-called super individuals are created. This process selects up to K_f and K_t from the $\mathcal{F}_{\text{pool}}$ and $\mathcal{S}_{\text{pool}}$ for $\mathbf{F}'$ and $\mathbf{T}'$ respectively. For the model hyperparameters, the current best solution is selected and added to all new super individuals, so they share common $\mathcal{M}$. In the next step, all of the newly created individuals are trained and

their fitness is calculated (based on $\mathbf{V}$). Training of SVM models involves selecting proper training vectors based on $\mathbf{T}'$ of chromosome, as well as filtering the set of features based on $\mathbf{F}'$. Each SVM should have a unique training set. The selection process keeps the constant size of the population by selecting N fittest individuals. At the end of each loop the K_f and K_t can be grown adaptively and the mode for parent selection could be adjusted. The process of evolution lasts till there is no improvement in the average fitness of the population. Here the process of regeneration is introduced once again and always R such regeneration are performed. Before each regeneration the current best solution is saved and stored in P_{best} . The result of the algorithm is the best model from entire evolutionary process selected from P_{best} .

3.5 Adaptive RBF kernel

Although the previous solutions already optimized training data and SVM hyperparameters, it may be hypothesized that a custom kernel function can further improve the SVM capabilities—this observation is built upon visualization of multiple SVM models using 2D datasets where different values of γ (used in RBF kernel) provided insights into how far given SV is affecting the decision hyperplane. Especially in the start of evolutionary algorithm, like ALMA or SE-SVM, where there were only K_t vectors selected most of those SVs provided classification information only in their neighborhood. Thus, this observation led to hypothesis that selecting different kernel hyperparameters for different training vectors may help better reflect the subtle characteristics of the space while determining the hyperplane. This of course brings quite a big optimization problem where the number of hyperparameters is equal to $|\mathbf{T}|$. In this approach, the $\mathbf{T}'$ optimization is joined with $\mathcal{M}$ in a form of a single algorithm, as each training vector is coupled with its own γ value for the RBF kernel. This algorithm focuses on RBF kernels, given as $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp(-\gamma|\mathbf{x} - \mathbf{x}'|^2)$ and have a hyperparameter γ because of its popularity and previous performance showed in literature [87].

It can be observed that the value of the RBF function is dependent on the Euclidean distance between $\mathbf{x}_i$ and $\mathbf{x}_j$, where the closer the vectors are in the input space, the larger the value of the kernel becomes. The hyperparameter γ

is used to control the “range” of influence of SVs. By attaching γ to training vectors, a much more detailed hyperplane could be produced. This situation is presented in Figure 3.3 where an example of using different γ values coupled to the $\mathbf{T}$ vectors that are selected as SVs is rendered. As presented with the orange SVs in Figure 3.3b (those vectors have $\gamma = 10^5$) the larger value of this hyperparameter results in a small radius of influence on decision hyperplane. What is more, those SVs are located near the decision boundary in regions that could be considered “difficult” to classify. These regions could also be characterized as ones where vectors from $\mathbf{T}$ lay close to one another. The SVs with relatively large gamma values provide fine details about this “local” part of the dataset. Contrary to them SVs with small γ ’s (blue and red ones) tend to lay further from the decision boundary and appear in more homogeneous regions of the dataset which can be considered as “easier” for correct classification.

Please note that the number of SVs is just a small subset of the training data (in Figure 3.3 each pixel is a vector in the dataset, so presented white and black dots actually includes tens of unique data vectors while SVs are just single pixels). This sample proved that SVMs could be further improved by not only evolving $\mathbf{T}'$ and its $\mathcal{M}$ but also joining it with an adaptive kernel. This new method is called **SVMs with Adaptive RBF kernels**—ARBF-SVM.

As discussed in Section 2.1.2 the kernel function needs to fulfil Mercer’s conditions [61]. This could also be stated that the kernel matrix (or Gram matrix) must always be positive semi-definite—PSD (also the kernel function needs to be symmetric). The work presented here provides an experimental approach, where those requirements might not always be satisfied. However, it does not mean that such SVM model would not work or SVM training could not be done. By allowing relaxation of the conditions we lose the guarantee that provided decision boundary will have the maximal possible margin. This is due to the fact that the optimization problem might no longer be a convex problem, hence local minimum could be different from the global minimum. The fulfilment of those conditions was tested by checking during the evaluation process in the mentioned algorithm if all of the kernel matrices were PSD. Although it was not true in all of the cases, the Gram matrix was PSD in the majority of cases. As shown in practice (see Section 4.3.2) this relaxation of conditions for kernel function does not deteriorate

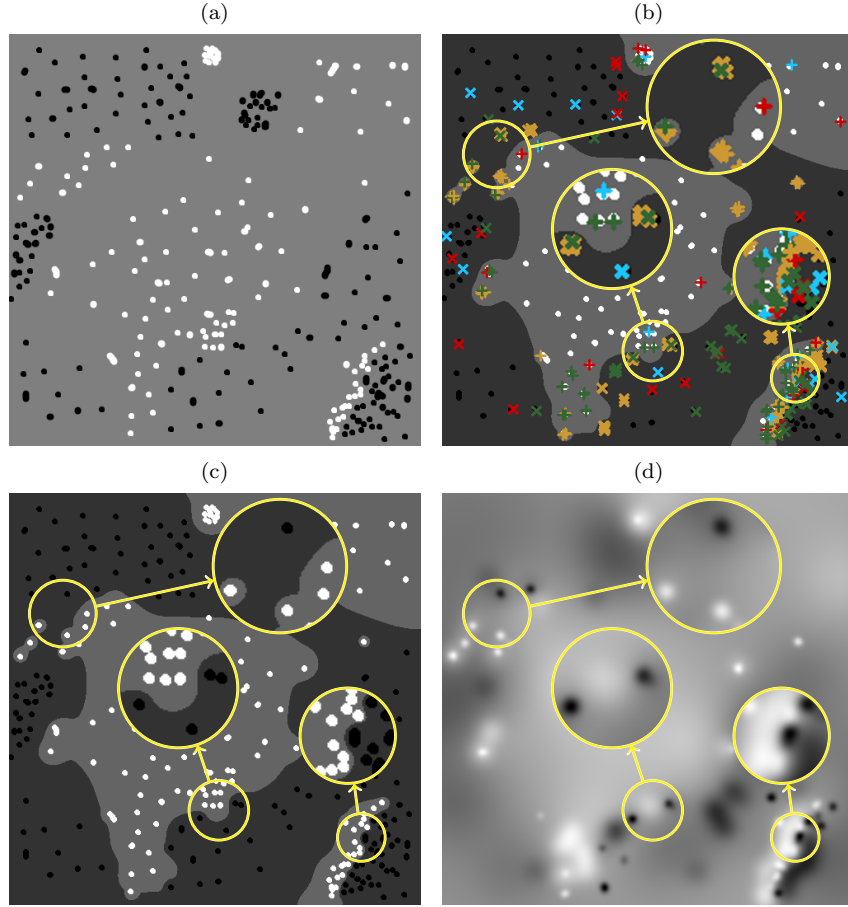


Figure 3.3: Assigning different γ 's in the RBF kernel to different $\mathbf{T}$ vectors can help better “model” the peculiarities of the SVM hyperplane: (a) our example set (we consider binary classification, white and black pixels present examples from two different classes), (b) the crosses in different colours render SVs with different γ 's: $\gamma = 10^2$ (blue), $\gamma = 10^3$ (red), $\gamma = 10^4$ (green), and $\gamma = 10^5$ (orange), together with (c) the different shades of gray show the decision boundary, (d) the “range of influence” of all SVs (the brighter/darker the pixel is, the distance to the hyperplane is larger). The SVM regularization parameter was grid-searched and was $C = 10$. This figure comes from our paper [93]

the performance of the final classifier.

One more important step in the problem of capturing fine-grained characteristics of the training set using different γ values assigned to the $\mathbf{T}$ vectors is how to aggregate those γ 's during the calculation of the kernel matrix. Therefore, it is important to determine a suitable aggregation function to calculate the kernel

value between $\mathbf{x}_i$ and $\mathbf{x}_j$, since they may have different values of γ_i and γ_j . The following aggregation functions are examined, resulting in various versions of the evolutionary algorithm being proposed (they will be discussed later in this section):

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma_i \|\mathbf{x}_i - \mathbf{x}_j\|^2} \quad \gamma\text{-One} \quad (3.3)$$

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = e^{-[(\gamma_i + \gamma_j)/2] \|\mathbf{x}_i - \mathbf{x}_j\|^2} \quad \gamma\text{-Avg} \quad (3.4)$$

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = e^{-\max(\gamma_i, \gamma_j) \|\mathbf{x}_i - \mathbf{x}_j\|^2} \quad \gamma\text{-Max} \quad (3.5)$$

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = e^{-\min(\gamma_i, \gamma_j) \|\mathbf{x}_i - \mathbf{x}_j\|^2} \quad \gamma\text{-Min} \quad (3.6)$$

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma_i \|\mathbf{x}_i - \mathbf{x}_j\|^2} + e^{-\gamma_j \|\mathbf{x}_i - \mathbf{x}_j\|^2} \quad \gamma\text{-Sum} \quad (3.7)$$

It should be noted that in the γ -One variant, a single γ is selected and therefore, aggregation of these hyperparameters is not carried out, as demonstrated in the our previous work regarding adaptive kernels [93]. Keep in mind that the γ could be interpreted as the range of influence of a given SV. Finally, once the training of an SVM is completed, the calculation of the kernel for the new test sample ($\mathbf{x}_{\text{test}}$) is performed in the following manner.

$$\mathcal{K}(\mathbf{x}_{\text{SV}}, \mathbf{x}_{\text{test}}) = e^{-\gamma_{\text{SV}} \|\mathbf{x}_{\text{SV}} - \mathbf{x}_{\text{test}}\|^2}, \quad (3.8)$$

where $\mathbf{x}_{\text{SV}}$ is an SV and γ_{SV} is γ value associated with this SV.

The algorithm starts with an initial preprocessing step performed over a random balanced subset (of size $c \cdot K_t$) of the entire training set $\mathbf{T}$, where c represents the number of classes in the input dataset, K_t is a hyperparameter of the method ($K \leq |\mathbf{T}_{\min}|$), and $|\mathbf{T}_{\min}|$ is the number of training vectors in the least numerous class. The goal is to obtain a single C value (that will remain unchanged throughout the optimization process), together with γ that will be used to generate a set of γ 's which are used in the evolutionary process (Algorithm 6 line 4).

For each $\gamma_i \in \vec{\gamma}$, an initial population P of N individuals is generated (p_j , $j = 1, 2, \dots, N$) which is evolved (line 20). Each chromosome p_j encompasses a (potentially imbalanced) subset of $\min\{K, |\mathbf{T}_c|\}$ training vectors, alongside their gamma values set to γ_i , randomly sampled from $\mathbf{T}$ for each class c . Once the

Algorithm 6 Evolving reduced training sets with adaptive γ values using DA-SVM.

```

1: function FINDGAMMAANDC
2:    $(C, \gamma) \leftarrow$  Grid search over a random  $\mathbf{T}'$  of size  $c \cdot K$ 
3:    $\vec{\gamma} \leftarrow \{\gamma/10, \gamma, 10 \cdot \gamma, 100 \cdot \gamma, 1000 \cdot \gamma\}$ 
4:   return  $\vec{\gamma}, C$ 

5: function EVOLVET( $P, \mathcal{S}_{\text{best}}$ )
6:    $\mathcal{S}_{\text{pool}} \leftarrow \emptyset$ 
7:   while termination condition not met do
8:      $P' \leftarrow$  Crossover( $P$ )
9:      $P' \leftarrow$  Educate( $P'$ )
10:     $P' \leftarrow$  Mutate( $P'$ )
11:     $P' \leftarrow$  Calculate fitness( $P', \mathcal{S}_{\text{best}}$ )
12:     $\mathcal{S}_{\text{pool}} \leftarrow$  Update SV pool( $P'$ )
13:     $P^{\text{SI}} \leftarrow$  Create super individuals( $\mathcal{S}_{\text{pool}}$ )
14:     $P \leftarrow$  Post select( $P, P', P^{\text{SI}}$ )
15:     $P'_{\text{best}} \leftarrow$  Find best individual( $P$ )
16:    Adapt( $P$ )
17:   return  $P, P'_{\text{best}}$ 

18: function OPTIMIZE( $\vec{\gamma}, C, P'_{\text{best}}, P_{\text{best}}, \mathcal{S}_{\text{best}}$ )
19:   for all  $\gamma_i$  in  $\vec{\gamma}$  do  $\blacktriangleright$   $\gamma$ 's sorted ascendingly
20:      $P \leftarrow$  Generate( $N, C, \gamma_i, \mathbf{T}$ )
21:      $P'_{\text{best}} \leftarrow$  Find best individual( $P$ )
22:     if  $\eta(P'_{\text{best}}) > \eta(P_{\text{best}})$  then
23:        $P, P'_{\text{best}} \leftarrow$  EVOLVET( $P, \mathcal{S}_{\text{best}}$ )
24:       if  $\eta(P'_{\text{best}}) > \eta(P_{\text{best}})$  then
25:         Add SV( $P'_{\text{best}}$ ) to  $\mathcal{S}_{\text{best}}$ 
26:          $\mathbf{T} \leftarrow$  Shrink( $\mathbf{T}, P$ )
27:          $P_{\text{best}} \leftarrow P'_{\text{best}}$ 
28:     Reset LGA,  $\mathcal{S}_{\text{pool}} \leftarrow \emptyset$ 
29:   return  $P_{\text{best}}$ 

30: function MAIN
31:    $P'_{\text{best}} \leftarrow \emptyset, P_{\text{best}} \leftarrow \emptyset, \mathcal{S}_{\text{best}} \leftarrow \emptyset$ 
32:    $\vec{\gamma}, C \leftarrow$  FINDGAMMAANDC
33:   OPTIMIZE( $\vec{\gamma}, C, P'_{\text{best}}, P_{\text{best}}, \mathcal{S}_{\text{best}}$ )

```

population is created, the *fitness* of all individuals is calculated. For each p_j , an SVM is trained using a combined set of $\mathbf{T}'_j$ and $\mathcal{S}_{\text{best}}$ vectors, where $\mathbf{T}'_j$ is the encoded reduced set of p_j , and $\mathcal{S}_{\text{best}}$ is a set of all vectors that have been selected as SVs for the *best* individuals in the previous generations (for the first γ_i $\mathcal{S}_{\text{best}} = \emptyset$).

The fitness η_j of the j -th chromosome is calculated as AUC obtained using the corresponding SVM over $\mathbf{V}$. If η of the best individual P'_{best} (line 21) is greater than η of the best individual P_{best} evolved so far (line 22), appending more $\mathbf{T}$ vectors with larger γ 's will help enhance the abilities of the SVM through more fine-grained modelling of the hyperplane. In that case, the refined sets are optimized using a memetic algorithm [95] (line 23). Otherwise, γ_i is skipped, and the algorithm jumps to the next γ .

Note that calculating the fitness value requires training an SVM using the corresponding $\mathbf{T}'$, therefore this algorithm is a *wrapper approach*. Since the training process is the most time-consuming part of the entire algorithm, it may be considered as an important drawback of the technique (although the reduced training sets are kept small). To further tackle this issue, the No-T variant is proposed in which training of an SVM is *not* performed for calculating the fitness of an individual. Instead, *all training vectors are selected as SVs*, hence build upon observation that the evolved training sets should encompass only important training vectors that are likely to be SVs. This approach may, however, slow down the classification process of an underlying SVM which is in a linear relation with the number of SVs.

Once the evolution over the i -th gamma is finished, it is verified if the fitness of the best individual from the final population is greater than the fitness of the best solution already found (line 22). If so, the SVs (together with the corresponding γ 's) of the fittest individual are added to $\mathcal{S}_{\text{best}}$, being the pool of SVs of the best individuals (line 25). These vector are removed from $\mathbf{T}$ which becomes $\mathbf{T} \leftarrow \mathbf{T} - \text{SV}(P'_{\text{best}})$, and—additionally—the training set is shrunk and the $\mathbf{T}$ vectors that have been correctly classified by all individuals in the population are removed (line 26), as they are often positioned far from the decision hyperplane. This reduction process can considerably decrease the size of $\mathbf{T}$, hence enhance the convergence abilities of the evolutionary optimization, because discarded examples are unlikely to be picked as SVs. It, however, may lead to extremely imbalanced

$\mathbf{T}$'s (even containing single-class vectors). To tackle this issue, the $\mathcal{S}_{\text{best}}$ vectors are incorporated in the final reduced training sets. Finally, the best individual P_{best} is returned (line 29).

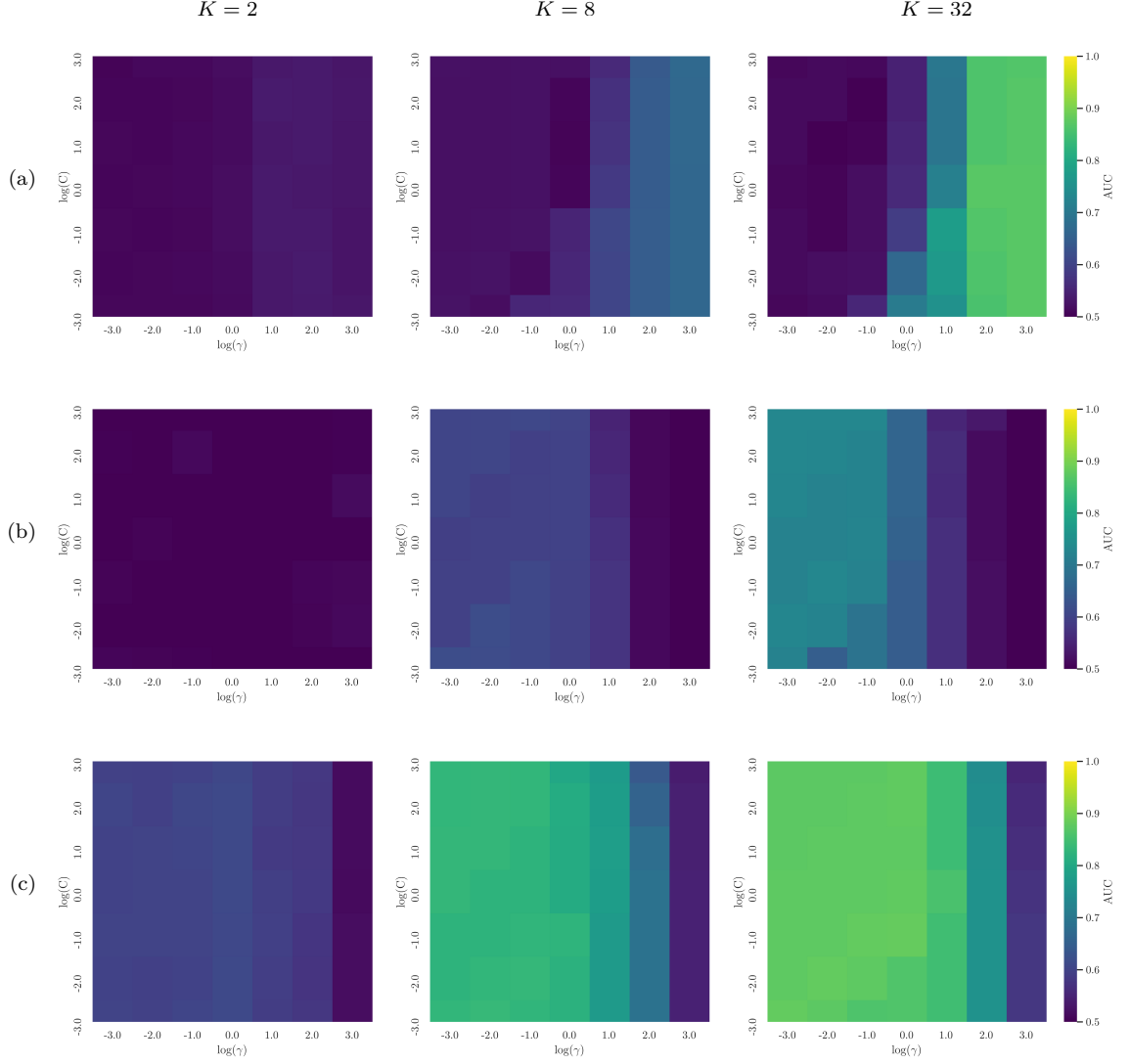


Figure 3.4: The impact of the K value on the quality of hyperparameter configurations extracted for example datasets: (a) 2D-Blobs, (b) german, (c) spambase. For the details of these sets (their sizes and characteristics), see Section 4.1. This figure comes from our paper [39]

This preliminary grid search (with a logarithmic step)¹ is based upon the ob-

¹Note that the grid search may be easily replaced with other algorithms, e.g., a selected

servation that the small subsets of the entire training set are often representative enough to locate high-quality hyperparameter configurations [76]. In Figure 3.4, the impact of the K values is visualized on the estimated quality of the hyperparameter configuration. For the performance measure the area under receiver operating characteristic curve (AUC) is calculated for the *validation* set ($\mathbf{V}$) using an SVM trained with a selected refined set. The three example datasets are selected for this investigation (the details of these sets are reported in Section 4.1). What can be appreciated is that $K = 8$, being the value of K used in as default value in the algorithm, is enough to observe the dependency between the (γ, C) pairs and the corresponding AUC obtained using an SVM trained over such reduced sets (note that $K = 2$ may be too small for some datasets, e.g., 2D-Blobs in this example). On the other hand, larger K 's, here $K = 32$, do not bring significant improvements in the estimation quality of the underlying hyperparameter pairs (the shape of the solution space in Figure 3.4 remains similar to $K = 8$), but adversely affect the SVM training time.

This initial method was refined and improved by DA-SVM method, where different kernel functions are combined in a single SVM. Figure 3.5 renders an example dataset (2D-LinearPool) which could benefit from exploiting a linear kernel in combination with the RBF one. Although an SVM with an adaptive RBF kernel (ARBF-SVM [93]) is able to effectively elaborate a high-quality decision hyperplane, the number of SVs is significantly larger than for a classifier which initially determines a linear hyperplane, and then fine-tunes it with RBF kernels. Also, since the hyperplane seems less “overfitted” to the training set, it could potentially deliver better generalization.

Joining of the kernels is based on the observation that the kernels can be divided into the *global* and *local* ones [123]. In the former case, data points that are far away from the test point may have a significant effect on the kernel value (as in e.g., the linear kernel), whereas in the latter kernels, only those vectors which lay close to the test example impact the kernel value (as in e.g., the RBF kernel). Hence, the best way to proceed is to elaborate an “approximate” decision hyperplane using the linear kernel, and locally improve its shape with the use of adaptive RBF.

metaheuristics. It was investigated in our publication [39] but exchanging this algorithm did not change the performance of the algorithm.

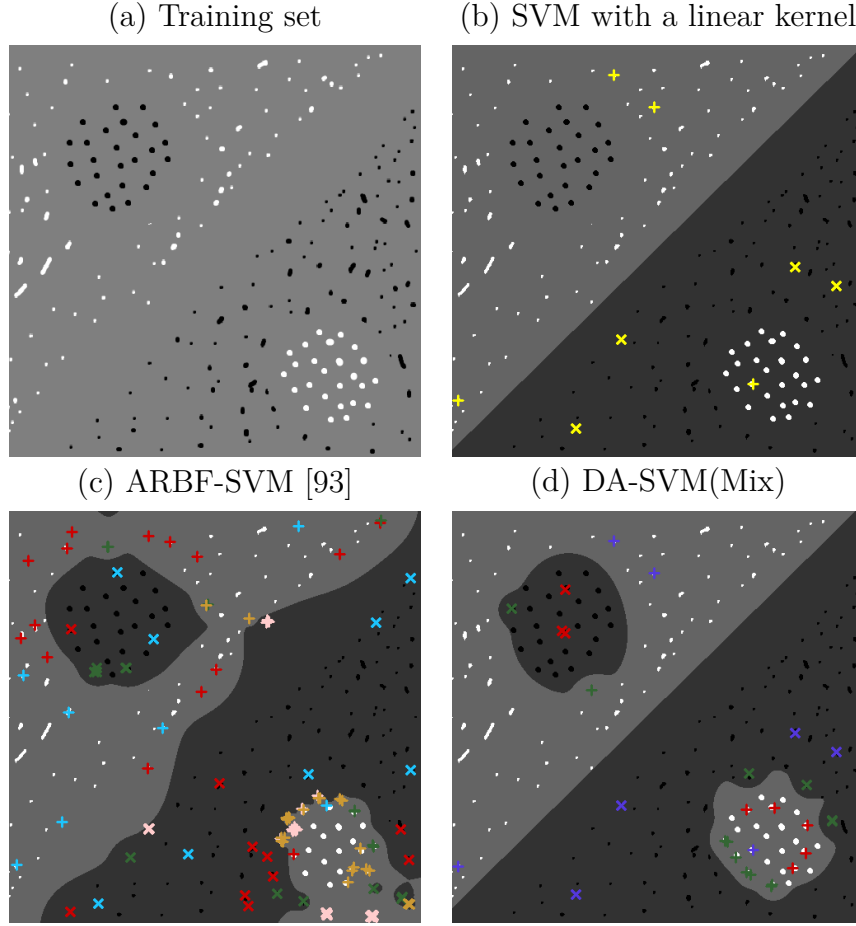


Figure 3.5: Example hyperplanes extracted using various methods show that mixing the linear and RBF kernels may help obtain “less complicated” hyperplanes (hence possibly not overfitted to $\mathbf{T}$). We visualize (a) an example training set (with white and black dots presenting two-class vectors), (b) a linear-kernel SVM (with yellow crosses showing the SVs), (c) a hyperplane obtained by ARBF-SVM [93] and (d) DA-SVM(Mix). In (d), the purple crosses render the SVs obtained for the linear kernel while the others visualize the SVs for various γ ’s for the RBF kernel. This figure comes from our paper [39]

The extension of the previous approach to incorporate this design of joining kernels is presented in Algorithm 7. For the linear kernel, the fitness of the individuals is quantified by using the balanced accuracy (instead of AUC), in which both sensitivity and specificity equally contributes to η , and can be safely used for imbalanced classification [48] (line 13).

Algorithm 7 Combining the linear and RBF kernels in a single SVM using DA-SVM(Mix).

```

1: function FINDGAMMA( $\mathcal{S}_{\text{best}}, C$ )
2:    $\vec{\gamma} \leftarrow \{0.001, 0.01, 0.1, 1, 10, 100, 1000\}$ 
3:    $\vec{\gamma}_\eta \leftarrow \emptyset$ 
4:   for all  $\gamma_i$  in  $\vec{\gamma}$  do
5:      $P \leftarrow \text{Generate}(N, C, \gamma_i, \mathbf{T}, \mathcal{S}_{\text{best}})$ 
6:      $P'_{\text{best}} \leftarrow \text{Find best individual}(P)$ 
7:      $\vec{\gamma}_\eta \leftarrow \vec{\gamma}_\eta \cup \eta(P'_{\text{best}})$ 
8:    $\gamma \leftarrow \vec{\gamma}[\text{index}_{\text{max}}(\vec{\gamma}_\eta)]$ 
9:    $\vec{\gamma} \leftarrow \{\gamma/5, \gamma, 10 \cdot \gamma, 50 \cdot \gamma\}$ 
10:  return  $\vec{\gamma}$ 

11: function MAIN
12:    $P'_{\text{best}} \leftarrow \emptyset, P_{\text{best}} \leftarrow \emptyset, \mathcal{S}_{\text{pool}} \leftarrow \emptyset, \mathcal{S}_{\text{best}} \leftarrow \emptyset$ 
13:   Metric = Balanced Acc
14:    $C \leftarrow$  Grid search over a random  $\mathbf{T}'$  of size  $c \cdot K$ 
15:    $P \leftarrow \text{Generate}(N, C, \mathbf{T})$ 
16:    $P, P'_{\text{best}} \leftarrow \text{EVOLVET}(P, \mathcal{S}_{\text{best}})$   $\blacktriangleright$  as in Algorithm 6
17:   Add SV( $P'_{\text{best}}$ ) to  $\mathcal{S}_{\text{best}}$ 
18:    $\mathbf{T} \leftarrow \text{Shrink}(\mathbf{T}, P'_{\text{best}})$ 
19:    $P_{\text{best}} \leftarrow P'_{\text{best}}$ 
20:   Metric = AUC,  $K = 2 \cdot K$ 
21:    $\vec{\gamma} \leftarrow \text{FINDGAMMA}(\mathcal{S}_{\text{best}}, C)$ 
22:  return OPTIMIZE( $\vec{\gamma}, C, P'_{\text{best}}, P_{\text{best}}, \mathcal{S}_{\text{best}}$ )  $\blacktriangleright$  as in Algorithm 6

```

The initial pre-processing is performed using grid search to optimize C , and evolve a population of refined training sets with linear kernels *only*. When the evolution ends, all SVs with the linear kernel is stored in $\mathcal{S}_{\text{best}}$ and utilized later in the optimization process (line 17). Afterwards, the shrink process is applied but with a modification that all the training vectors correctly classified by the best individual are removed. This approach may be considered as "more aggressive" as the size of $\mathbf{T}$ will decrease much faster in such a setting. Once the process of optimizing the linear SVM has been finalized, the fitness is switched back to AUC (line 20), and K_t is doubled to make capturing the more "difficult" parts of the space using an RBF kernel easier. Then a process of finding $\vec{\gamma}$ for adaptive RBF kernel is performed. The random populations for each γ value (line 2) are created

and evaluated. The best-fitted individual (line 6) is extracted and its fitness is stored in $\vec{\gamma}_\eta$ (line 7). At this point, SVMs may already contain vectors with both linear and RBF kernels as $\mathcal{S}_{\text{best}}$ is utilized. Afterwards, the γ value with the largest corresponding fitness is found (line 8) and $\vec{\gamma}$ is build upon it (line 9). Finally, the $\mathbf{T}'$ sets are evolved in the same way as presented in Algorithm 6.

The one disadvantage of such an approach is that it is hard to know if a given dataset could be “coarsely” modelled, by linear SVM and forcing this kind of solution may be harmful to the end result. It was proven in initial experimentation that this is true for some of the benchmark datasets where this algorithm worsens the performance of the model. That is why a co-evolution scheme is proposed in Algorithm 8 to combine these two approaches and benefit from the concurrent optimization of different kernel combinations. In order to tackle the computational cost a competitive-like approach [115, 118] is used in which the better (sub-)populations trigger the process of pruning the worse populations that are “outperformed”

First, a separate linear-kernel population P^i of size N is generated for each $C_i \in \vec{C}_L$ (lines 1–4). Each population P^i undergoes the evolution presented in Algorithm 6—each P^i has its own SV pool $\mathcal{S}_{\text{best}}^i$, and for each population a separate $\vec{\gamma}_i$ is determined (see Algorithm 7, line 1 for more details). Note the γ values may be dependent on the selected C , hence they are likely to be different across the populations. For each P^i , $\mathbf{T}$ is shrunk (Algorithm 8, line 6) using the procedure introduced in Algorithm 7 (based on the best individual performance). Afterwards, all populations are added to the pool G (Algorithm 8, line 7).

At this point, all of the linear-kernel SVMs are evolved, together with the corresponding $\vec{\gamma}_i$, $\mathcal{S}_{\text{best}}^i$, and $\mathbf{T}_P^i$ for each of $P^i \in G$. In the next step a separate population P^γ is generated (lines 8–9) that will optimize the RBF kernels, as in Algorithm 6. For P^γ , its training set becomes $\mathbf{T}$. Additionally, all of the current best individuals $\tilde{P}_{\text{best}}^i$ for each population are stored (line 11)—it will allow checking if those should be optimized at a later stage of the algorithm.

In the co-evolutionary part of the algorithm, G populations are evolved which exploit their unique training sets $\mathbf{T}_P^i$. To prune the entire set of all populations, the most promising ones are selected for evolution. To equally capture the quality of the entire population, alongside the quality of the best individual, the populations are sorted according to the sum of their average $\eta(P^i)$ and best $\eta(P_{\text{best}}^i)$ fitness

Algorithm 8 Combining Algorithms 6 and 7 into a competitive co-evolutionary scheme.

```

1:  $\vec{C}_L = \{0.01, 0.1, 1, 10, 100\}$ ,  $G \leftarrow \emptyset$ 
2: for all  $C_i$  in  $\vec{C}_L$  do
3:    $P^i \leftarrow \text{Generate}(N, C_i, \text{Linear Kernel}, \mathbf{T})$ 
4:    $P^i \leftarrow \text{EVOLVET}(P, \mathcal{S}_{\text{best}}^i)$ 
5:    $\vec{\gamma} \leftarrow \text{FINDGAMMA}(\mathcal{S}_{\text{best}}^i, C_i)$ 
6:    $\mathbf{T}_P^i \leftarrow \text{Shrink}(\mathbf{T}, P_{\text{best}}^i)$ 
7:   Add  $P^i$  to  $G$ 
8:  $P^\gamma \leftarrow \text{Generate}(N, \mathbf{T}, \text{FINDGAMMAANDC})$ 
9: Add  $P^\gamma$  to  $G$ 
10: for all  $P^i$  in  $G$  do
11:    $P_{\text{best}}^i \leftarrow \tilde{P}_{\text{best}}^i$ 
12: while all  $P^i$  in  $G$  not finished do
13:   for all  $P^i$  in  $G$  do
14:      $P^i \leftarrow \text{Generate}(N, \mathbf{T}_P^i, P^i)$ 
15:      $P^i \leftarrow \text{Calculate fitness}(P^i, \mathcal{S}_{\text{best}}^i)$ 
16:     if  $|G| > 3$  then
17:       Sort  $G$  by  $\eta(P^i) + \eta(P_{\text{best}}^i)$ 
18:        $G \leftarrow \text{Select three best } P^i \text{ in } G$ 
19:       if  $\eta(\tilde{P}_{\text{best}}^i) > \eta(P_{\text{best}}^i)$  then
20:          $\text{EVOLVET}(P^i, \mathcal{S}_{\text{best}}^i)$ 
21:       Sort  $G$  by  $\eta(P^i) \blacktriangleright$  descendingly
22:       for all  $i$  in  $\text{range}(1, |G|)$  do
23:          $|P^i| \leftarrow |P^i| - 2$ 
24:         if  $P^i \text{ size} \leq 2$  then
25:            $G \leftarrow G - P^i$ 
26:         if  $\eta(\tilde{P}_{\text{best}}^i) > \eta(P_{\text{best}}^i)$  then
27:           Add  $\text{SV}(\tilde{P}_{\text{best}}^i)$  to  $\mathcal{S}_{\text{best}}^i$ 
28:            $\mathbf{T}_P^i \leftarrow \text{Shrink}(\mathbf{T}, P^i)$ 
29:            $P_{\text{best}}^i \leftarrow \tilde{P}_{\text{best}}^i$ 
30: return Best individual from all  $P^i \in G$ 

```

scores (lines 16–18).

Afterwards, the training sets for each population are optimized using the memetic algorithm (line 20). Once the evolutions have finished, the populations are sorted descendingly (line 21) and gradually decrease the size $|P^i|$ of the “weaker” populations through removing random individuals, excluding the best-fitted one

(line 23). The pruning of the populations that are less promising helps to reduce the computational cost of the optimization. If the population is smaller than two chromosomes, then the population is excluded from further processing (line 25). For each population, it is verified if the best fitness P_{best}^i has improved—in this case, $\mathcal{S}_{\text{best}}^i$ is updated with its SVs, and $\mathbf{T}$ shrunk for this population (line 28). Otherwise, if there is no fitness improvement, we those vectors selected for γ_i are discarded and progress to the next γ . Ultimately, the best individual among all populations is returned (line 30). Only alive populations are considered.

In the our paper [39], many different variants of the algorithm were proposed and tested. Using this knowledge in this work the most successful ones will be presented in experimental validation. Those variants will be denoted using following naming convention **DA-SVM** ([FS], [CE], **Kernel aggregation**, [No-T]), where $[\cdot]$ is an optional parameter. These possible variants are:

- **FS**—if this optional parameter is present, then the feature selection (i.e., the recursive feature elimination [38]) is switched on, and it is performed before the evolutionary optimization (as a pre-processing step).
- **CE**—if this optional parameter is present, then the co-evolutionary optimization is performed.
- **Kernel aggregation**—this mandatory parameter indicates which aggregation function was used for handling multiple γ values in the RBF kernel.
- **No-T**—if this optional parameter is present, then all training vectors are selected as SVs, hence no SVM training is performed while calculating the fitness of individuals in a population.

The most successful variants included ones that use co-evolutionary optimization joined with feature selection (based on result in our paper [39]). The No-T variant is interesting and also was presnet among top configuration to consider. Because of that other variants will not be presented in detailed by experiments.

3.6 Building ensembles

Building an ensemble of multiple classifiers is often considered the last step toward a possible improvement of classifier performance. This popular paradigm is based on the assumption of leveraging the strength of individual classifiers and mitigating their weaknesses [110]. This opens an interesting direction that could help further improve the performance and help to deal with large datasets (as one can be using multiple lightweight SVM in the ensemble).

The initial work focused on building a novel cascade structure. As building ensembles involves training of multiple base classifiers many of such techniques [31, 56] proposed to use a subset of $\mathbf{T}$ in order to train those base models. Compared to those ensemble techniques developed for SVMs the major difference is that the presented solution will optimize $\mathbf{T}'$ using the memetic algorithm. This addresses two problems, first, it mitigates the importance of selecting the proper size of the reduced $\mathbf{T}$ for each base classifier secondly, the optimized $\mathbf{T}'$ should improve the classification performance compared to the randomly selected subset of $\mathbf{T}$. What is more, a notion of competence regions for each node in a cascade is proposed (denoted also as a certain region). This enables to split the input space into *certain* and *uncertain* parts with higher and lower expected classification performance.

The high-level flowchart of this new method called CE-SVM—**C**ascades of **E**volutionarily optimized **S**VMs is present in Figure 3.6. Each SVM node within the cascade is designed to specialize in a specific part of the input space whereas, the final SVM node is used to classify examples that do not fit within these certain regions of the space.

To divide the dataset into certain and uncertain parts all decisions elaborated by a trained SVM are analyzed (being the distances from the decision hyperplane) and mapped onto the axis after sorting as presented in Figure 3.7. Individual vectors are represented by vertical black lines (evenly spread for clarity), whereas the green line indicates the SVM decision boundary with zero bias. Certain regions should hold all of the training vectors that are correctly classified. To confront these distances across different SVMs, they are normalized according to the minimal and maximal responses of each model.

In order to find such thresholds all of the decision values for the data available

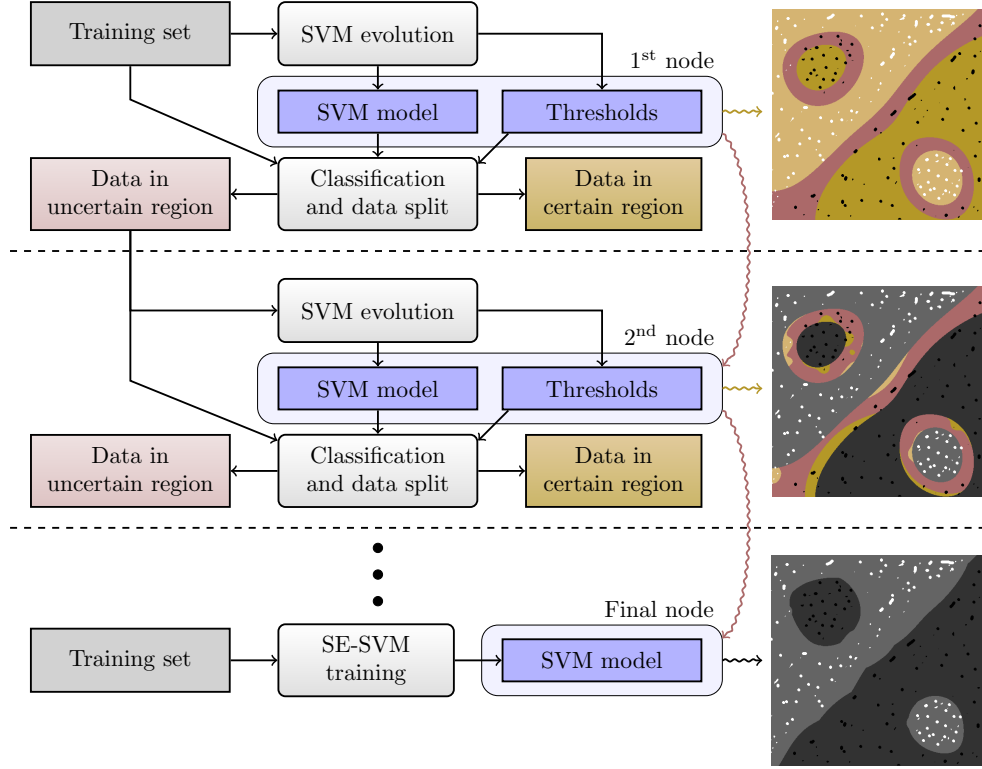


Figure 3.6: An outline of our cascade. At each level, a new node is evolved (blue boxes), composed of a lightweight SVM model and thresholds that split the input space into certain (yellow shades) and uncertain (red shade) regions. The training data falling into the uncertain region are passed to the next level, and the final node is based on SE-SVM. Classification (waved arrows) is performed with the evolved cascade. This figure comes from our paper [40]

for training and validation ($\mathbf{T}_u \cup \mathbf{V}$) are calculated. The $\mathbf{T}_u$ represents part of the training set that lay in the uncertain region (at start $\mathbf{T} = \mathbf{T}_u$). All of those values are sorted in ascending order. Then, starting from the smallest value (the left part of the X-axis in Figure 3.7) the first instance $\mathbf{x}_+$ for which the ground-truth class label is positive is found. Next, the average of the decision value obtained for $\mathbf{x}_+$ and the decision value obtained for the last correctly-classified instance with the negative ground-truth label becomes the negative certainty threshold. If the threshold is increased beyond $\mathcal{H}_{margin}$, it is cropped to $\mathcal{H}_{margin}$ to incorporate an additional regularization term that assists in avoiding the memorization of training and validation data by overfitting the threshold value to these sets. The positive

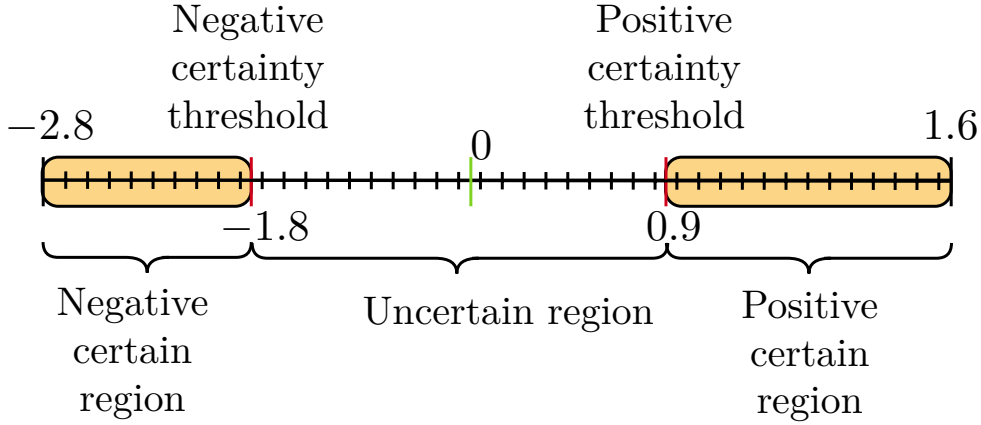


Figure 3.7: Visualization of certainty thresholds.

threshold for the positive certain regions is extracted in a similar fashion.

Building the Cascade of SVMs

To construct the cascade of specialized SVMs, it is assumed that all input data ($\mathbf{T}$ and $\mathbf{V}$) belongs to the uncertain region. At each node of the cascade, the training set and hyperparameters of the SVM kernel are evolutionarily optimized using SE-SVM with one modification (detailed in Section 3.4). This modification is that the feature set is not optimized at all and always all of the features are used. That is due to the fact that each node is modelling part of the input space but gets SVs from all of the previous nodes. Those SVs are passed down the cascade along with their corresponding γ values and an adaptive RBF kernel is utilized (as presented in Section 3.5). This adaptive kernel is used to guarantee the smoothness and continuity of the decision boundary and to provide knowledge to the current node about the previous results in the cascade. The current uncertain region of the input space is reduced using this evolved SVM by classifying all vectors from the previous uncertain region and extracting the thresholds that define the certain region in which the new node specializes. The cascade is iteratively extended with nodes until either the training set is exhausted or it is impossible to enlarge certain regions further by introducing more training and validation vectors using additional SVM nodes. The final node in the cascade is responsible for classifying the uncertain region of the input space.

For the fitness of node evolution, the number of correctly classified samples within a certain region in relation to all of the available samples is used. This fitness encourages selecting the largest possible certain regions. One important detail here is that during the calculation of threshold bias term of SVM should not be taken into a certain region. Otherwise, SVM can become "certain" for vectors laying far from all of the SVs where all of the kernel function evaluations are zero (or very close) and the bias term is used to make the decision about the classification result.

Although as proved in the conference paper [40] the method provides great classification performance. What is more selected certain regions showed increased accuracy of classification. However, that are several problems with this method after initial tests were conducted. First of all, for some of the sets, the cascade generation was stopped early thus the coverage of the test set by certain regions was small. This was especially visible in the case of imbalanced sets when the minority class was entirely included in a certain region. Another problem was occurring in the datasets with the high number of features in which certain regions are limited due to data sparsity. One last problem was that the "inheritance" of SVs from previous nodes has caused a long training time especially if the cascade keeps growing bigger (it may also happen when the first few nodes are not covering a large part of the dataset). This led to the development of an improved method presented in the next subsection.

Building ensemble of evolutionary cascades

Firstly, to address the issue of poor scaling and long training times, the adaptive RBF kernel was removed from the design. This modification helped to streamline the overall computational complexity of the architecture, reducing both the training time and computational requirements for each individual node. Additionally, to further enhance the speed and efficiency of the cascade, a random validation set was selected for each cascade (as fitness evaluation takes even longer than SVM training). This allowed for more focused training on specific regions of the input space and increased the diversity among the cascades (which in turn resulted in more effective classification and higher accuracy). As previously mentioned joining

different kernels could be beneficial. In this case, two separate populations were evolved simultaneously, one with a linear SVM and the other with an RBF kernel. It means that nodes in a single cascade could have different kernels used. This approach allowed for greater flexibility in the design of the cascade, enabling it to adapt to a wider range of input data. Finally, to further improve the classification performance of the system, the uncertain region of the input space was handled by an Extra Tree classifier. This allowed for more accurate and efficient classification of data in this region, helping to reduce errors and improve the overall performance of the system.

Using multiple cascades allowed to build them in a more simplistic way. If one of them fails to cover a large part of the input space and provides poor results it can be improved and mitigated by other ones.

Algorithm 9 Code for building ensembles of evolutionary SVM cascades.

```

1:  $E \leftarrow \emptyset$ ,  $\epsilon \leftarrow 0.001$ 
2: repeat
3:    $\mathbf{T}, \mathbf{V} \leftarrow \text{Resample}(\mathbf{T} \cup \mathbf{V})$ 
4:    $\varepsilon_i \leftarrow \text{BuildCascade}(\mathbf{T}, \mathbf{V})$ 
5:    $\varsigma_i \leftarrow \text{ScoreLevelWise}(\varepsilon_i, \mathbf{T} \cup \mathbf{V})$ 
6:    $MCC_{\Delta}, \mathbf{T}_u, \mathbf{V}_u \leftarrow \text{Evaluate}(E, \varepsilon_i, \varsigma_i, \mathbf{T} \cup \mathbf{V})$ 
7:    $Uncertain_{\Delta} = |\mathbf{T}_u|/|\mathbf{T}| + |\mathbf{V}_u|/|\mathbf{V}|$ 
8:    $E \leftarrow E + (\varepsilon_i, \varsigma_i)$ 
9: until  $MCC_{\Delta} > \epsilon$  or  $Uncertain_{\Delta} > 1\%$ 
10: return  $E$ 

11: function BUILDCASCADE( $\mathbf{T}, \mathbf{V}$ )
12:    $\varepsilon \leftarrow \emptyset$ ,  $\mathbf{T}_u, \mathbf{V}_u \leftarrow \mathbf{T}, \mathbf{V}$ 
13:   repeat
14:      $SVM_{\text{Line}} \leftarrow \text{Evolve}(\mathbf{T}_u, \mathbf{V}_u, \text{Linear})$ 
15:      $SVM_{\text{RBF}} \leftarrow \text{Evolve}(\mathbf{T}_u, \mathbf{V}_u, \text{RBF})$ 
16:      $SVM \leftarrow \max(\eta(SVM_{\text{Line}}), \eta(SVM_{\text{RBF}}))$ 
17:      $\varepsilon \leftarrow \varepsilon + SVM$ 
18:      $\mathbf{T}_u, R_T \leftarrow \text{GetUncertain}(SVM, \mathbf{T}_u)$ 
19:      $\mathbf{V}_u, R_V \leftarrow \text{GetUncertain}(SVM, \mathbf{V}_u)$ 
20:   until  $R_T$  or  $R_V$  or  $|\mathbf{T}_u| > K_t$ 
21:   return  $\varepsilon$ 

```

The process of building such ensemble is presented in Algorithm 9. The first

line begin, with initialization of empty ensemble and ϵ value for improvement in MCC score required by new cascade. After that, the process of building ensemble begins. At the beginning (line 3) the training and validation set are joined and resample where up to 1000 vectors are selected for $\mathbf{V}$ and no more than 20% of all data. This division is done in a random manner and passed to process of building cascade (line 4). During this process all of the training and validation data is considered as uncertain region at first (line 12). Then the nodes are trained using an SE-SVM algorithm but without feature selection process (fitness is the same as with building single cascade). Two population are trained independently using different kernels namely Linear and RBF (lines 14-15). After the training the model with higher fitness score is selected (line 16) and appened to the cascade. Then this model is used to find new uncertain regions of the space (shrink current $\mathbf{T}_u$ and $\mathbf{V}_u$). During that process there is additional check whether size of the given uncertain set was reduced (R_T for $\mathbf{T}_u$ and R_V for $\mathbf{V}_u$). The cascade is being built until there are new vectors added to certain region or the training data have sufficient size allowing to build new nodes. Otherwise the evolution of the cascade finished and it is returned to the main algorithm that build ensemble (line 21). Later in line 5, this cascade is scored where each node receives MCC score in its certain region. Those scores are used during the classification process which uses voting scheme and these MCC scores acts as weights. Thanks to that if just single node has poor performance it is known and its answer could be easily discarded (or outvoted). Then the new cascade is evaluated with the whole ensemble on the joined training and validation set. Here the improvemnt of MCC score over whole ensemble (MCC_{Δ}) is calculated and uncertain sets are selected (line 6). During the evaluation at least half of the cascades present in the ensemble need to answer within the certain region to consider this answer. Otherwise such vector is classified as lying in the uncertain region. In next line 7 the size of uncertain set with this new cascade is calculated and the cascade is being added to the ensemble (line 8). The stop condition when new cascades should not be added into the ensemble is when MCC score of the whole ensemble did not improve (by at least ϵ) when adding new cascade or the uncertain region did not decrease in size by more than 1% of the $\mathbf{T} \cup \mathbf{V}$. When one of that condition is fulfilled the process ends and ensemble is ready.

3.7 Summary

All of the presented methods are summarized by their main features in Table 3.3. Please note that the first two techniques (GASVM [74] and MASVM [95]) are earlier work by Nalepa and Kawulok. While the presented work uses both of those methods, it extends them significantly by combining them with hyperparameter optimization and feature selection. The last of the presented algorithms ECE-SVM is not yet published but is it planned to be publicized in journal.

Table 3.3: The most important aspects of SVMs optimized by evolutionary techniques.

Method	Year	F	Model			T	Uncertainty	Ensemble
			Single kernel	Var. γ in RBF	Mixture			
GASVM [74]	2012	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
MASVM [95]	2014	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
ALGA [75]	2017	$\times$	$\checkmark$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
ALMA [38]	2018	$\times$	$\checkmark$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
FSALMA [38]	2018	$\checkmark$	$\checkmark$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
SE-SVM [37]	2019	$\checkmark$	$\checkmark$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
E-SVM [36]	2019	$\checkmark$	$\checkmark$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
ARBF-SVM [93]	2020	$\times$	$\times$	$\checkmark$	$\times$	$\checkmark$	$\times$	$\times$
DA-SVM [39]	2021	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$
CE-SVM [40]	2022	$\times$	$\times$	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$
ECE-SVM	-	$\times$	$\times$	$\times$	$\checkmark^*$	$\checkmark$	$\checkmark$	$\checkmark$

GASVM—Genetic Algorithm for selecting **SVM** training sets.

MASVM—Memetic Algorithm for selecting **SVM** training sets.

ALGA—ALternating Genetic Algorithm for selecting **SVM** training sets and models.

ALMA—ALternating Memetic Algorithm for selecting **SVM** training sets and models.

FSALMA—ALternating Memetic Algorithm for selecting **SVM** training sets and models with **F**eature **S**election.

SE-SVM—Simultaneously-Evolved **SVMs**.

E-SVM—Evolutionarily-tuned **SVMs**.

ARBF-SVM—**SVMs** with Adaptive **RBF** kernels.

DA-SVM—Data-Adaptive Support Vector Machines.

CE-SVM—Cascades of Evolutionarily optimized **SVMs**.

ECE-SVM—Ensembles of Cascades of Evolutionarily optimized **SVMs**.

* the mixture of kernels in ECE-SVM is not using the same mixing as in DA-SVM algorithm.

In ECE-SVM different kernels are present in separate nodes not in single SVM.

Chapter 4

Experimental validation

This chapter shows the results of experimental validation where the most important algorithms introduced in this dissertation are tested. Their performance is investigated using two types of datasets: (1) the artificially generated 2D datasets and (2) multiple benchmark datasets taken from popular repositories.

The chapter is organized as follows: first, the datasets are briefly described and presented (Section 4.1) with reasoning why they were selected for this study. Next in Section 4.2 the experimental setup and implementation details are presented. Following that, in Section 4.3.1 each algorithm is validated independently using 2D datasets and all of them are compared using those artificially generated datasets. In Section 4.3.3 those proposed approaches are compared to each other and compared to other popular classifiers and state-of-the-art methods using benchmark datasets.

4.1 Datasets

The effectiveness of the proposed algorithms will be tested using two kinds of datasets:

- **Artificially generated datasets**—they include vectors that are either manually created or produced to adhere to a predetermined distribution, such as the Gaussian distribution. As a result, the underlying characteristics of the data are known, which is not always the case with benchmark and real-world datasets. Additionally, artificially generated datasets are often simple

to visualize. These datasets are used to study the behavior of new algorithms. This study includes several artificially generated datasets (see examples in Figure 4.1, where white and black pixels represent vectors from the positive and negative classes, respectively).

- Benchmark datasets—datasets of this type have various characteristics and are used to evaluate the effectiveness of algorithms. They ease comparison among the methods and provide points of reference as well as prove the usefulness of the methods as these types of datasets are often based on real-life data. There are three popular repositories from which datasets were gathered for this study:
 1. UC Irvine (UCI) machine learning repository: <https://archive.ics.uci.edu/ml/index.php5>
 2. Knowledge Extraction based on Evolutionary Learning (KEEL) repository: <http://www.keel.es/>
 3. LibSVM repository: <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/>

In Table 4.2, benchmark datasets used are denoted with the name of the repository where they came from. Although the sizes of most of these benchmark datasets are not very large, they are widely used in the literature to compare different algorithms that optimize various aspects of SVMs.

All of the presented datasets concern binary problems. Even though SVMs could be extended to multiclass classification problems by employing the one-vs-one or one-vs-all scheme, this kind of solution might be suboptimal as presented in our previous work [39]. The algorithms discussed in the previous section could be used with specialized methods that handle multiclass classification as there is no inherent limitation to the number of classes. However, this is out of the scope of this dissertation.

All of the artificially generated datasets are presented in Table 4.1. They were created by “painting” a 2D canvas with black and white vectors as presented in Figure 4.1, where all test folds (Ψ) are visualized. Each dataset contains a single

Table 4.1: The artificial created 2D datasets use in the analysis. IR stands for imbalance ratio, sum is the number of vectors in the dataset where Tr stands for the number of vectors in the training set and test is the number of vectors in the test set.

	IR	Sum	Tr	Test		IR	Sum	Tr	Test
2D-Shapes	1.61	6458	1160	5298	2D-Dense	1.10	102301	61381	20460
2D-Blobs	1.26	34957	6371	16767	2D-Linear pool	1.11	18666	4943	6767
2D-dots	1.42	2842	242	1552	2D-Linear pool flip	1.11	18666	4943	6767
2D-Chessboard	1.29	69971	19949	22169	2D-Many pools	1.24	37110	22266	7422
2D-Circles noise	1.51	60462	36278	12092	2D-Mixed	1.18	34434	20662	6886
2D-Grain blobs	1.95	46555	27933	9311	2D-Mixed heavy	4.07	19084	11452	3816
2D-Imbalanced line	2.00	26500	15900	5300	2D-Stripes	1.60	40237	24143	8047
2D Line	1.25	13228	3600	5606	2D-X	1.45	80506	48304	16101
2D-Line wit Dot	1.64	55165	33099	11033					

division into training, validation, and test data. These datasets were created in order to better understand the behavior and performance of the algorithm during the design phase. Thus, each of such datasets has its own unique character, which will be briefly described.

The 2D_Shapes present a wiggled line with a small margin between them where the test set is much bigger than the training set (Figure 4.1a). The 2D_Blobs dataset (Figure 4.1b) was created to test patterns of different densities with large empty spaces, contrary to that 2D_points show four sparse patterns in each corner of the image which can be treated mostly independently. The chessboard pattern seen in Figure 4.1d introduced a high number of samples and difficult-to-model decision hyperplane with varying densities (see bottom right corner). The circles with noise (Figure 4.1e) are an interesting pattern of concentric circles that belongs to different classes. Its purpose was to test if the selected SVs will also create such a pattern and be evenly distributed. The grain blobs (Figure 4.1f) test the behavior of non-linear data that has a sparse representation but covers most of the input space without much of the “empty” space without any vectors. Next datasets in figures 4.1g - 4.1l present different variants of data that has a clear linear division (Figure 4.1g) with the intrusion of non-linearly separable data (the rest of examples). Those datasets were intended to test the selection process of the training set as well as help to test adaptive kernel and ensemble building schemes—whether the “pools” of non-linear data will be handled first or it does not matter. As it can be seen, most datasets provide quite a clear margin between classes

so algorithms were expected to get results close to perfect classification even on the test set. Nonetheless, a lot of those datasets helped to catch problems at the early stages of development and were a good benchmark for the tested solution. The last two datasets (Figure 4.1n and 4.1o) were introduced to test how well the algorithm would work if there was not a clear boundary between data points and if the resulting model did not overfit. Here the expected scores should be far from perfect (meaning no mistakes on Ψ). Please note that these datasets were added during the development of subsequent algorithms so not all of them were created at the beginning for the ALMA method (the first algorithm chronologically). What is more with the ability to control the size of the sets they ranged from quite small (as in the case of 2D_points) to what could already be considered as a large dataset for SVM with over 100 000 samples. Furthermore, most of the datasets were kept balanced.

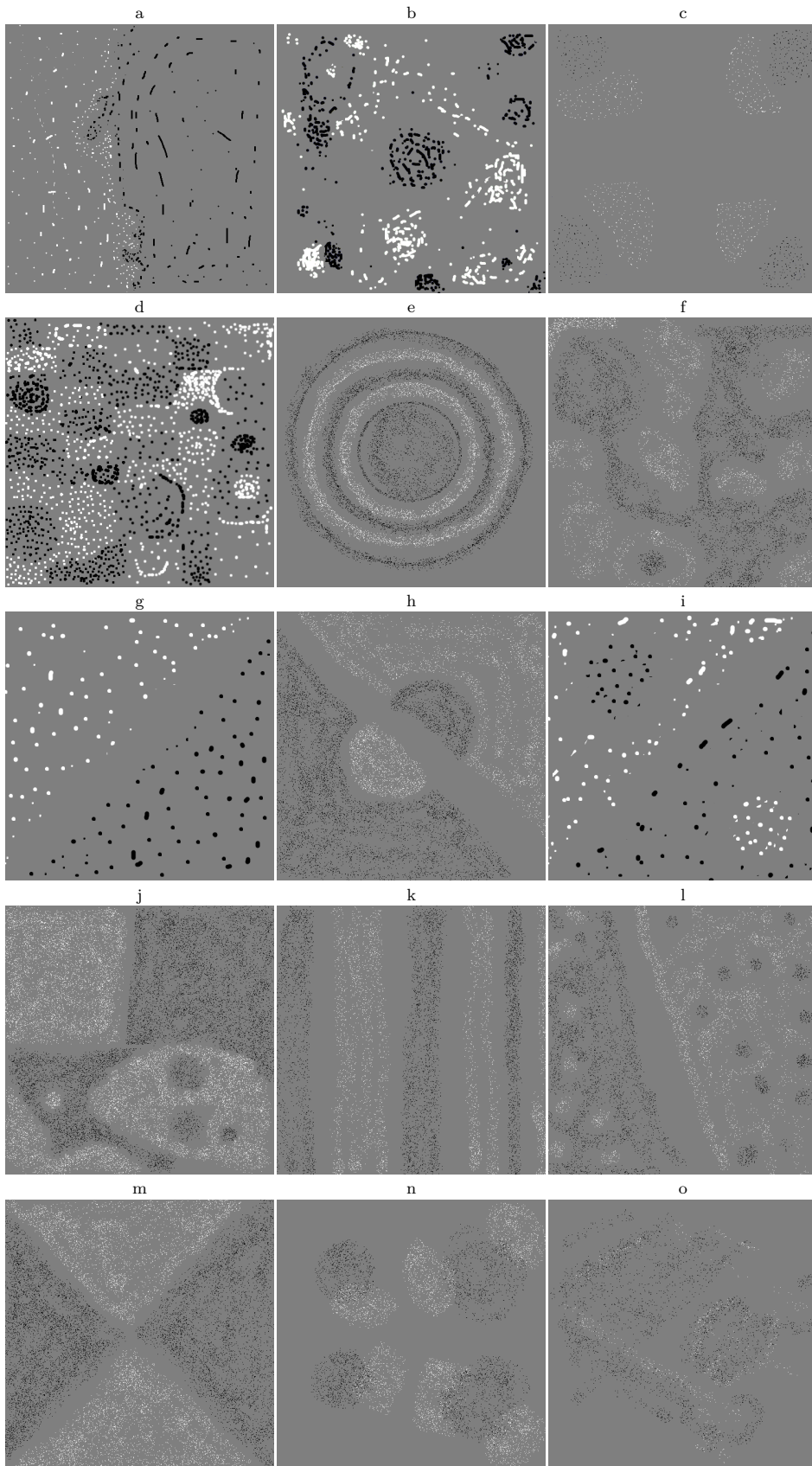


Figure 4.1: Visualization of tests sets for 2D datasets.

Table 4.2: The benchmark datasets used in the analysis.

Dataset	IR	Sum	Tr	Test	F	Dataset	IR	Sum	Tr	Test	F
a1a	3.15	32561	1605	30956	123	german	2.33	1000	800	200	24
a2a	3.15	32561	2265	30296	123	magic	1.84	19020	15215	3805	10
a3a	3.15	32561	3185	29376	123	mnist	1.03	70000	60000	10000	784
a4a	3.15	32561	4781	27780	123	Page-blocks	8.79	5472	4377	1095	10
a5a	3.15	32561	6414	26147	123	phoneme	2.41	5404	4322	1082	5
a6a	3.15	32561	11220	21341	123	pima	1.87	768	614	154	8
a7a	3.15	32561	16100	16461	123	ring	1.02	7400	5919	1481	20
a8a	3.15	32561	22696	9865	123	segment	6.02	2308	1846	462	19
a9a	3.18	48842	32561	16281	123	skin	3.82	245057	196045	49012	3
australian	1.25	690	551	139	14	spambase	1.54	4597	3662	935	57
banana	1.23	5300	4239	1061	2	twonorm	1.00	7400	5919	1481	20
banknote	1.25	1372	1097	275	4	vowel	9.98	988	790	198	13
cod	2.00	488565	331152	157413	8	wdbc	1.68	569	454	115	30
coil2000	15.76	9822	7856	1966	85	winequality	1.73	6497	5197	1300	11
covtype	1.05	581012	464809	116203	54	Wisconsin breast	1.86	683	546	137	9
Credit card clients	3.52	30000	23999	6001	23						

The benchmark datasets were selected to account for many different scenarios with diverse imbalance ratios as well as the dataset sizes and number of features. As training set selection is one of the vital points in the presented algorithms, very small datasets were excluded from the study. In the case where the number of samples is small, it would be better to train a classifier on all of the available data or even use a dedicated method to generate new samples (e.g., Synthetic Minority Oversampling Technique (SMOTE) algorithm [25]). In the case of UCI sets, a five-fold cross-validation approach is employed with stratification to ensure that each fold contains an equal ratio of vectors from both classes. Furthermore the $\mathbf{T}$ was divided into four non-overlapping parts and one of them is used as the $\mathbf{V}$. Hence there are five stratified folds divided into $\mathbf{T}$, $\mathbf{V}$ and Ψ with a 3:1:1 proportion. On the other hand, in KEEL, the sets are initially divided into five non-overlapping folds with distinct training ($\mathbf{T}$) and test (Ψ) sets and are kept without any changes. The datasets taken from LibSVM repository were also divided into the distinct training ($\mathbf{T}$) and test (Ψ) sets in all. Hence there is no separate $\mathbf{V}$ for both the KEEL and LibSVM datasets— $\mathbf{T} = \mathbf{V}$ was used in that case as the validation set is needed during the evolution process of presented algorithms. There is no risk in overfitting the SVM classifier as thanks to training set selection the final training set used for SVM classifier is different from the validation set $\mathbf{T}' \neq \mathbf{V}$.

All datasets were normalized by rescaling the features to $[0, 1]$ [64]. The rescaling was done on the training set so the actual values in the test set might be

out of $[0, 1]$ range. This is important as no information from the test set should be leaked in the process of learning the classifier.

4.2 Experimental setup

The algorithms were implemented in C++ (Visual Studio Community 2017) with the LibSVM library [24], and the experiments ran on the machine equipped with the Intel i7-9750H CPU and 32 GB of RAM under Windows 10 operating system. For other classifiers described, we exploited the Scikit-learn (version 0.20.2) implementation in Python 3.6 if not stated otherwise.

4.3 Results

This section will present all of the results and analysis of the proposed methods. First, it will start with 2D visualizations of algorithms, their main principles and a qualitative comparison over selected examples of the 2D datasets. Furthermore, the quantitative analysis will be performed over artificial datasets to select the best methods for final comparison among other algorithms and state-of-the-art methods for SVM optimization.

Not all algorithms will be tested in the following sections. First of all GASVM (created by Nalepa et. al. [74]) and ALGA methods had a major drawback in the need to select proper K_t value in order to achieve good performance which requires checking multiple values (this is time costly process). However, this drawback was removed in the later work with MASVM (Nalepa et. al [95]) and ALMA which will be used as baseline techniques. Moreover, FSALMA and E-SVM introduced a feature selection mechanism which is not useful in the case of 2D artificial datasets as always all of the features are selected. Besides that, both FSALMA and E-SVM are surpassed by the SE-SVM algorithm which presented superior results to both of those methods [37]. Regarding the DA-SVM method, only the best variants presented in previous work [39] would be tested (in order to save computation time).

4.3.1 Visualizations of principles

First, the process of evolution of the algorithms will be visualized and discussed based on 2D datasets. The algorithm will be presented in the same order as they were introduced in Section 3. Those visualizations will not present each subsequent generation but would rather pinpoint fragments where the best solution has changed and is worth discussing.

The evolution process of ALMA is presented in Figure 4.2. First, in Figure 4.2a, a random subset of vectors is selected for the training set alongside default parameters. This pair is used to train the presented SVM classifier. This initial solution offers a quite poor solution to the problem. In Figure 4.2b the optimization process of hyperparameters starts. The training set remains unchanged throughout this phase which can be seen that the same SVs are present in Figure 4.2a and 4.2b. In later stages of evolution, this might not be so obvious as when $\mathbf{T}'$ grows not all of the training vectors might become support vectors. The sheer process of changing hyperparameters values affects which vectors become SVs. When no more improvements could be made optimization switches to the training set selection in Figure 4.2c. What is crucial in this algorithm (and this phase) is the fact that the size of $\mathbf{T}'$ can grow as seen in Figure 4.2d and Figure 4.2e. At the beginning of the optimization process phases may change more rapidly, where each phase lasts just for a few generations. Often a pivotal point of the algorithm is when the training set size grows multiple times in a single phase, as this means that the values of the current hyperparameters are well selected. In the last stages the overall improvements are rather minor as seen in Figure 4.2e and 4.2f where an adjustment to hyperparameters is made (the γ value gets larger between those two). Finally when no more improvements could be made the process stops and the best solution is returned (as seen in Figure 4.2f).

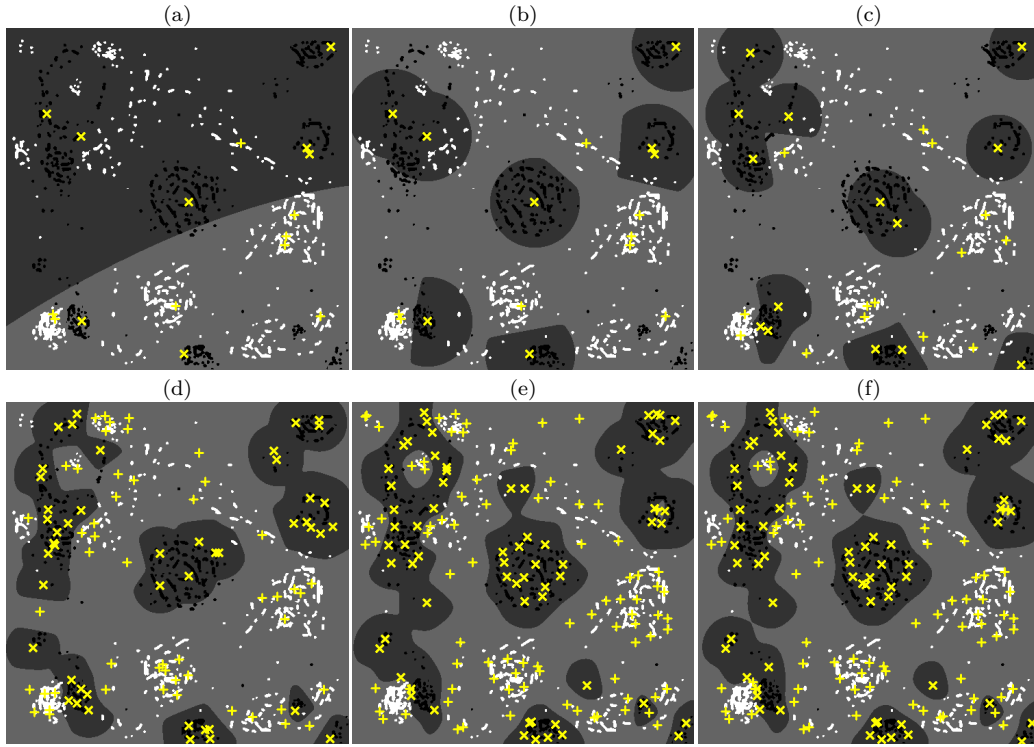


Figure 4.2: Visualization of the ALMA algorithm run. Results are visualized on 2D-Blobs test set. The yellow ticks and crosses mark the positions of support vectors. (a) presents a solution after initialization, (b) presents the end solution after kernel evolution phase, (c)—(e) shows progress of $\mathbf{T}'$ evolution and growing of its size (f) presents final solution where hyperparameters were adjusted.

Contrary to that SE-SVM can change both the $\mathbf{T}'$ and hyperparameters at the same time. The first advantage of this approach is visible by comparing Figure 4.2a and Figure 4.3a where in the case of SE-SVM initialization provides better modeling of input space. Here initially the γ values tend to be lower (have a larger range of influence) as the training set size is small and can adapt more dynamically when it grows. This is visible comparing Figure 4.3a to Figure 4.3d. This algorithm proves to provide better results as the mutual dependence between the training set and its “optimal” hyperparameters is visible and used as an advantage. In this particular case, SE-SVM also shows the ability to change the dynamics of adjustment for γ . Between Figures 4.3e to Figure 4.3f γ value is lowered while in all other examples (a-d), it was getting bigger (lowering the range of influence). One more advantage which is not visible in this case is that SE-SVM handles also

the process of feature selection while ALMA always uses whole $\mathbf{F}$.

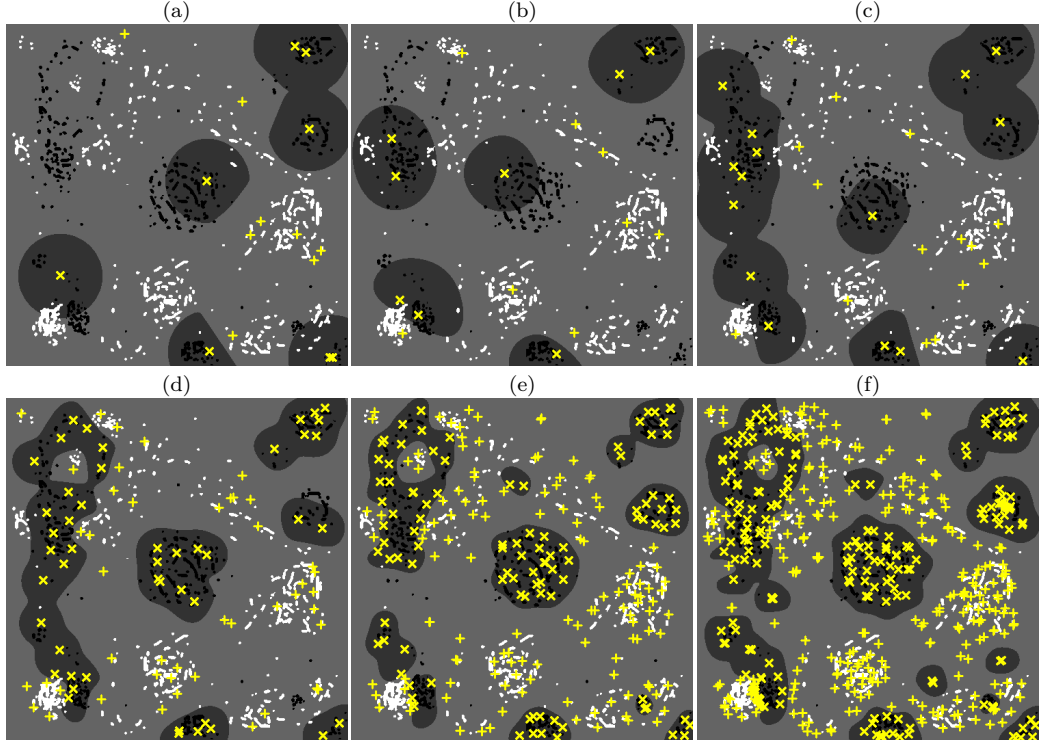


Figure 4.3: Visualization of the SESVM algorithm run.

The DA-SVM (and ARBF-SVM) works in a different manner than previously shown examples. First of all, it uses an adaptive kernel so vectors with different γ 's are denoted with colors, where each color presents a unique value of γ . In Figure 4.4a the best solution from the initial population is shown which is then evolved. This evolution process end and yields the solution visible in Figure 4.4b. Now according to this solution, the original $\mathbf{T}$ is being shrunk so only vectors visible in Figure 4.4c will be used in subsequent optimization that will use larger γ value. The result of the next round of iteration is presented in Figure 4.4d. In Figure 4.4e the next solution with new green vectors is present. However, it provides worse classification performance so these results are discarded. This means that the selected γ had a too low value and added too much fluctuation into the decision hyperplane. Please note that when results are discarded the shrunk $\mathbf{T}$ used for the next iteration is the same as the one obtained after the solution presented in

Figure 4.4d. Finally, the resulting solution is presented in Figure 4.4f where three distinctive γ values are present. This process was presented on a different dataset than ALMA and SE-SVM as in previous examples the process of discarding certain γ did not occur. This process was not present because 2D-Blobs datasets present more variability in the terms of heterogenous and homogenous regions as well as their sparsity so each time adding new γ helped to improve the classification performance and better model the data.

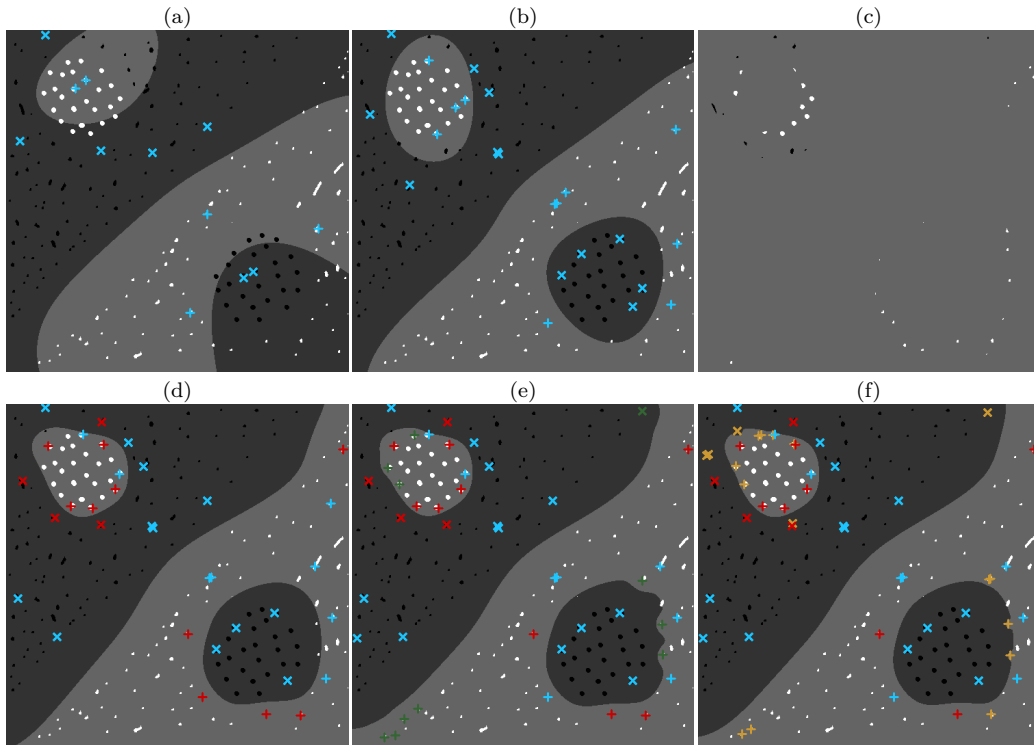


Figure 4.4: Visualization of ARBF-SVM algorithm. (a) presents the best solution from the initial population, (b) The best solution after finishing evolution with first γ from $\vec{\gamma}$, (c) Shrank training set that will be used in next iteration with subsequent γ . The shrinking procedure is based on the whole population, (d) Solution after second evolution has ended. Added new support vectors marked with red color crosses, (e) Adding next γ value marked with green vectors provided worse classification performance, these support vectors will be removed, (f) final solution for a given dataset containing three different γ values.

The evolution process of CE-SVM uses SE-SVM algorithm as its base for building the nodes of the cascades, although the process of feature selection is disabled

(only $\mathcal{M}$ and $\mathbf{T}'$ are being optimized). In Figure 4.5a the beginning of the process of building cascade is presented. For clarity, SVs are not visualized. The dark and light gray areas present certain regions of previous nodes, the yellow colors newly added (by the next node in the cascade) certain regions and red regions represent uncertain regions that are not yet covered by the cascade. As presented in the first row (Figure 4.5a-c) the evolution of such a cascade adds new certain regions, those could have different coverage of each class and could be imbalanced. Note that in this solution previously selected SVs are passed into the lower levels of the cascade to make the next node aware of previously obtained classification. This provides a much “smoother” decision hyperplane and enables the extension of some regions by a small extent as seen in Figure 4.5b. Moreover, all vectors from the certain region are removed for the building process of a new node in the cascade (from both the training and validation set). The final cascade is visualized in Figure 4.5d. What could be appreciated is that the majority of the uncertain regions (red color) do not have any vectors in them. For this region, the answer of such a cascade would tell that the sample could not be classified (the “unknown” class). In order to compare this algorithm with other solutions, a SE-SVM is run to cover all of the uncertain parts of the dataset. The behavior of this final node is presented in Figure 4.5e where previously uncertain regions are shown to belong to positive (light yellow) or negative class (gold). The resulting classification of such a combination of cascade with the SE-SVM to solve uncertainty is presented in Figure 4.5f.

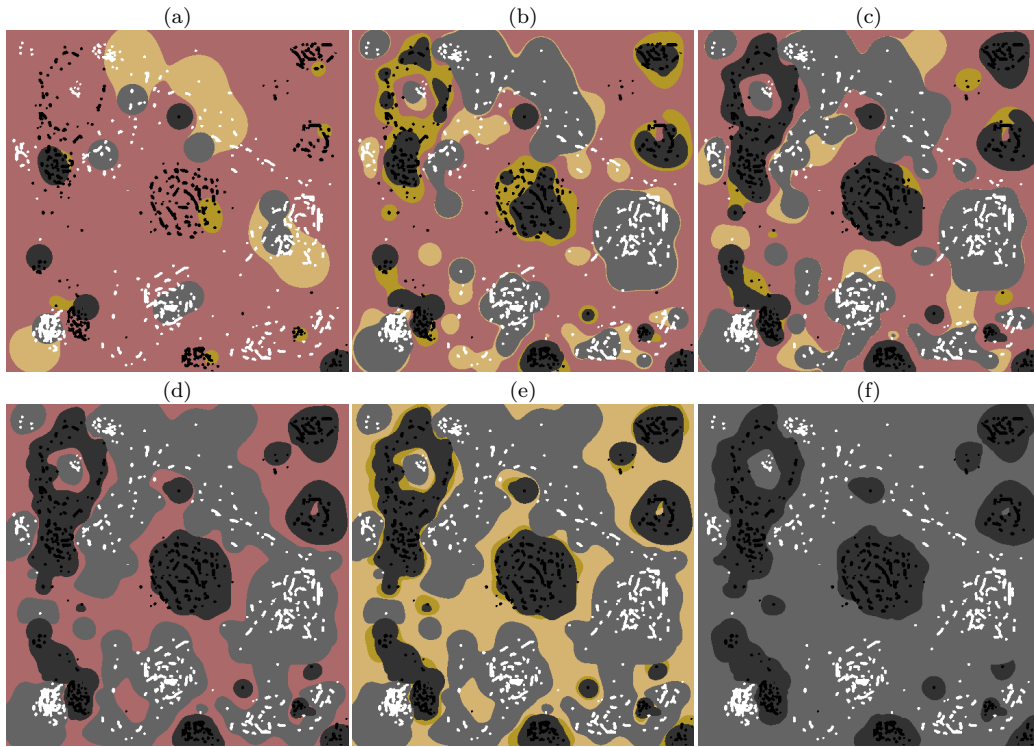


Figure 4.5: Visualization of the CE-SVM algorithm run. (a)-(c) shows the evolution process and adding new nodes. Red color denotes uncertain region, dark and light grey presents certain regions of previous nodes while yellow colors are used to depict new certain regions added by the current node. (d) shows final classification with uncertain regions (e) presents how SE-SVM node to resolve those regions works and (f) shows the final classification result of joining cascade with SE-SVM.

The last algorithm of ECE-SVM is presented in Figure 4.6. It works by building a list of cascades, giving them a weight based on their performance on $\mathbf{V}$ and employ a voting strategy to combine the results of individual cascades. What is more, only cascades which respond within the certain region is considered and at least half of the cascades need to provide a valid response. Otherwise, the region is considered as uncertain. In Figure 4.6a and 4.6d the example of cascades with denoted all of their SVs are presented. In Figure 4.6b-c and 4.6e there are examples of the cascades with uncertain regions present. Compared to CE-SVM the process of building a single cascade is changed. The SVs are not inherited and there is a much smaller validation set selected plus there are a few other minor changes (more details are provided in Section 3.6). The main reasoning behind

those changes is to speed up the building process of the single cascade while its classification performance could be worse. This is covered by the usage of multiple cascades that form the final ensemble which is presented in Figure 4.6f. Although the uncertain region is smaller than the one of CE-SVM (Figure 4.5f) it does not contain any vectors. This is a trade-off between those two algorithms where generally ECE-SVM provides much smaller uncertain regions although as will be presented later this does not affect classification performance negatively.

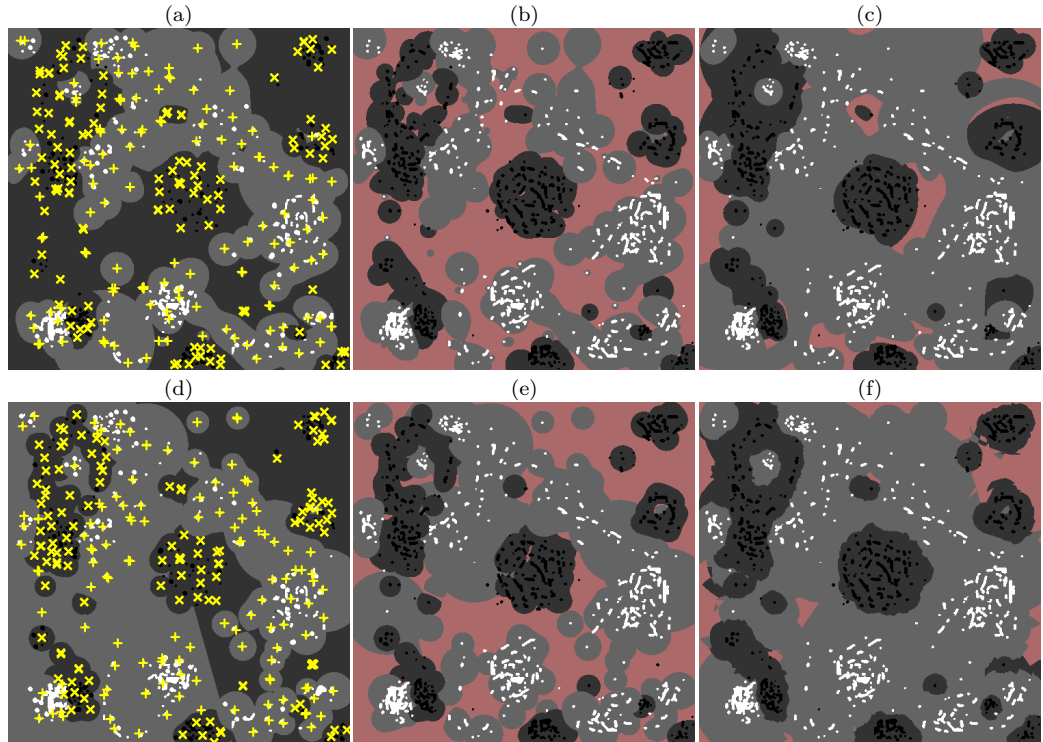


Figure 4.6: Visualization of the ECE-SVM algorithm run. (a) and (d) Present a single cascade result while (b) and (e) shows the same but with uncertain regions depicted in red, (c) presents another example of a different cascade. (f) presents the final combination of all of the cascades.

As such comparing uncertain regions of 2D datasets of those two algorithms (CE-SVM vs ECE-SVM) is presented in Figure 4.7. Comparing those images much smaller uncertain regions in the case of 2D-Shapes and 2D-Blobs is provided by the ECE-SVM method. Although most often those uncertain regions remain empty (no vectors from the whole dataset are present there). Another difference in those

decision hyperplanes is their “smoothness”. As ECE-SVM uses multiple cascades which are built without the knowledge of the previous level’s (nodes) performance the decision hyperplanes tend to be much more ragged. This is also present in those visualizations however this does not affect the final classification performance. What can be appreciated is that in the case of 2D-Dots, both methods provide similar uncertain regions and in both cases, they cover empty regions of input space. This means that although ECE-SVM tends to provide much smaller uncertain regions it did not lose the ability to select them. This is an important difference between those two algorithms because CE-SVM (as shown in [40]) proved to have problems in some of the benchmark datasets and did not cover much of the test set.

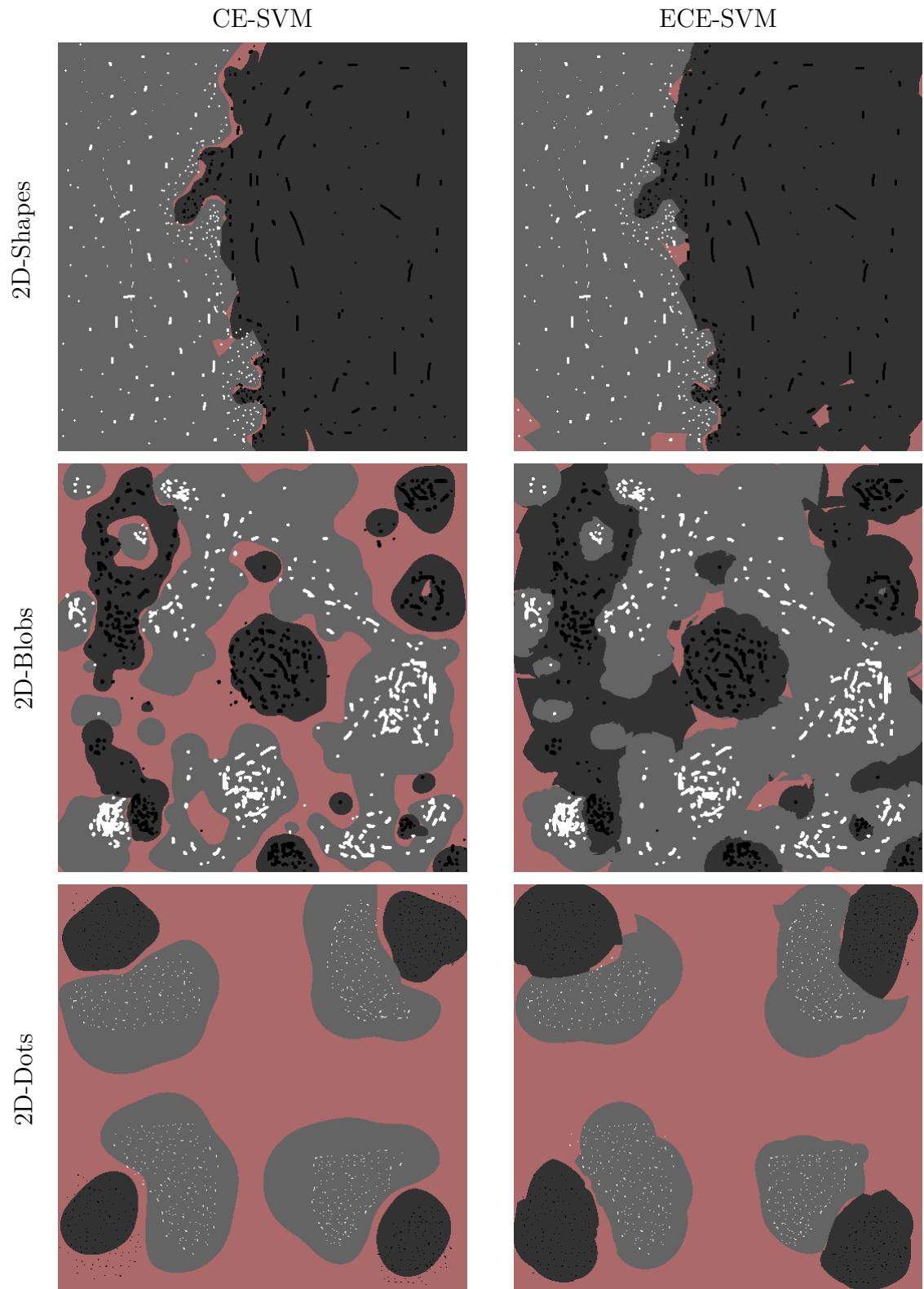


Figure 4.7: Qualitative comparisons of uncertain regions of CE-SVM and ECE-SVM ensembles.

Another advantage of building ensembles in the form of the cascade is their ability to adapt to new data. They also make it easier to discover new samples that came from other distribution (covers new regions of input space) with their ability to decide whether they are certain about the answer. An example of such a situation is shown in Figure 4.8. On the left-hand side (4.8a) the dataset is shown (inspired by 2D-Dots) on which the CE-SVM method is applied. The result of training the new classifier on this data is shown in Figure 4.8b. After the method is trained, new samples are obtained for this dataset. They are denoted with the red circle in Figure 4.8c. Most classifiers (including SVM) would be able to classify those samples in some way but would perform poorly in terms of the accuracy of such classification (probably all samples would be annotated with the positive label belonging to white vectors). In the case of CE-SVM all of those samples reside in the uncertain region of the classifier. This makes them easy to detect as the model itself is able to provide a proper answer. In such a case a procedure to update the current cascade could be made. In this particular example this results in adding a new node that will handle this region correctly. Moreover, in Figure 4.8d those newly added regions cover just the new data and still leave the rest of the empty space as an uncertain region. This update procedure does not require retraining the whole model. As such the update procedure could be extended in the future to cover the case of retraining single nodes that perform poorly. This could also be later extended to ECE-SVM method although using multiple cascades would make this procedure more difficult and time-consuming. The reason is the voting scheme of ECE-SVM where at least half of the cascades present there need to be certain, so at least half of them would need to go through an update process. On the other hand, ECE-SVM cascades are much faster to build so it might still be viable to select just a few (best) cascades and build the rest from scratch.

Lastly, we can compare the results of those methods on a single (2D-Blobs) dataset presented in Figure 4.9. In the first row, there are baseline methods to compare to, the SVM(RBF) refers to a solution provided by using a grid-search algorithm to find the optimal set of hyperparameters to SVM classifies using RBF kernel. The grid-searched solution provides great performance although it took almost 300 seconds to compute that. All other algorithms required less than 100 seconds to obtain the final result. The fastest MASVM required only 4 seconds.

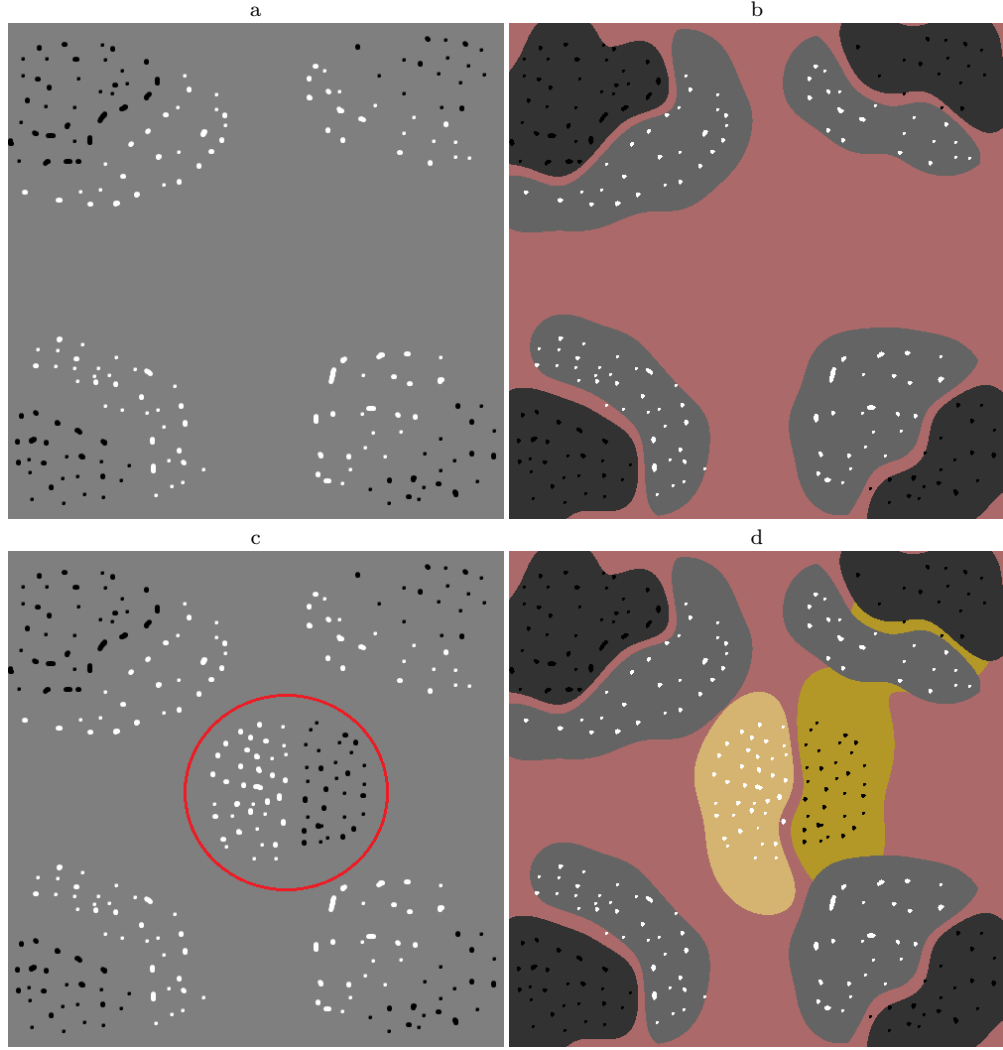


Figure 4.8: Updating the ensemble classifier. (a) show the original dataset, (b) present the trained CE-SVM classifier, (c) shows new data that arrived after the training, (d) shows an update to the current classifier.

The 2D example shown here cannot be considered a difficult classification problem which is why all of the methods return high MCC values over 0.95 (except MASVM). The number of SVs remains comparable to the grid search solution, where only MASVM was able to greatly reduce that number. On the other hand, SE-SVM increased this number over twofold but this also resulted in an increased MCC score. What is interesting is that DA-SVM(CE, No-T) provided results that are on par with other methods while no SVM training was performed in this

method. This proves that the training set selection can provide the $\mathbf{T}'$ that contains important samples which ultimately become SVs. For clarity reasons, the SVs are not visualised in the last two methods (CE-SVM and ECE-SVM). As expected those methods using an ensemble of classifiers provided the best scores among all of the presented ones although the number of SVs is much higher. However, this number is the sum of all vectors in the cascade or ensemble of cascades while during the classification process, not all of them are needed. The final number of SVs used for classification depends directly on the sample and which level of cascade provides the answer. In the next section, the quantitative results will be presented.

4.3.2 Comparison of proposed algorithms

The results of experiments on 2D datasets are presented in Table 4.3 while corresponding ranks are shown in Table 4.4. At first, let appreciated that all of the methods worked well and provided high-quality classification. It is the expected result as 2D datasets are fairly simple to model and expect a few datasets are fully separable (meaning that there are no examples from different classes mixed in the same area). There is a visible clear progression of results from the earliest methods (ALMA) to later ones, where presented ensembles (the latest method) provide the best scores among all of the methods. This is natural as each new method tried to improve on the drawbacks of previous ones. It is pretty interesting that DA-SVM(CE, No-T) scored the best from DA-SVM variants considering that no SVM training is performed. All of those approaches are compared with a standard grid search algorithm (SVM(GS)). The grid search algorithm provided scorers that are better than most of the evolutionary methods, except ensembles. Comparing the average results of different metrics with the ranking of the methods similar conclusion could be made. That means that there are no outliers in the performance among the datasets. The MASVM(GS) presents results of MASVM optimization which used hyperparameters learnt during the grid search algorithm. Although it scored the lowest it was the quickest method. In its defence looking into more detailed results shows that in some cases the training set failed to grow and evolution was stopped prematurely. This might happen as hyperparameters discovered during grid search that might not be appropriate for training when K_t

Table 4.3: Test metrics on 2D datasets

	Accuracy	F1	Precision	Recall	MCC
MASVM(GS)	0.932	0.886	0.917	0.878	0.837
ALMA	0.946	0.919	0.925	0.925	0.874
SE-SVM	0.973	0.959	0.959	0.961	0.937
ARBF-SVM	0.963	0.921	0.933	0.923	0.896
DA-SVM(CE,MAX)	0.960	0.926	0.935	0.926	0.897
DA-SVM(CE,No-T)	0.971	0.954	0.955	0.954	0.929
DA-SVM(CE,ONE)	0.965	0.939	0.948	0.939	0.911
CE-SVM	0.981	0.964	0.974	0.958	0.950
ECE-SVM	0.984	0.971	0.973	0.970	0.958
SVM(GS)	0.979	0.967	0.978	0.958	0.949

was low. In order to better understand the performance of each method a box plot containing the MCC score over all 2D datasets is presented in Figure 4.10. In there the difference between algorithms are easier to spot where both proposed ensemble provided the best results. However, there are two more important aspects of SVM - its training and classification time. The average values obtained by each algorithm are visualized in Figure 4.11. There are two Pareto plots where classification time and training time are composed with MCC score on the test set. It is easy to spot that although SVM(GS) provides high scores with regard to classification performance in both cases it is the slowest method often orders of magnitude slower. Please note that in this case all of the algorithms used the same implementation for training SVM classifier (LibSVM implemented in C version 3.19). This is important as depending on the implementation the training time could much different (e.g. when using GPU [131]). What can be seen here is that most of the evolutionary techniques provide fast training times.

4.3.3 Comparison with other methods

This section will compare the performance of the best and most promising algorithms (based on 2D results analysis) to other popular classifiers and state-of-the-art algorithms for SVM optimization. First, let's briefly describe the methods

Table 4.4: Test rankings on 2D datasets

	Accuracy	F1	Precision	Recall	MCC
MASVM(GS)	10.24	10.24	9.47	10.18	10.24
ALMA	9.06	9.00	9.82	8.47	9.00
SE-SVM	6.06	6.18	6.00	5.59	6.12
AKSVM	7.29	7.29	7.24	7.53	7.35
DA-SVM(CE,MAX)	7.94	7.94	7.71	8.35	8.00
DA-SVM(CE,No-T)	6.06	6.06	5.24	6.59	6.00
DA-SVM(CE,ONE)	7.76	7.65	7.71	7.59	7.88
CE-SVM	3.12	3.12	2.65	3.24	3.00
ECE-SVM	2.29	2.24	2.47	2.18	2.24
SVM(GS)	3.41	3.41	1.76	3.94	3.41

used for comparison:

- The K-NN classifier [103] is a non-parametric classification algorithm that makes predictions based on the closest data points in the feature space. Given a new data point, the K-NN classifier searches for the K closest data points in the training set and assigns the class label that is most common among those K neighbors. The choice of K is a hyperparameter that determines the number of neighbors to consider. In the presented results, three distinct values are tested $K=\{3,5,7\}$.
- The Random Forest (RF) [103] classifier is an ensemble method that combines multiple decision trees to make predictions. Each decision tree is trained on a bootstrap sample of the training set and a random subset of the features. During prediction, the Random Forest classifier aggregates the predictions of all the decision trees to make a final prediction.
- The Extra Tree (ET) [103] classifier is another ensemble method that builds a forest of decision trees. However, unlike the Random Forest classifier, the Extra Tree classifier constructs decision trees using random splits instead of the best splits. During training, the Extra Tree classifier selects random thresholds for each feature and chooses the split that maximizes the information gain.

- The Logistic Regression (LR) [103] classifier is a parametric classification algorithm that models the probability of each class using a logistic function. Given a set of input features, the Logistic Regression classifier estimates the probability of belonging to each class and assigns the class with the highest probability as the prediction. During training, the Logistic Regression classifier optimizes the parameters of the logistic function using a likelihood-based objective function.
- The Gaussian Naive Bayes (GNB) [103] classifier is a probabilistic classification algorithm that models the class-conditional probability distribution of each feature as a Gaussian distribution. Given a set of input features, the Gaussian Naive Bayes classifier estimates the probability of belonging to each class using Bayes' theorem and assigns the class with the highest probability as the prediction.
- CascadeSVM [56]—The approach involves dividing the data into subsets and optimizing them separately using multiple SVMs, then combining and filtering the results through a cascade of SVMs until the global optimum is reached. However this cascade is different then one that was presented, as SVMs are being joined throughout the process into single SVM at the end and no expert regions are selected (the name collision is coincidental).
- EnsembleSVM [31] is a method of building multiple SVMs trained of different subsamples of the training set. All of the base models share the same kernel and its hyperparameters. The main drawback of this method is searching for proper hyperparameters values for the size of subsamples and the number of base models that are used. The number of base models used was 100 (the same as in ET and RF methods) while for kernel the RBF was selected with $\gamma = 1/|\mathbf{F}| \cdot \text{var}(\mathbf{T})$ where $\text{var}(\cdot)$ is variance of the dataset. For size of the subsample $K_t = \{64, 128, 256\}$ were used and the best model was selected based on cross-validation score.
- Particle Swarm Optimization (PSO) [136] is a metaheuristic optimization algorithm inspired by the behavior of social organisms, such as bird flocks. In PSO, a population of particles moves through the search space seeking to

optimize an objective function by updating their position and velocity based on their personal and social best solutions. In this case the PSO algorithm is used to optimize hyperparameters of SVM and select one of four kernel functions (linear, polynomial, RBF, sigmoid)

- Hyperband (HB) [136] is a randomized search strategy algorithm. The key idea behind Hyperband is to allocate more computational resources to promising configurations early in the optimization process, while quickly discarding poorly performing ones.
- Genetic algorithm (GA) [136, 79] is based on TPOT library that is an advanced tool that conducts an automated exploration of machine learning pipelines. These pipelines may include supervised classification models, preprocessors, feature selection approaches, and other estimators or transformers that comply with the scikit-learn API. In addition, the TPOTClassifier conducts a search for the optimal hyperparameters for each pipeline component. For this study classifier was limited to SVM where C and kernel type were optimized.

Table 4.5 show averaged results of running all of the presented algorithms on 31 benchmark datasets (see section 4.1). The corresponding ranking is presented in Table 4.6. The results showed that SE-SVM outperformed ALMA when analyzing all the metrics, and this conclusion was confirmed when comparing the ranking of the two methods. This proves that simultaneous optimization is better than alternating approach where different aspects are optimized independently. What is interesting, CE-SVM failed to perform well on some of the a1a-a9a series of datasets, resulting in much lower average scores compared to any of the presented methods. DA-SVM provided average results compared to other methods, and its variants that used additional feature selection scored even lower, indicating that feature selection is not useful for the selected datasets as it ultimately lowered the performance of the classifier. On the other hand, ECE-SVM performed the best in all metrics except accuracy, where it was on par with SVM(RBF) that used the grid search algorithm and full training set. SVM(Linear) and LR provided comparable results, but since not all the data is linearly separable, their results

could not be as good as other methods. Different state-of-the-art methods for optimizing hyperparameters like PSO, HB and GA provided good results which could be compared with SE-SVM algorithm. Two methods for building ensembles of SVMs, EnsembleSVM and cascades, provided interesting results. EnsembleSVM had a problem with selecting its hyperparameters correctly, as the method itself does not address the problem of hyperparameter selection. Furthermore, it adds another two important hyperparameters, the size of the subset and the number of base models that need to be trained. So ultimately increasing the computational budget for checking different hyperparameter setting could improve the presented results but may be impractical. On the other hand, the cascades proved to provide much better results, which were comparable with SVM(RBF) while these results were obtained in shorter training time. Finally, RF and ET also provided high metrics, indicating their effectiveness in classification tasks.

Looking at the ranking there is one important thing visible. There is no one algorithm that is the best among all of the datasets. As seen the ECE-SVM provided the best results in F1 and MCC while it was outperformed by CascadeSVM and a few others when analyzing accuracy. However, the difference are visible when comparing the training and classification times among the methods as presented in Figure 4.12.

Table 4.5: Results of test performed on benchmark datasets. The results are averaged score over all of the dataset.

	Accuracy	F1	Precision	Recall	MCC
k-NN(3)	0.858	0.723	0.761	0.705	0.625
k-NN(5)	0.861	0.724	0.774	0.702	0.631
k-NN(7)	0.861	0.719	0.777	0.693	0.628
GNB	0.680	0.586	0.596	0.769	0.450
LR	0.854	0.694	0.787	0.661	0.610
SVM(Linear)	0.853	0.680	0.751	0.649	0.597
SVM(RBF)	0.878	0.734	0.812	0.692	0.663
HB	0.876	0.722	0.779	0.684	0.641
PSO	0.876	0.723	0.777	0.688	0.642
GA	0.843	0.689	0.740	0.655	0.606
RF	0.873	0.732	0.803	0.691	0.654
ET	0.873	0.735	0.802	0.696	0.656
EnsembleSVM	0.840	0.749	0.705	0.851	0.657
CascadeSVM	0.874	0.739	0.787	0.716	0.660
ALMA	0.874	0.722	0.780	0.691	0.639
SE-SVM	0.872	0.732	0.789	0.703	0.649
DA-SVM(CE, No-T)	0.863	0.711	0.777	0.677	0.623
DA-SVM(CE, One)	0.866	0.723	0.777	0.700	0.633
DA-SVM(FS, CE, No-T)	0.866	0.703	0.781	0.663	0.619
DA-SVM(FS, CE, One)	0.865	0.704	0.777	0.663	0.618
CE-SVM	0.849	0.611	0.802	0.578	0.553
ECE-SVM	0.878	0.762	0.794	0.749	0.681

Table 4.6: Ranking of methods for classification performance on benchmark data-sets.

	Accuracy	F1	Precision	Recall	MCC
k-NN(3)	14.29	13.06	14.61	10.19	14.06
k-NN(5)	13.10	11.77	13.68	10.45	12.71
k-NN(7)	11.94	11.35	12.06	11.29	11.94
GNB	19.42	17.45	18.13	9.26	18.23
LR	8.90	9.81	9.58	11.06	9.00
SVM(Linear)	9.00	10.26	9.84	11.13	9.68
SVM(RBF)	7.87	9.32	6.06	11.45	8.23
HB	7.55	8.74	9.10	9.97	9.06
PSO	7.68	8.48	9.90	9.84	9.00
GA	8.84	9.39	10.55	10.48	10.06
RF	11.19	12.10	9.58	13.48	11.42
ET	12.00	11.84	10.13	12.45	11.32
EnsembleSVM	15.52	7.39	17.00	4.26	8.32
CascadeSVM	6.84	7.48	7.87	7.81	6.97
ALMA	13.37	14.60	12.67	14.87	14.20
SE-SVM	11.16	11.74	11.61	12.35	11.06
DA-SVM(CE, No-T)	11.48	13.52	11.90	13.90	13.23
DA-SVM(CE, One)	11.68	11.65	13.26	11.84	12.39
DA-SVM(FS, CE, No-T)	12.45	14.19	11.52	14.87	13.77
DA-SVM(FS, CE, One)	13.32	15.03	11.74	15.48	14.84
CE-SVM	15.74	16.03	9.39	16.55	16.13
ECE-SVM	8.03	5.81	10.13	5.55	5.87

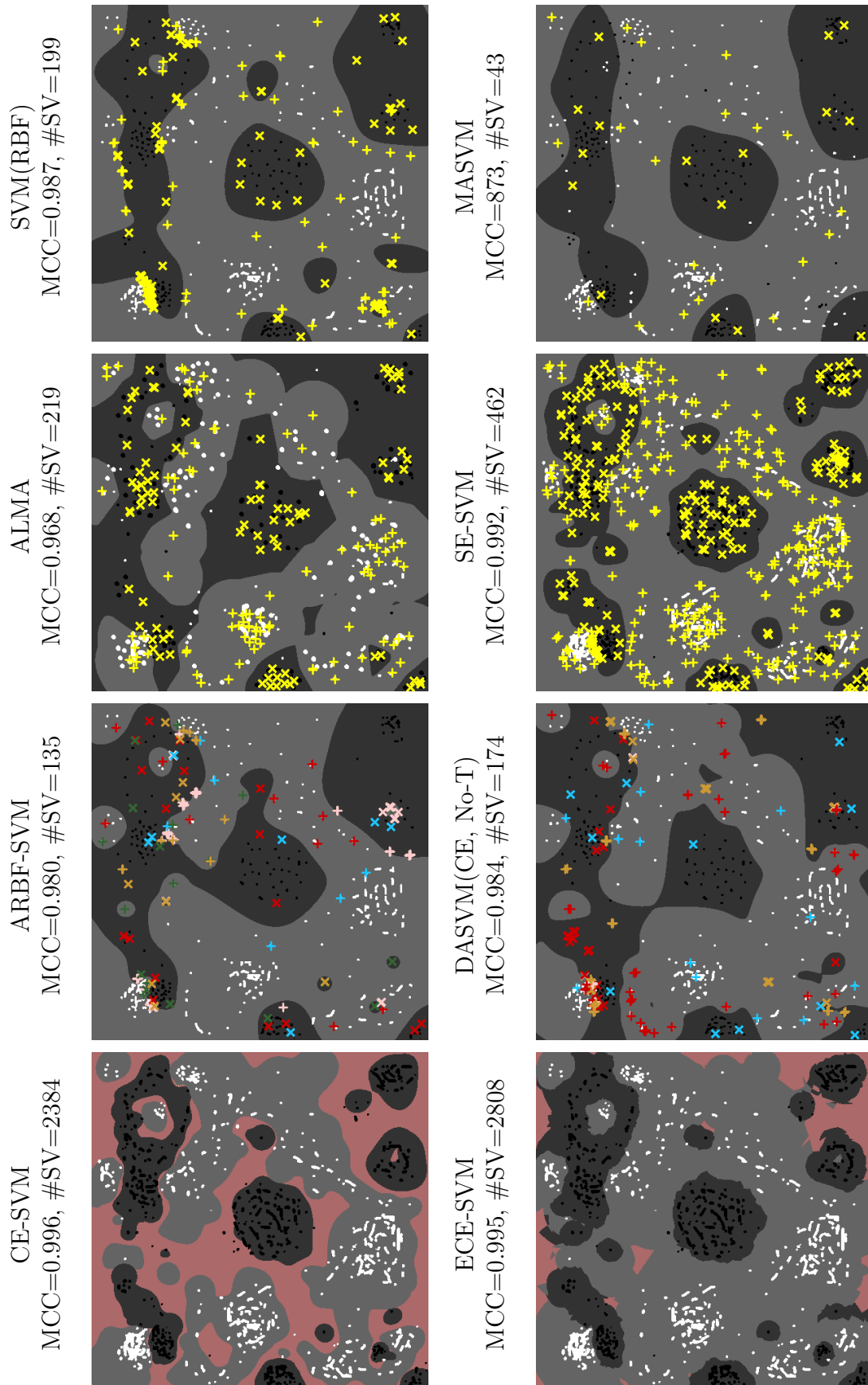


Figure 4.9: Qualitative comparisons on selected 2D dataset.

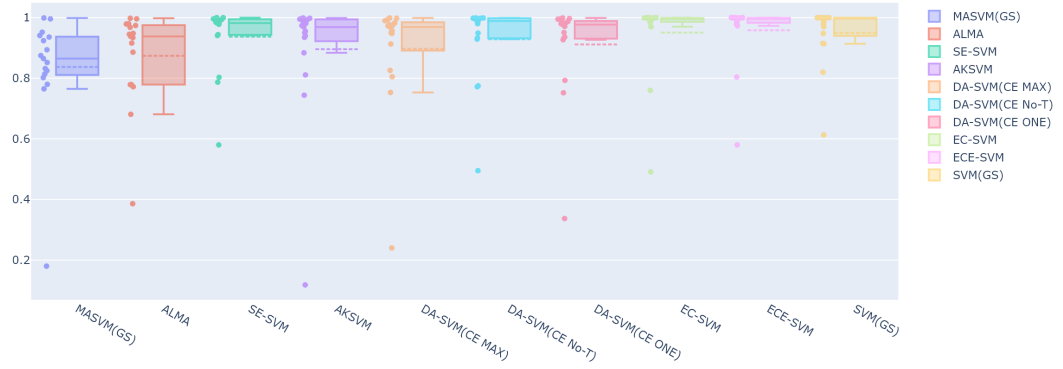


Figure 4.10: Box plot of MCC scores obtained on 2D datasets.



Figure 4.11: Pareto plots of MCC scores against a) classification time b) training time.

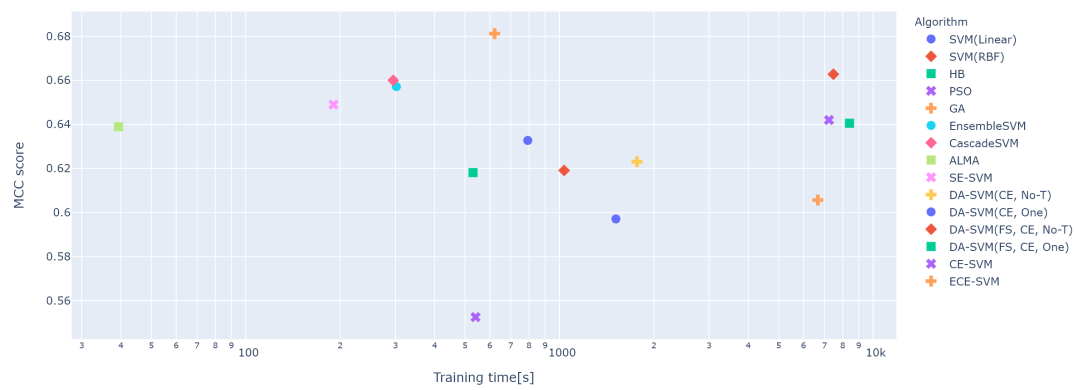


Figure 4.12: Pareto plot of MCC scores against training time.

Chapter 5

Conclusions

The focus of this dissertation was on the optimization of the training set, hyperparameters, and feature set for support vector machines by using evolutionary computation and building different forms of ensembles. In the beginning, the SVM and the theory behind it were introduced and the literature was reviewed. Based on that it could be spotted that optimizing multiple aspects of SVM simultaneously could be researched further to improve existing methods. The optimization of these parameters is a challenging task, and different approaches have been proposed to address this problem while there was no method available to tackle all of them. This research proposes novel methods that combine the optimization of these aspects ($\mathcal{M}$, $\mathbf{T}$, $\mathbf{F}$) using alternating and simultaneous optimization methods. Those methods are introduced in detail in Chapter 3. All of them are based on evolutionary computations as they proved to be efficient in finding reduced training sets as seen in MASVM method [95]. This method serves as a baseline for finding and optimizing $\mathbf{T}'$ while the new methods for optimizing hyperparameters and feature sets are proposed. These methods are then combined into alternating optimization and finally, simultaneous optimization is proposed. One of the advantage of developing such approaches is that each of those components (optimizing only one of $\mathcal{M}$, $\mathbf{T}$, $\mathbf{F}$) could be easily exchanged as proven with ALGA and ALMA algorithms. The latter one uses an improved MASVM method (over the original GASVM) where the whole scheme of alternating optimization is unchanged. With some additional work, the same approach could be used in SE-SVM algorithm

which brings interesting direction to future research where components of the algorithm could be exchanged. Then the algorithms are analyzed experimentally in section 4.3 where the results of these approaches are discussed and visualized using artificial datasets. The methods are also compared to other popular classifiers and state-of-the-art SVM optimization techniques using benchmark datasets.

The first thesis stating that **simultaneous optimization of the training set, the feature set and the hyperparameters of the SVM improve training and classification time compared to other state-of-the-art methods proposed for this purpose without affecting the classification quality** is proven by the SE-SVM methods compared to ALMA and MASVM methods. Analyzing both, the qualitative and quantitative results clearly show the SE-SVM approach can better utilize the mutual dependence of optimizing $\mathcal{M}$, $\mathbf{T}$ and $\mathbf{F}$ and improve classifier performance compared to the alternating optimization techniques. What is more the training and classification time were also analyzed. First of all those times are greatly reduced compared to regular grid search algorithm in both cases of ALMA and SE-SVM. Although ALMA algorithm had faster training and classification time it have worse classification performance when compared to SE-SVM. This means that SE-SVM is a pareto-optimal method which proves this thesis. Moreover, the DASVM provided promising results on 2D datasets but ultimately this method did not perform so well in benchmark setting where its quality is between ALMA and SE-SVM methods but training and classification times are longer.

The second thesis stating that **building ensembles of SVM using evolutionary algorithms provide improved classification performance compared to other SVM ensemble building algorithms and other well-established methods** is addressed with CE-SVM and ECE-SVM algorithms. These algorithms divide the dataset into different regions, and by building these regions into the cascade structure of classifiers, the SVM's performance and applicability are improved. One of the major improvements is to introduce the concept of certainty regions. The idea behind this approach is to identify regions within the dataset where the SVM can make accurate predictions with high confidence. Studies have shown that such expert regions can indeed be identified and that they can be classified with higher accuracy than the entire dataset. While this approach may require a longer

classification time, the results have been shown to be superior to other methods. It is important to note that although the training time for SVMs with expert regions may be increased compared to other methods such as ALMA and SE-SVM, the increased training time is a natural consequence of building multiple models to identify and focus on expert regions. What is more, those results are also pareto-optimal meaning that the longer training and classification time is associated with improved classification performance. Overall, incorporating the concept of expert regions into SVMs is a promising approach for improving their performance, particularly in scenarios where classification performance and confidence in predictions are crucial. While the approach may require more computational resources, the potential gains in classification quality and confidence make it a valuable technique for further usage.

5.1 Future work

The presented results and methods could be used as a great starting point for further research. There are numerous things that were introduced and tested while some of those seem (in the author's opinion) to be too briefly touched on.

The optimization of hyperparameters is a critical aspect of SVMs, as the performance of the model is highly dependent on the choice of hyperparameters. The hyperparameter optimization method proposed in this research is fairly simple, and coupling proposed algorithms with more advanced techniques could potentially lead to further improvements. For instance, the use of Bayesian optimization or gradient-based optimization techniques could lead to more efficient and accurate hyperparameter tuning. Additionally, incorporating prior knowledge or domain expertise into the optimization process could also be explored as a way to guide the search towards more promising hyperparameter configurations. Yet another interesting direction is finding promising hyperparameters by the greatly reducing size of the training set as shown with initial experimentation in Section 3.5 where grid search was used.

This research focused on binary classification problems, and extending it into multiclassification problems using specialized SVMs is another interesting direction. As shown in earlier work [93] using one-vs-one strategy to tackle those is not

efficient and state-of-the-art methods are better. Multiclassification problems are commonly encountered in many applications, and developing efficient and accurate methods for solving these problems is an important research area. The extension of the proposed algorithms to handle multiclassification problems could lead to new insights and improvements in performance. This should not be specifically hard to follow as there is no inherent limitation to the number of classes handled by the evolutionary method proposed.

Coupling ensemble methods with feature selection is another interesting direction for future research. Ensemble methods are known to improve the accuracy and robustness of machine learning models by combining multiple models' predictions. Feature selection, on the other hand, is a technique that selects the most relevant features from the dataset to improve the model's performance and reduce overfitting. Combining ensemble methods with feature selection could lead to further improvements in performance by selecting the most relevant features and increasing the diversity of included models. What is more this could make CE-SVM and ECE-SVM easier to use for non-expert users.

Lastly, the DA-SVM algorithm with the No-T variant presented in this research is an interesting approach that could be extended to other algorithms presented here. The selection of a small yet representative validation set is crucial for this algorithm to be effective, as it is used to estimate the fitness of the classifier and could be considered as a replacement for the training process. Further research could explore the use of the No-T variant in other algorithms and investigate the impact of the validation set size on performance.

Bibliography

- [1] ABDEL-BASSET, M., DING, W., AND EL-SHAHAT, D. A hybrid harris hawks optimization algorithm with simulated annealing for feature selection. *Artificial Intelligence Review* 54 (2021), 593–637.
- [2] ABE, S., AND INOUE, T. Fast training of support vector machines by extracting boundary data. In *Proc. International Conference on Artificial Neural Networks* (2001), Springer, pp. 308–313.
- [3] ABUSITTA, A., LI, M. Q., AND FUNG, B. C. Malware classification and composition analysis: A survey of recent developments. *Journal of Information Security and Applications* 59 (2021), 102828.
- [4] AIOLLI, F., AND DONINI, M. Easymkl: a scalable multiple kernel learning algorithm. *Neurocomputing* 169 (2015), 215–224.
- [5] ALADEEMY, M., TUTUN, S., AND KHASAWNEH, M. T. A new hybrid approach for feature selection and support vector machine model selection based on self-adaptive cohort intelligence. *Expert Systems with Applications* 88 (2017), 118–131.
- [6] ALBASHISH, D., HAMMOURI, A. I., BRAIK, M., ATWAN, J., AND SAHRAN, S. Binary biogeography-based optimization based svm-rfe for feature selection. *Applied Soft Computing* 101 (2021), 107026.
- [7] ANGIULLI, F., AND ASTORINO, A. Scaling up support vector machines using nearest neighbor condensation. *IEEE Transactions on Neural Networks* 21, 2 (2010), 351–357.

- [8] ASLAN, M. F., SABANCI, K., DURDU, A., AND UNLERSSEN, M. F. Covid-19 diagnosis using state-of-the-art cnn architecture features and bayesian optimization. *Computers in Biology and Medicine* 142 (2022), 105244.
- [9] AYAT, N., CHERIET, M., AND SUEN, C. Automatic model selection for the optimization of svm kernels. *Pattern Recognition* 38, 10 (2005), 1733–1745.
- [10] AYUSH, K., AND SINHA, A. Improving classification performance of support vector machines via guided custom kernel search. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (New York, NY, USA, 2019), GECCO '19, Association for Computing Machinery, p. 159–160.
- [11] BALCÁZAR, J., DAI, Y., AND WATANABE, O. A random sampling technique for training support vector machines. In *Algorithmic Learning Theory* (Berlin, Heidelberg, 2001), N. Abe, R. Khardon, and T. Zeugmann, Eds., Springer, pp. 119–134.
- [12] BALIARSINGH, S. K., DING, W., VIPITA, S., AND BAKSHI, S. A memetic algorithm using emperor penguin and social engineering optimization for medical data classification. *Applied Soft Computing* 85 (2019), 105773.
- [13] BENKESSIRAT, A., AND BENBLIDIA, N. Fundamentals of feature selection: An overview and comparison. In *2019 IEEE/ACS 16th International Conference on Computer Systems and Applications (AICCSA)* (2019), pp. 1–6.
- [14] BERGSTRA, J., BARDENET, R., BENGIO, Y., AND KÉGL, B. Algorithms for hyper-parameter optimization. *Advances in neural information processing systems* 24 (2011).
- [15] BERGSTRA, J., AND BENGIO, Y. Random search for hyper-parameter optimization. *Journal of Machine Learning Research* 13, 10 (2012), 281–305.
- [16] BERGSTRA, J., YAMINS, D., AND COX, D. D. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision

-
- architectures. In *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28* (2013), ICML'13, JMLR.org, p. I-115–I-123.
- [17] BISCHL, B., BINDER, M., LANG, M., PIELOK, T., RICHTER, J., COORS, S., THOMAS, J., ULLMANN, T., BECKER, M., BOULESTEIX, A.-L., ET AL. Hyperparameter optimization: Foundations, algorithms, best practices and open challenges. *arXiv preprint arXiv:2107.05847* (2021).
- [18] BOMMERT, A., SUN, X., BISCHL, B., RAHNENFÜHRER, J., AND LANG, M. Benchmark for filter methods for feature selection in high-dimensional classification data. *Computational Statistics & Data Analysis* 143 (2020), 106839.
- [19] BOSER, B. E., GUYON, I. M., AND VAPNIK, V. N. A training algorithm for optimal margin classifiers. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory* (New York, NY, USA, 1992), COLT '92, Association for Computing Machinery, p. 144–152.
- [20] BROWN, T., MANN, B., RYDER, N., SUBBIAH, M., KAPLAN, J. D., DHARIWAL, P., NEELAKANTAN, A., SHYAM, P., SASTRY, G., ASKELL, A., ET AL. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [21] CARCILLO, F., LE BORGNE, Y.-A., CAELEN, O., KESSACI, Y., OBLÉ, F., AND BONTEMPI, G. Combining unsupervised and supervised learning in credit card fraud detection. *Information sciences* 557 (2021), 317–331.
- [22] CERVANTES, J., LAMONT, F. G., LÓPEZ-CHAU, A., MAZAHUA, L. R., AND RUÍZ, J. S. Data selection based on decision tree for SVM classification on large data sets. *Applied Soft Computing* 37 (2015), 787–798.
- [23] CHANDRASHEKAR, G., AND SAHIN, F. A survey on feature selection methods. *Computers & Electrical Engineering* 40, 1 (2014), 16–28.

- [24] CHANG, C.-C., AND LIN, C.-J. Libsvm: A library for support vector machines. *ACM transactions on intelligent systems and technology (TIST)* 2, 3 (2011), 1–27.
- [25] CHAWLA, N. V., BOWYER, K. W., HALL, L. O., AND KEGELMEYER, W. P. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research* 16 (2002), 321–357.
- [26] CHEN, C., LI, X., BELKACEM, A. N., QIAO, Z., DONG, E., TAN, W., AND SHIN, D. The mixed kernel function SVM-based point cloud classification. *International Journal of Precision Engineering and Manufacturing* 20, 5 (2019), 737–747.
- [27] CHEN, L., LI, S., BAI, Q., YANG, J., JIANG, S., AND MIAO, Y. Review of image classification algorithms based on convolutional neural networks. *Remote Sensing* 13, 22 (2021), 4712.
- [28] CHENG, F., CHEN, J., QIU, J., AND ZHANG, L. A subregion division based multi-objective evolutionary algorithm for SVM training set selection. *Neurocomputing* 394 (2020), 70–83.
- [29] CHICCO, D., AND JURMAN, G. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC Genomics* 21, 1 (Jan 2020), 6.
- [30] CHICCO, D., TÖTSCH, N., AND JURMAN, G. The matthews correlation coefficient (mcc) is more reliable than balanced accuracy, bookmaker informedness, and markedness in two-class confusion matrix evaluation. *BioData Mining* 14, 1 (Feb 2021), 13.
- [31] CLAESEN, M., DE SMET, F., SUYKENS, J. A. K., AND DE MOOR, B. Ensemblesvm: A library for ensemble learning using support vector machines. *J. Mach. Learn. Res.* 15, 1 (jan 2014), 141–145.
- [32] CORTES, C., AND VAPNIK, V. Support vector networks. *Machine Learning* 20 (1995), 273–297.

-
- [33] CZARNOWSKI, I. Cluster-based instance selection for machine classification. *Knowledge and Information Systems* 30, 1 (2012), 113–133.
 - [34] DHIMAN, H. S., DEB, D., MUYEEN, S., AND KAMWA, I. Wind turbine gearbox anomaly detection based on adaptive threshold and twin support vector machines. *IEEE Transactions on Energy Conversion* 36, 4 (2021), 3462–3469.
 - [35] DIXON, M. F., HALPERIN, I., AND BILOKON, P. *Machine learning in Finance*, vol. 1170. Springer, 2020.
 - [36] DUDZIK, W., KAWULOK, M., AND NALEPA, J. Evolutionarily-tuned support vector machines. In *Genetic and Evolutionary Computation Conference Companion* (New York, NY, USA, 2019), GECCO '19, Association for Computing Machinery, pp. 165–166.
 - [37] DUDZIK, W., KAWULOK, M., AND NALEPA, J. Optimizing training data and hyperparameters of support vector machines using a memetic algorithm. In *Man-Machine Interactions - 6th International Conference on Man-Machine Interactions ICMMI* (2019), pp. 229–238.
 - [38] DUDZIK, W., NALEPA, J., AND KAWULOK, M. Automated optimization of non-linear support vector machines for binary classification. In *Advances in Intelligent Networking and Collaborative Systems* (Cham, 2019), F. Xhafa, L. Barolli, and M. Greguš, Eds., Springer International Publishing, pp. 504–513.
 - [39] DUDZIK, W., NALEPA, J., AND KAWULOK, M. Evolving data-adaptive support vector machines for binary classification. *Knowledge-Based Systems* 227 (2021), 107221.
 - [40] DUDZIK, W., NALEPA, J., AND KAWULOK, M. Cascades of evolutionary support vector machines. In *GECCO '22: Genetic and Evolutionary Computation Conference, Companion Volume, Boston, Massachusetts, USA, July 9 - 13, 2022* (2022), ACM, pp. 240–243.

- [41] FALKNER, S., KLEIN, A., AND HUTTER, F. BOHB: Robust and efficient hyperparameter optimization at scale. In *Proceedings of the 35th International Conference on Machine Learning* (10–15 Jul 2018), J. Dy and A. Krause, Eds., vol. 80 of *Proceedings of Machine Learning Research*, PMLR, pp. 1437–1446.
- [42] FARIS, H., HASSONAH, M. A., AL-ZOUBI, A. M., MIRJALILI, S., AND ALJARAHA, I. A multi-verse optimizer approach for feature selection and optimizing SVM parameters based on a robust system architecture. *Neural Computing and Applications* 30, 8 (Oct 2018), 2355–2369.
- [43] FAWCETT, T. An introduction to roc analysis. *Pattern Recognition Letters* 27, 8 (2006), 861–874. ROC Analysis in Pattern Recognition.
- [44] FLOREA, A. C., AND ANDONIE, R. A dynamic early stopping criterion for random search in svm hyperparameter optimization. In *IFIP International Conference on Artificial Intelligence Applications and Innovations* (2018), Springer, pp. 168–180.
- [45] FRIEDMAN, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 29, 5 (2001), 1189 – 1232.
- [46] GAO, L., QIAN, W., LI, X., AND WANG, J. Application of memetic algorithm in assembly sequence planning. *The International Journal of Advanced Manufacturing Technology* 49, 9 (Aug 2010), 1175–1184.
- [47] GARCIA, S., DERRAC, J., CANO, J., AND HERRERA, F. Prototype selection for nearest neighbor classification: Taxonomy and empirical study. *IEEE transactions on pattern analysis and machine intelligence* 34, 3 (2012), 417–435.
- [48] GARCÍA, V., MOLLINEDA, R. A., AND SÁNCHEZ, J. S. Index of balanced accuracy: A performance measure for skewed class distributions. In *Pattern Recognition and Image Analysis* (Berlin, Heidelberg, 2009), H. Araujo, A. M. Mendonça, A. J. Pinho, and M. I. Torres, Eds., Springer, pp. 441–448.

-
- [49] GARCÍA-PEDRAJAS, N., DE HARO-GARCÍA, A., AND PÉREZ-RODRÍGUEZ, J. A scalable memetic algorithm for simultaneous instance and feature selection. *Evolutionary Computation* 22, 1 (2014), 1–45.
 - [50] GEURTS, P., ERNST, D., AND WEHENKEL, L. Extremely randomized trees. *Machine Learning* 63, 1 (Apr 2006), 3–42.
 - [51] GHAMISI, P., AND BENEDIKTSSON, J. A. Feature selection based on hybridization of genetic algorithm and particle swarm optimization. *IEEE Geoscience and Remote Sensing Letters* 12, 2 (Feb 2015), 309–313.
 - [52] GHAMISI, P., COUCEIRO, M. S., AND BENEDIKTSSON, J. A. A novel feature selection approach based on fodpso and SVM. *IEEE Transactions on Geoscience and Remote Sensing* 53, 5 (May 2015), 2935–2947.
 - [53] GOEL, A., AND SRIVASTAVA, S. K. Role of kernel parameters in performance evaluation of svm. In *2016 Second international conference on computational intelligence & communication technology (CICT)* (2016), IEEE, pp. 166–169.
 - [54] GOMEZ-URIBE, C. A., AND HUNT, N. The netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems (TMIS)* 6, 4 (2015), 1–19.
 - [55] GOPI, A. P., JYOTHI, R. N. S., NARAYANA, V. L., AND SANDEEP, K. S. Classification of tweets data based on polarity using improved rbf kernel of svm. *International Journal of Information Technology* (Jan 2020).
 - [56] GRAF, H., COSATTO, E., BOTTOU, L., DOURDANOVIC, I., AND VAPNIK, V. Parallel support vector machines: The cascade svm. In *Advances in Neural Information Processing Systems* (2004), L. Saul, Y. Weiss, and L. Bottou, Eds., vol. 17, MIT Press.
 - [57] GUO, L., AND BOUKIR, S. Fast data selection for SVM training using ensemble margin. *Pattern Recognition Letters* 51 (2015), 112–119.

- [58] GUO, L., BOUKIR, S., AND CHEHATA, N. Support vectors selection for supervised learning using an ensemble approach. In *2010 20th International Conference on Pattern Recognition* (2010), pp. 37–40.
- [59] GUYON, I., WESTON, J., BARNHILL, S., AND VAPNIK, V. Gene selection for cancer classification using support vector machines. *Machine Learning* 46, 1 (Jan 2002), 389–422.
- [60] HERRERA, F., LOZANO, M., AND VERDEGAY, J. L. Tackling real-coded genetic algorithms: Operators and tools for behavioural analysis. *Artificial Intelligence Review* 12, 4 (Aug 1998), 265–319.
- [61] HOFMANN, T., SCHÖLKOPF, B., AND SMOLA, A. J. Kernel methods in machine learning. *The Annals of Statistics* 36, 3 (2008), 1171 – 1220.
- [62] HOLLAND, J. H. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, 1992.
- [63] HOUSSEIN, E. H., HOSNEY, M. E., OLIVA, D., MOHAMED, W. M., AND HASSABALLAH, M. A novel hybrid harris hawks optimization and support vector machines for drug design and discovery. *Computers & Chemical Engineering* 133 (2020), 106656.
- [64] HSU, C.-W., CHANG, C.-C., AND LIN, C.-J. A practical guide to support vector classification. Tech. rep., Department of Computer Science, National Taiwan University, 2003.
- [65] HUANG, C.-L., AND DUN, J.-F. A distributed pso–svm hybrid system with feature selection and parameter optimization. *Applied Soft Computing* 8, 4 (2008), 1381–1391.
- [66] HUANG, S., CAI, N., PACHECO, P. P., NARRANDES, S., WANG, Y., AND XU, W. Applications of support vector machine (SVM) learning in cancer genomics. *Cancer Genomics Proteomics* 15, 1 (Jan. 2018), 41–51.
- [67] HUANRUI, H. New mixed kernel functions of SVM used in pattern recognition. *Cybernetics and Information Technologies* 16, 5 (2016), 5–14.

-
- [68] HUSSAIN, S. F. A novel robust kernel for classifying high-dimensional data using support vector machines. *Expert Systems with Applications* 131 (2019), 116–131.
- [69] HUTTER, F., HOOS, H. H., AND LEYTON-BROWN, K. Sequential model-based optimization for general algorithm configuration. In *Proceedings of the 5th International Conference on Learning and Intelligent Optimization* (Berlin, Heidelberg, 2011), LION’05, Springer-Verlag, p. 507–523.
- [70] IBRAHIM, H. T., MAZHER, W. J., UCAN, O. N., AND BAYAT, O. A grasshopper optimizer approach for feature selection and optimizing SVM parameters utilizing real biomedical data sets. *Neural Computing and Applications* 31 (2019), 5965–5974.
- [71] JAMIESON, K., AND TALWALKAR, A. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics* (Cadiz, Spain, 09–11 May 2016), A. Gretton and C. C. Robert, Eds., vol. 51 of *Proceedings of Machine Learning Research*, PMLR, pp. 240–248.
- [72] JIANG, H., CHING, W.-K., YIU, K. F. C., AND QIU, Y. Stationary mahalanobis kernel SVM for credit risk evaluation. *Applied Soft Computing* 71 (2018), 407–417.
- [73] KATOCH, S., CHAUHAN, S. S., AND KUMAR, V. A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications* 80, 5 (Feb 2021), 8091–8126.
- [74] KAWULOK, M., AND NALEPA, J. Support vector machines training data selection using a genetic algorithm. In *Structural, Syntactic, and Statistical Pattern Recognition* (Berlin, Heidelberg, 2012), G. Gimel’farb, E. Hancock, A. Imiya, A. Kuijper, M. Kudo, S. Omachi, T. Windeatt, and K. Yamada, Eds., Springer, pp. 557–565.
- [75] KAWULOK, M., NALEPA, J., AND DUDZIK, W. An alternating genetic algorithm for selecting SVM model and training set. In *Mexican Conference on Pattern Recognition* (2017), Springer, pp. 94–104.

- [76] KLEIN, A., FALKNER, S., BARTELS, S., HENNIG, P., AND HUTTER, F. Fast bayesian optimization of machine learning hyperparameters on large datasets. In *Artificial intelligence and statistics* (2017), PMLR, pp. 528–536.
- [77] KUO, B.-C., HO, H.-H., LI, C.-H., HUNG, C.-C., AND TAUR, J.-S. A kernel-based feature selection method for svm with rbf kernel for hyperspectral image classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 7, 1 (2013), 317–326.
- [78] KURANI, A., DOSHI, P., VAKHARIA, A., AND SHAH, M. A comprehensive comparative study of artificial neural network (ann) and support vector machines (svm) on stock forecasting. *Annals of Data Science* 10, 1 (Feb 2023), 183–208.
- [79] LE, T. T., FU, W., AND MOORE, J. H. Scaling tree-based automated machine learning to biomedical big data with a feature set selector. *Bioinformatics* 36, 1 (2020), 250–256.
- [80] LESSMANN, S., STAHLBOCK, R., AND CRONE, S. F. Genetic algorithms for support vector machine model selection. In *The 2006 IEEE International Joint Conference on Neural Network Proceedings* (2006), IEEE, pp. 3063–3069.
- [81] LI, L., JAMIESON, K., DESALVO, G., ROSTAMIZADEH, A., AND TALWALKAR, A. Hyperband: A novel bandit-based approach to hyperparameter optimization. *The Journal of Machine Learning Research* 18, 1 (2017), 6765–6816.
- [82] LIN, C.-C., KANG, J.-R., LIANG, Y.-L., AND KUO, C.-C. Simultaneous feature and instance selection in big noisy data using memetic variable neighborhood search. *Applied Soft Computing* 112 (2021), 107855.
- [83] LOUPPE, G., AND GEURTS, P. Ensembles on random patches. In *Machine Learning and Knowledge Discovery in Databases* (Berlin, Heidelberg, 2012), P. A. Flach, T. De Bie, and N. Cristianini, Eds., Springer Berlin Heidelberg, pp. 346–361.

-
- [84] LU, C., ZHANG, B., GAO, L., YI, J., AND MOU, J. A knowledge-based multiobjective memetic algorithm for green job shop scheduling with variable machining speeds. *IEEE Systems Journal* 16, 1 (2021), 844–855.
- [85] MALEKI, N., ZEINALI, Y., AND NIAKI, S. T. A. A k-nn method for lung cancer prognosis with the use of a genetic algorithm for feature selection. *Expert Systems with Applications* 164 (2021), 113981.
- [86] MEINSHAUSEN, N., AND BÜHLMANN, P. Stability selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 72, 4 (2010), 417–473.
- [87] MENEZES, M. V., TORRES, L. C., AND BRAGA, A. P. Width optimization of RBF kernels for binary classification of support vector machines: A density estimation-based approach. *Pattern Recognition Letters* 128 (2019), 1–7.
- [88] MICHALEWICZ, Z. *Genetic algorithms + data structures = evolution programs (3rd, extended ed.)*. Springer-Verlag New York, Inc., New York, NY, USA, 1996.
- [89] MITCHELL, M. Genetic algorithms: An overview. In *Complex.* (1995), vol. 1, Citeseer, pp. 31–39.
- [90] MOLINA, J. C., SALMERON, J. L., AND EGUIA, I. An acs-based memetic algorithm for the heterogeneous vehicle routing problem with time windows. *Expert Systems with Applications* 157 (2020), 113379.
- [91] MOSCATO, P. On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. Tech. Rep. C3P Report 826, California Institute of Technology, 1989.
- [92] MOSCATO, P., AND MATHIESON, L. *Memetic Algorithms for Business Analytics and Data Science: A Brief Survey*. Springer International Publishing, Cham, 2019, pp. 545–608.
- [93] NALEPA, J., DUDZIK, W., AND KAWULOK, M. Memetic evolution of training sets with adaptive radial basis kernels for support vector machines.

- In *International Conference on Pattern Recognition* (2020), pp. 1–8. (in press).
- [94] NALEPA, J., AND KAWULOK, M. Adaptive genetic algorithm to select training data for support vector machines. In *Applications of Evolutionary Computation* (Berlin, Heidelberg, 2014), A. I. Esparcia-Alcázar and A. M. Mora, Eds., Springer, pp. 514–525.
- [95] NALEPA, J., AND KAWULOK, M. A memetic algorithm to select training data for support vector machines. In *Genetic and Evolutionary Computation Conference* (New York, NY, USA, 2014), GECCO '14, Association for Computing Machinery, pp. 573–580.
- [96] NALEPA, J., AND KAWULOK, M. Adaptive memetic algorithm enhanced with data geometry analysis to select training data for SVMs. *Neurocomputing* 185 (2016), 113–132.
- [97] NALEPA, J., AND KAWULOK, M. Selecting training sets for support vector machines: a review. *Artificial Intelligence Review* 52, 2 (2019), 857–900.
- [98] NALEPA, J., SIMINSKI, K., AND KAWULOK, M. Towards parameter-less support vector machines. In *Proc. ACPR* (2015), pp. 211–215.
- [99] NANGLIA, S., AHMAD, M., ALI KHAN, F., AND JHANJHI, N. An enhanced predictive heterogeneous ensemble model for breast cancer prediction. *Biomedical Signal Processing and Control* 72 (2022), 103279.
- [100] NEUMANN, J., SCHNÖRR, C., AND STEIDL, G. Combined SVM-based feature selection and classification. *Machine Learning* 61, 1 (Nov 2005), 129–150.
- [101] OGUTU, J. O., SCHULZ-STREECK, T., AND PIEPHO, H.-P. Genomic selection using regularized linear regression models: ridge regression, lasso, elastic net and their extensions. *BMC Proceedings* 6, 2 (May 2012), S10.
- [102] PANCH, T., SZOLOVITS, P., AND ATUN, R. Artificial intelligence, machine learning and health systems. *J. Glob. Health* 8, 2 (Dec. 2018), 020303.

-
- [103] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M., AND DUCHESNAY, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [104] PLATT, J. Sequential minimal optimization: A fast algorithm for training support vector machines. Tech. Rep. MSR-TR-98-14, Microsoft, April 1998.
- [105] PUGALENTI, R., RAJAKUMAR, M., RAMYA, J., AND RAJINIKANTH, V. Evaluation and classification of the brain tumor mri using machine learning technique. *Journal of Control Engineering and Applied Informatics* 21, 4 (2019), 12–21.
- [106] PŁAWIAK, P., ABDAR, M., AND RAJENDRA ACHARYA, U. Application of new deep genetic cascade ensemble of svm classifiers to predict the australian credit scoring. *Applied Soft Computing* 84 (2019), 105740.
- [107] QAYYUM, A., QADIR, J., BILAL, M., AND AL-FUQAHA, A. Secure and robust machine learning for healthcare: A survey. *IEEE Reviews in Biomedical Engineering* 14 (2020), 156–180.
- [108] RAMAN, M. G., SOMU, N., KIRTHIVASAN, K., LISCANO, R., AND SRIRAM, V. S. An efficient intrusion detection system based on hypergraph-genetic algorithm for parameter optimization and feature selection in support vector machine. *Knowledge-Based Systems* 134 (2017), 1–12.
- [109] RAMIREZ-MORALES, A., SALMON-GAMBOA, J. U., LI, J., SANCHEZ-REYNA, A. G., AND PALLI-VALAPPIL, A. Boosted support vector machines with genetic selection. *Applied Intelligence* 53, 5 (Mar 2023), 4996–5012.
- [110] SENI, G., AND ELDER, J. *Ensemble methods in data mining: improving accuracy through combining predictions*. Morgan & Claypool Publishers, 2010.

- [111] SHEN, X.-J., MU, L., LI, Z., WU, H.-X., GOU, J.-P., AND CHEN, X. Large-scale support vector machine classification with redundant data reduction. *Neurocomputing* 172 (2016), 189–197.
- [112] SIVANANDAM, S., DEEPA, S., SIVANANDAM, S., AND DEEPA, S. *Genetic algorithms*. Springer, 2008.
- [113] SNOEK, J., LAROCHELLE, H., AND ADAMS, R. P. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems* 25 (2012).
- [114] SPENCER, R., THABTAH, F., ABDELHAMID, N., AND THOMPSON, M. Exploring feature selection and classification methods for predicting heart disease. *Digital health* 6 (2020), 2055207620914777.
- [115] STANLEY, K. O., AND MIIKKULAINEN, R. Competitive coevolution through evolutionary complexification. *Journal of Artificial Intelligence Research* 21, 1 (Feb. 2004), 63–100.
- [116] SUDHOLT, D. *The Benefits of Population Diversity in Evolutionary Algorithms: A Survey of Rigorous Runtime Analyses*. Springer International Publishing, Cham, 2020, pp. 359–404.
- [117] SULTAN, R., TAMIMI, H., AND ASHHAB, Y. Improving classification performance using genetic programming to evolve string kernels. *The International Arab Journal of Information Technology (IAJIT)* 16, 3 (2019), 454–459.
- [118] TAN, T. G., LAU, H. K., AND TEO, J. Cooperative versus competitive coevolution for pareto multiobjective optimization. In *Bio-Inspired Computational Intelligence and Applications* (Berlin, Heidelberg, 2007), K. Li, M. Fei, G. W. Irwin, and S. Ma, Eds., Springer, pp. 63–72.
- [119] TANG, J., ALELYANI, S., AND LIU, H. Feature selection for classification: A review. *Data classification: Algorithms and applications* (2014), 37.

-
- [120] TANG, J., LIM, M. H., AND ONG, Y. S. Diversity-adaptive parallel memetic algorithm for solving large scale combinatorial optimization problems. *Soft Computing* 11, 9 (2007), 873–888.
 - [121] TAO, P., SUN, Z., AND SUN, Z. An improved intrusion detection algorithm based on ga and svm. *IEEE Access* 6 (2018), 13624–13631.
 - [122] TAO, Z., HUILING, L., WENWEN, W., AND XIA, Y. GA-SVM based feature selection and parameter optimization in hospitalization expense modeling. *Applied Soft Computing* 75 (2019), 323–332.
 - [123] TIAN, D., ZHAO, X., AND SHI, Z. Support vector machine with mixture of kernels for image classification. In *International Conference on Intelligent Information Processing* (2012), Springer, pp. 68–76.
 - [124] TING, K. M., ZHU, Y., AND ZHOU, Z.-H. Isolation kernel and its effect on SVM. In *The International Conference on Knowledge Discovery & Data Mining* (New York, NY, USA, 2018), KDD '18, Association for Computing Machinery, pp. 2329–2337.
 - [125] WAINER, J., AND FONSECA, P. How to tune the rbf svm hyperparameters? an empirical evaluation of 18 search algorithms. *Artificial Intelligence Review* 54, 6 (Aug 2021), 4771–4797.
 - [126] WAINER, J., AND FONSECA, P. How to tune the rbf svm hyperparameters? an empirical evaluation of 18 search algorithms. *Artificial Intelligence Review* 54, 6 (2021), 4771–4797.
 - [127] WANG, D., QIAO, H., ZHANG, B., AND WANG, M. Online support vector machine based on convex hull vertices selection. *IEEE Transactions on Neural Networks and Learning Systems* 24, 4 (2013), 593–609.
 - [128] WANG, D., WANG, X., AND LV, S. An overview of end-to-end automatic speech recognition. *Symmetry* 11, 8 (2019), 1018.
 - [129] WANG, L.-Y., ZHANG, J., AND LI, H. An improved genetic algorithm for tsp. In *2007 International Conference on Machine Learning and Cybernetics* (2007), vol. 2, IEEE, pp. 925–928.

- [130] WARING, J., LINDVALL, C., AND UMETON, R. Automated machine learning: Review of the state-of-the-art and opportunities for healthcare. *Artificial Intelligence in Medicine* 104 (2020), 101822.
- [131] WEN, Z., SHI, J., LI, Q., HE, B., AND CHEN, J. ThunderSVM: A fast SVM library on GPUs and CPUs. *Journal of Machine Learning Research* 19 (2018), 797–801.
- [132] WOLPERT, D. H. Stacked generalization. *Neural Networks* 5, 2 (1992), 241–259.
- [133] WOLPERT, D. H. *The Supervised Learning No-Free-Lunch Theorems*. Springer London, London, 2002, pp. 25–42.
- [134] XIAO, J. Svm and knn ensemble learning for traffic incident detection. *Physica A: Statistical Mechanics and its Applications* 517 (2019), 29–35.
- [135] XU, Z., AGHAABBASI, M., ALI, M., AND MACIOSZEK, E. Targeting sustainable transportation development: The support vector machine and the bayesian optimization algorithm for classifying household vehicle ownership. *Sustainability* 14, 17 (Sep 2022), 11094.
- [136] YANG, L., AND SHAMI, A. On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing* 415 (2020), 295–316.
- [137] YAO, G., HU, X., AND WANG, G. A novel ensemble feature selection method by integrating multiple ranking information combined with an svm ensemble model for enterprise credit risk prediction in the supply chain. *Expert Systems with Applications* 200 (2022), 117002.
- [138] ZANTALIS, F., KOULOURAS, G., KARABETSOS, S., AND KANDRIS, D. A review of machine learning and iot in smart transportation. *Future Internet* 11, 4 (2019), 94.
- [139] ZENG, N., QIU, H., WANG, Z., LIU, W., ZHANG, H., AND LI, Y. A new switching-delayed-pso-based optimized svm algorithm for diagnosis of alzheimer’s disease. *Neurocomputing* 320 (2018), 195–202.

-
- [140] ZHANG, S., XU, J., HUANG, E., AND CHEN, C.-H. A new optimal sampling rule for multi-fidelity optimization via ordinal transformation. In *2016 IEEE International Conference on Automation Science and Engineering (CASE)* (2016), IEEE Press, p. 670–674.
 - [141] ZHANG, W., HONG, B., LIU, W., YE, J., CAI, D., HE, X., AND WANG, J. Scaling up sparse support vector machines by simultaneous feature and sample reduction. In *Proc. International Conference on Machine Learning* (2017), pp. 4016–4025.
 - [142] ZHANG, W., AND KING, I. Locating support vectors via β -skeleton technique. In *Proc. International Conference on Neural Information Processing* (2002), pp. 1423–1427.
 - [143] ZHOU, J., QIU, Y., ZHU, S., ARMAGHANI, D. J., LI, C., NGUYEN, H., AND YAGIZ, S. Optimization of support vector machine through the use of metaheuristic algorithms in forecasting tbm advance rate. *Engineering Applications of Artificial Intelligence* 97 (2021), 104015.

Extras

Acknowledgements

TODO

Firstly, I would like to thank my supervisor, Michał Kawulok, for his exceptional guidance and patience throughout this journey. His invaluable insights and continuous support have been instrumental in shaping my research, and I have learned a great deal from our discussions. I would also like to extend my sincere thanks to my co-supervisor, Jakub Nalepa, for his countless comments, all of the grammar corrections, and guidance in the scientific community. His unwavering support and feedback have been indispensable in helping me achieve my goals.

I cannot thank my wife enough for her unwavering support, encouragement, and belief in me. She has been my pillar of strength, reminding me to take much-needed breaks and providing me with the support I needed to push through the difficult times. Thank you for all the laughs along the way. My heartfelt thanks also go out to my entire family for their support throughout my academic journey. I am grateful for all the help they have provided over the years.

This work has been co-financed by the European Union through the European Social Fund (grant POWR.03.02.00-00-I029).

Appendix

.1 Recursive features elimination

For initial testing, whether optimizing $\mathcal{M}$ and $\mathbf{T}$ together with $\mathbf{F}$ could further improve the results the Recursive Feature Elimination with cross-validation (RFECV) was exploited. It is a simple yet effective way of finding a refined feature set that can be used with a variety of models.

It works by recursively removing the least important features until the desired number of features is reached. Hence it could be computationally expensive for a large number of features as for each iteration RFE requires training of the machine learning algorithm. In order to decrease the number of training's required, more than one feature could be removed in each iteration. A variant of RFE is presented in Algorithm 10.

In the beginning, a range of steps to test is created, where a step of 10% of features (line 1) is utilized. For each of these steps (starting from full feature set), a 5-fold cross-validation (lines 3-7) is performed. A classifier ($R_{\mathbf{T}'}$) is trained and scored in each fold (lines 5-6). The score from CV is averaged before getting to the next smaller set of features (line 8). The training process (line 10) results in the selection of Z_{best} that yields the best score. The R is trained one more time for the entire $\mathbf{T}$ (line 11) and the least important features are pruned from R_{best} to obtain Z_{best} number of features, and returned as final $\mathbf{F}'$ (line 12).

An extremely randomized tree classifier [50] is used here. The motivation for using this method is its short training time, its non-linearity and the discarding of features based on importance that is natural for this classifier. It may happen that this process discards no features.

Algorithm 10 Recursive Feature Elimination with Cross-Validation (RFECV).

- 1: $\{Z_i\} = \{1.0, 0.9, 0.8, \dots, 0.1\};$ $\triangleright$ Set of steps
 - 2: **for all** $\{Z_i\}$ **do**
 - 3: $\mathbf{T}', \Psi \leftarrow$ Split $\mathbf{T}$ with $\mathbf{F}' = \mathbf{F} \times Z_i$ for 5-fold CV .
 - 4: **for all** $\mathbf{T}'$ **do** $\triangleright$ cross-validation for $\mathbf{T}$
 - 5: $R_{\mathbf{T}'} \leftarrow$ Train classifier on $\mathbf{T}'$
 - 6: $\{Score_{\mathbf{T}'}\} \leftarrow$ EVALUATE($R_{\mathbf{T}'}, \Psi$)
 - 7: $\{Score_{\mathbf{T}'}\} \leftarrow$ AVERAGE($\{Score_{\mathbf{T}'}\}$)
 - 8: $Z_{best} \leftarrow$ Take Step for MAX($\{Score_{\mathbf{T}'}\}$)
 - 9: $R_{best} \leftarrow$ Train classifier for $\mathbf{T}$
 - 10: **return** Take $\mathbf{F}'$ from R_{best} with Z_{best}
-

List of symbols

training set Refers to part of dataset used to train supervised classifier. 42, 54

List of Figures

2.1	(a) An illustrative example of simple 2D datasets with two classes (black and white dots) where multiple separating hyperplanes are denoted with dotted lines. (b) An example of a linear SVM trained on the presented data. The support vectors are denoted with red color, while the decision boundary is presented with a solid line. . .	12
2.2	An example of the soft-margin linear SVM. The misclassified example is denoted with a yellow circle.	15
2.3	An example of data that is non-linearly separable. Green triangles present a positive class while red squares present a negative class. The hyperplane is rendered with a blue solid line.	17
2.4	One of possible mapping of data from Figure 2.3 into 3D space. Here the decision boundary is seen as a blue 3D plane.	18
2.5	Comparison of different C hyperparameter setting for the linear kernel. Yellow crosses denote SVs, whereas dark and light gray denote values of decision function – the boundary between them is the decision hyperplane.	20
2.6	Comparison of different SVM kernels (linear, RBF and polynomial) and their hyperparameters. Yellow crosses denote SVs, whereas dark and light gray denote values of decision function – the boundary between them is the decision hyperplane. Above each example, the kernel with a set of hyperparameters is given together with the accuracy of the model.	21

2.7	Example of RS and GS of nine trials for optimizing function $f(x, y) = g(x) + h(x)$. The importance of the hyperparameters is illustrative and can be obtained <i>a posteriori</i> . This figure is inspired by [15]. . .	29
2.8	General categories of approaches for selecting SVM training sets. This figure is inspired by [97].	37
2.9	Overview of MASVM algorithm. Source: [97].	41
3.1	Flow diagram of the GA used for hyperparameter optimization. . .	55
3.2	The design of chromosome for SE-SVM method. Source: [37].	64
3.3	Assigning different γ 's in the RBF kernel to different $\mathbf{T}$ vectors can help better “model” the peculiarities of the SVM hyperplane: (a) our example set (we consider binary classification, white and black pixels present examples from two different classes), (b) the crosses in different colours render SVs with different γ 's: $\gamma = 10^2$ (blue), $\gamma = 10^3$ (red), $\gamma = 10^4$ (green), and $\gamma = 10^5$ (orange), together with (c) the different shades of gray show the decision boundary, (d) the “range of influence” of all SVs (the brighter/darker the pixel is, the distance to the hyperplane is larger). The SVM regularization parameter was grid-searched and was $C = 10$. This figure comes from our paper [93]	68
3.4	The impact of the K value on the quality of hyperparameter configurations extracted for example datasets: (a) 2D-Blobs, (b) german, (c) spambase. For the details of these sets (their sizes and characteristics), see Section 4.1. This figure comes from our paper [39]	72

3.5	Example hyperplanes extracted using various methods show that mixing the linear and RBF kernels may help obtain “less complicated” hyperplanes (hence possibly not overfitted to $\mathbf{T}$). We visualize (a) an example training set (with white and black dots presenting two-class vectors), (b) a linear-kernel SVM (with yellow crosses showing the SVs), (c) a hyperplane obtained by ARBF-SVM [93] and (d) DA-SVM(Mix). In (d), the purple crosses render the SVs obtained for the linear kernel while the others visualize the SVs for various γ ’s for the RBF kernel. This figure comes from our paper [39]	74
3.6	An outline of our cascade. At each level, a new node is evolved (blue boxes), composed of a lightweight SVM model and thresholds that split the input space into certain (yellow shades) and uncertain (red shade) regions. The training data falling into the uncertain region are passed to the next level, and the final node is based on SE-SVM. Classification (waved arrows) is performed with the evolved cascade. This figure comes from our paper [40]	80
3.7	Visualization of certainty thresholds.	81
4.1	Visualization of tests sets for 2D datasets.	91
4.2	Visualization of the ALMA algorithm run. Results are visualized on 2D-Blobs test set. The yellow ticks and crosses mark the positions of support vectors. (a) presents a solution after initialization, (b) presents the end solution after kernel evolution phase, (c)—(e) shows progress of $\mathbf{T}'$ evolution and growing of its size (f) presents final solution where hyperparameters were adjusted.	95
4.3	Visualization of the SESVM algorithm run.	96

4.4	Visualization of ARBF-SVM algorithm. (a) presents the best solution from the initial population, (b) The best solution after finishing evolution with first γ from $\vec{\gamma}$, (c) Shrank training set that will be used in next iteration with subsequent γ . The shrinking procedure is based on the whole population, (d) Solution after second evolution has ended. Added new support vectors marked with red color crosses, (e) Adding next γ value marked with green vectors provided worse classification performance, these support vectors will be removed, (f) final solution for a given dataset containing three different γ values.	97
4.5	Visualization of the CE-SVM algorithm run. (a)-(c) shows the evolution process and adding new nodes. Red color denotes uncertain region, dark and light grey presents certain regions of previous nodes while yellow colors are used to depict new certain regions added by the current node. (d) shows final classification with uncertain regions (e) presents how SE-SVM node to resolve those regions works and (f) shows the final classification result of joining cascade with SE-SVM.	99
4.6	Visualization of the ECE-SVM algorithm run. (a) and (d) Present a single cascade result while (b) and (e) shows the same but with uncertain regions depicted in red, (c) presents another example of a different cascade. (f) presents the final combination of all of the cascades.	100
4.7	Qualitative comparisons of uncertain regions of CE-SVM and ECE-SVM ensembles.	102
4.8	Updating the ensemble classifier. (a) show the original dataset, (b) present the trained CE-SVM classifier, (c) shows new data that arrived after the training, (d) shows an update to the current classifier.	104
4.9	Qualitative comparisons on selected 2D dataset.	113
4.10	Box plot of MCC scores obtained on 2D datasets.	114
4.11	Pareto plots of MCC scores against a) classification time b) training time.	114
4.12	Pareto plot of MCC scores against training time.	115

List of Tables

1.1	The list of proposed algorithms that are already published in international conferences and peer-reviewed journals.	5
2.1	Popular kernel functions.	16
2.2	Visualization of confusion matrix, where green cells represent correct classification and red cell represents different types of errors. . .	21
3.1	A summary of algorithms used in the proposed methods.	52
3.2	Summary of symbols used for algorithms description.	53
3.3	The most important aspects of SVMs optimized by evolutionary techniques.	85
4.1	The artificial created 2D datasets use in the analysis. IR stands for imbalance ratio, sum is the number of vectors in the dataset where Tr stands for the number of vectors in the training set and test is the number of vectors in the test set.	89
4.2	The benchmark datasets used in the analysis.	92
4.3	Test metrics on 2D datasets	106
4.4	Test rankings on 2D datasets	107
4.5	Results of test performed on benchmark datasets. The results are averaged score over all of the dataset.	111
4.6	Ranking of methods for classification performance on benchmark datasets.	112